Московский политехнический университет

Отчёт

по дисциплине «Программирование криптографических алгоритмов»

Выполнил студент
группы 241-372:
Сергеев Н.А.
Преподаватель:
Бутакова Н.Г.

Москва, 2026 г.

Содержание

Содержание 2
Аннотация 3
Введение 4
Общий интерфейс 4
Блок A: Шифры однозначной замены 9
1. Шифр Атбаш 9
Блок B: Шифры многозначной замены 14
5. Шифр Белазо 14
Блок C: Шифры блочной замены 20
8. Матричный шифр 20
Блок D: Шифры перестановки 28
10. Вертикальная перестановка 28
Блок E: Шифры гаммирования 37
13. Одноразовый блокнот Шеннона 37
Блок F: Поточные шифры 46
15. Поточный шифр A5/1 46
Блок G: Комбинационные шифры 55
19. Шифр AES 55
Блок H: Ассиметричные шифры 62
21. Шифр RSA 62
Блок I: алгоритмы цифровых подписей 70
24. Цифровая подпись RSA 70
Блок J: Стандарты цифровых подписей 77
27. Цифровая подпись ГОСТ Р 34.10-2012 77
Блок K: обмен ключами 83
28. Обмен ключами по алгоритму Диффи-Хеллмана 83
Заключение 93
Приложение А. Дополнительные реализации алгоритмов 94
17. Шифр МАГМА 95
20. Шифр Кузнечик 98
Приложение Б. Полная таблица расчёта A5/1 для слова ПЕРЕСДАЧА 102

Аннотация

Среда программирования: PyCharm

Язык программирования: python

Процедуры для запуска программ: скачать папку с программами, открыть файл main.py, запустить его, при необходимости выбрать интерпретатор, откроется меню со всеми шифрами, дальше действовать по требованиям программы.

Пословица — текст по варианту 20: Леопард не может изменить своих пятен.

Контрольный текст длиной 1000 символов:

Ночь опускалась на город медленно, будто нехотя. Фонари зажигались один за другим, и их жёлтый свет ложился на мокрый асфальт длинными дрожащими полосами. Ветер гнал по улице обрывки газет и сухие листья, и они шелестели под ногами редких прохожих, спешивших домой. Анна стояла у окна и смотрела, как дождь чертит по стеклу тонкие извилистые линии. Ей казалось, что где-то там, за этой пеленой, ещё живёт что-то важное, что она так и не успела понять за все эти годы. Внизу хлопнула дверь подъезда, и во двор вышел старик с собакой. Собака долго обнюхивала каждый куст, а старик терпеливо ждал, подняв воротник пальто и глядя в небо, где сквозь тучи пробивалась одинокая бледная звезда. Анна улыбнулась — впервые за весь день. Она налила себе чаю, села в кресло и раскрыла книгу, которую откладывала уже целую неделю. За окном шумел дождь, в комнате тикали часы, и мир постепенно становился на место. В такие минуты кажется, что всё ещё можно исправить, что до утра далеко и что самое главное в жизни — это просто дождаться рассвета.

Введение

Цель работы — изучить реализацию криптографических алгоритмов и проверить их работу на практических примерах. В отчёте рассматриваются шифрование и расшифрование, формирование и проверка цифровой подписи, а также согласование общего ключа.

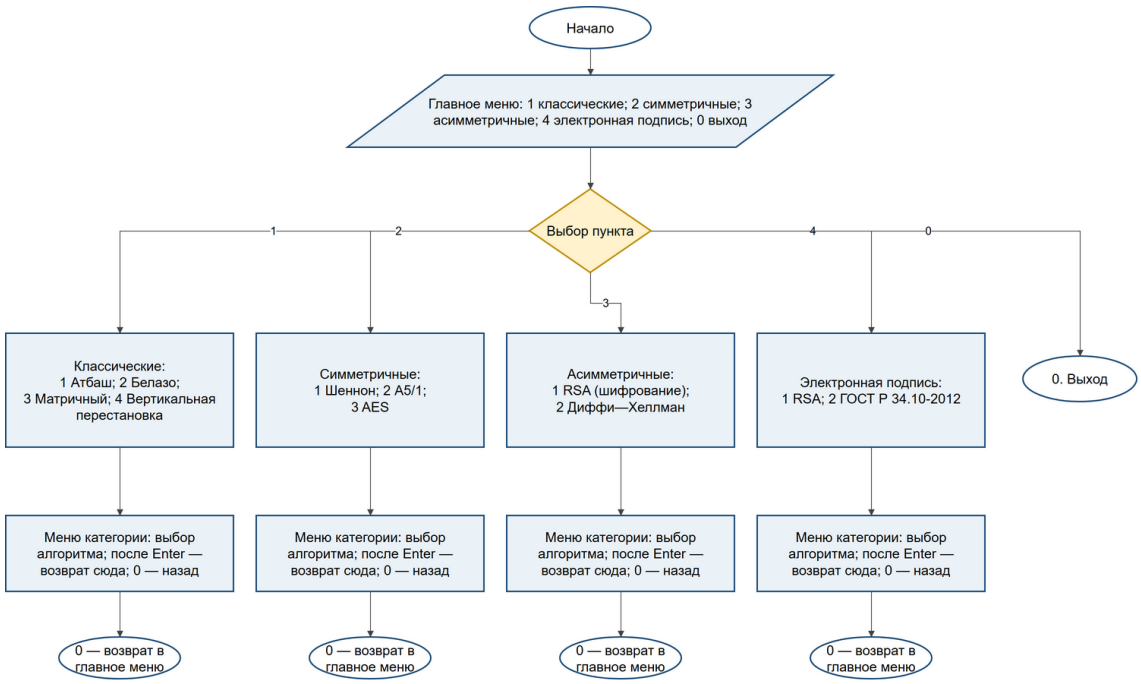
Для этого реализации объединены общим консольным интерфейсом. Для алгоритмов приведены описание, блок-схемы, код и результаты выполнения; параметры и сообщения вводятся согласно логике соответствующей программы.

Общий интерфейс

Данный интерфейс объединяет все программы с шифрами в одну программу.

Для создания приложения выбран консольный интерфейс. Пользователь выбирает шифр, после чего пользователю предоставляется выбор: зашифровать, расшифровать, выход из программы, а также в индивидуальном порядке ввести какие-то дополнительные данные.

Блок-схема:



Код программы:

```
import runpy
from pathlib import Path


BASE_DIR = Path(__file__).parent


def run_script(relative_path):
    path = BASE_DIR / relative_path
    try:
        print("\n" + "=" * 60)
        print(f"Запуск: {path.name}")
        print("=" * 60 + "\n")
        runpy.run_path(str(path), run_name="__main__")
    except SystemExit:
        # Возврат в меню, если вложенная лабораторная программа вызвала exit.
        pass
    except FileNotFoundError:
        print(f"\nФайл не найден:\n{path}")
    except Exception as exc:
        print(f"\nОшибка:\n{exc}")
    input("\nНажмите Enter для продолжения...")


def classic_menu():
    options = {
        "1": ("Атбаш", "Лабы Тема 1/Лаб 1_1/1.Шифр Атбаш.ру"),
        "2": ("Белазо", "Лабы Тема 1/Лаб 1_2/5.Шифр Белазо.ру"),
        "3": ("Матричный шифр", "Лабы Тема 1/Лаб 1_3/8.Матричный шифр.ру"),
        "4": ("Вертикальная перестановка", "Лабы Тема 1/Лаб 1_4/10.Вертикальная перестановка.ру"),
    }
    while True:
        print("\n" + "=" * 60)
        print("КЛАССИЧЕСКИЕ ШИФРЫ")
        print("=" * 60)
        for number, (name, _) in options.items():
            print(f"{number} - {name}")
        print("0 - Назад")
        choice = input("\nВаш выбор: ").strip()
        if choice == "0":
            return
        if choice in options:
            run_script(options[choice][1])
        else:
            print("Неверный выбор")


def symmetric_menu():
```

```
    options = {
        "1": ("Одноразовый блокнот Шеннона", "Лабы Тема 1/Лаб 1_5/13.Одноразовый блокнот К.Шеннона.py"),
        "2": ("A5/1", "Лабы Тема 1/Лаб 1_6/15.A5_1.py"),
        "3": ("AES", "Лабы Тема 1/Лаб 1_7/19.AES.py"),
    }
    while True:
        print("\n" + "=" * 60)
        print("СИММЕТРИЧНЫЕ АЛГОРИТМЫ")
        print("=" * 60)
        for number, (name, _) in options.items():
            print(f"{number} - {name}")
        print("0 - Назад")
        choice = input("\nВаш выбор: ").strip()
        if choice == "0":
            return
        if choice in options:
            run_script(options[choice][1])
        else:
            print("Неверный выбор")


def asymmetric_menu():
    options = {
        "1": ("RSA (шифрование)", "Лабы Тема 2/Лаб 2_1/21-23.RSA_Elgamal_ECC.py"),
        "2": ("Диффи–Хеллман", "Лабы Тема 4/Лаб 4_1/28.Диффи-Хеллман.py"),
    }
    while True:
        print("\n" + "=" * 60)
        print("АСИММЕТРИЧНЫЕ АЛГОРИТМЫ И ОБМЕН КЛЮЧАМИ")
        print("=" * 60)
        for number, (name, _) in options.items():
            print(f"{number} - {name}")
        print("0 - Назад")
        choice = input("\nВаш выбор: ").strip()
        if choice == "0":
            return
        if choice in options:
            run_script(options[choice][1])
        else:
            print("Неверный выбор")


def signature_menu():
    options = {
        "1": ("Цифровая подпись RSA", "Лабы Тема 3/Лаб 3_1/24.RSA.py"),
        "2": ("ГОСТ Р 34.10-2012", "Лабы Тема 3/Лаб 3_2/27.ГОСТ_Р_34.10-2012.py"),
    }
    while True:
        print("\n" + "=" * 60)
        print("ЭЛЕКТРОННАЯ ПОДПИСЬ")
```

```
        print("=" * 60)
        for number, (name, _) in options.items():
            print(f"{number} - {name}")
        print("0 - Назад")
        choice = input("\nВаш выбор: ").strip()
        if choice == "0":
            return
        if choice in options:
            run_script(options[choice][1])
        else:
            print("Неверный выбор")


def main():
    menus = {
        "1": ("Классические шифры", classic_menu),
        "2": ("Симметричные алгоритмы", symmetric_menu),
        "3": ("Асимметричные алгоритмы и обмен ключами", asymmetric_menu),
        "4": ("Электронная подпись", signature_menu),
    }
    while True:
        print("\n" + "=" * 60)
        print("ГЛАВНОЕ МЕНЮ КРИПТОГРАФИЧЕСКИХ АЛГОРИТМОВ")
        print("=" * 60)
        for number, (name, _) in menus.items():
            print(f"{number} - {name}")
        print("0 - Выход")
        choice = input("\nВаш выбор: ").strip()
        if choice == "0":
            print("\nДо свидания!")
            break
        if choice in menus:
            menus[choice][1]()
        else:
            print("\nНеверный выбор")


if __name__ == "__main__":
    main()
```

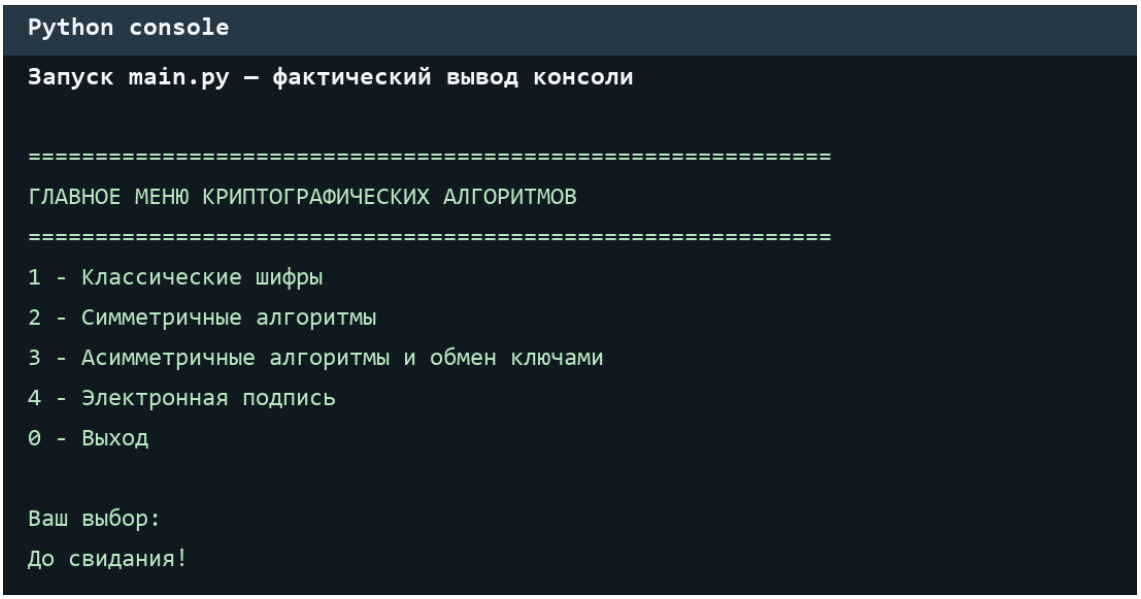


Рисунок 1. Общий интерфейс.

Блок А: Шифры однозначной замены

1. Шифр Атбаш

Шифр АТБАШ - шифр простой замены. Алгоритм представляет собой замену i-ой буквы алфавита на n-i+1 букву алфавита, где n – размерность алфавита.

Блок-схема:

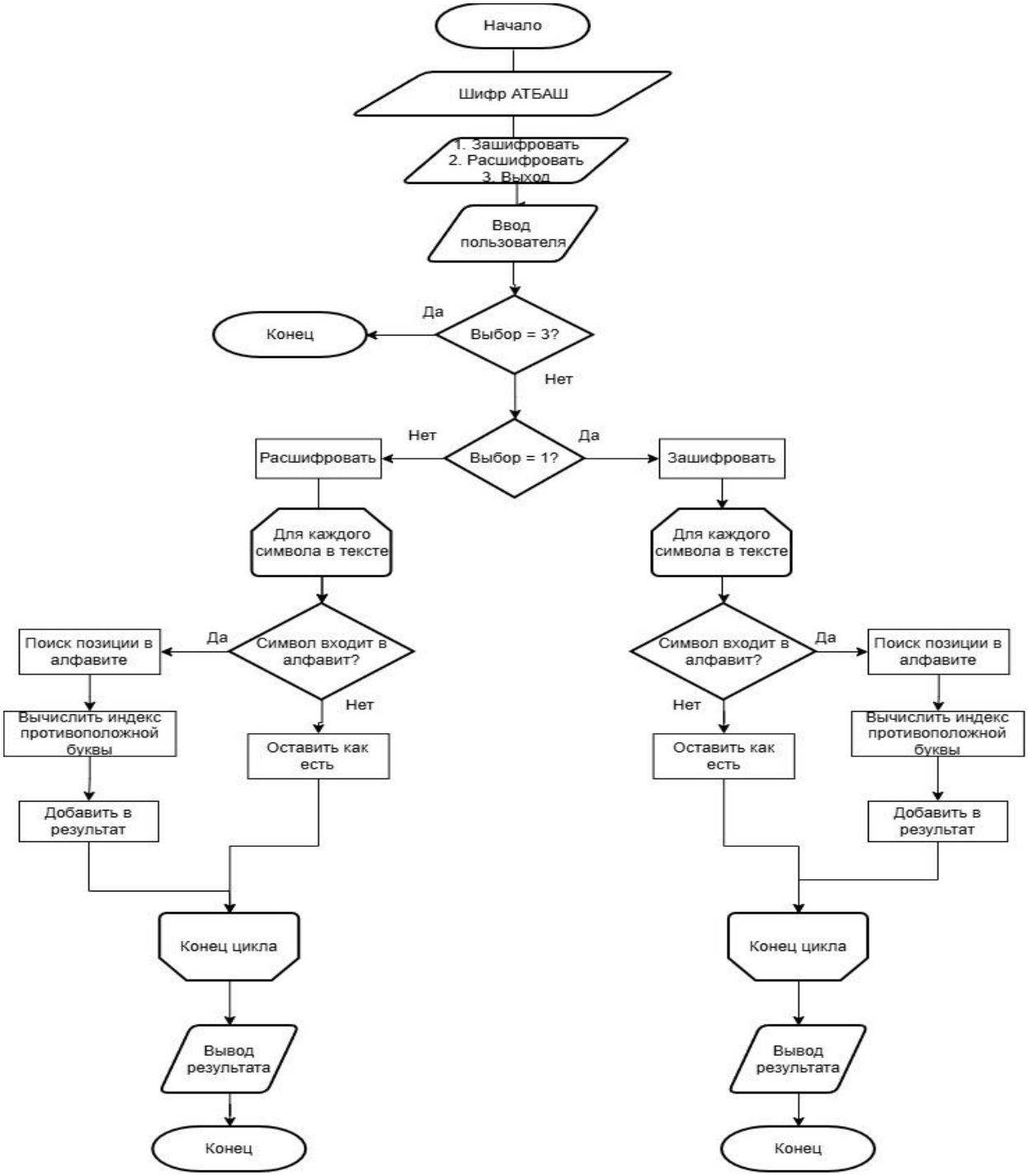


Рисунок 4. Блок-схема АТБАШ

Код программы:

```
"""
ШИФР АТБАШ
Русский алфавит (33 буквы с Ъ)
Первая буква алфавита заменяется последней, вторая — предпоследней и т.д.
"""

ALPHABET = 'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'
N = len(ALPHABET)

char_to_num = {char: i for i, char in enumerate(ALPHABET)}
num_to_char = {i: char for i, char in enumerate(ALPHABET)}

def encrypt(text):
    text = text.upper()
    result = []

    for char in text:
        if char == ' ':
            result.append(' ')
        elif char in char_to_num:
            m = char_to_num[char]
            L = (N - 1 - m) % N
            result.append(num_to_char[L])
        else:
            result.append(char)

    return ''.join(result)

def decrypt(ciphertext):
    return encrypt(ciphertext)  # Атбаш симметричен

while True:
    print("\n" + "=" * 50)
    print("ШИФР АТБАШ")
    print("=" * 50)
    print("1. Зашифровать")
    print("2. Расшифровать")
    print("3. Выход")

    choice = input("\nВыберите действие: ")

    if choice == "1":
        text = input("Введите текст для шифрования: ")
        encrypted = encrypt(text)

        print("\nРезультат:")
        print(encrypted)

    elif choice == "2":
        text = input("Введите текст для расшифрования: ")
        decrypted = decrypt(text)

        print("\nРезультат:")
        print(decrypted)

    elif choice == "3":
        print("Выход из программы...")
        break

    else:
        print("Неверный выбор!")
```

Результат работы программы:

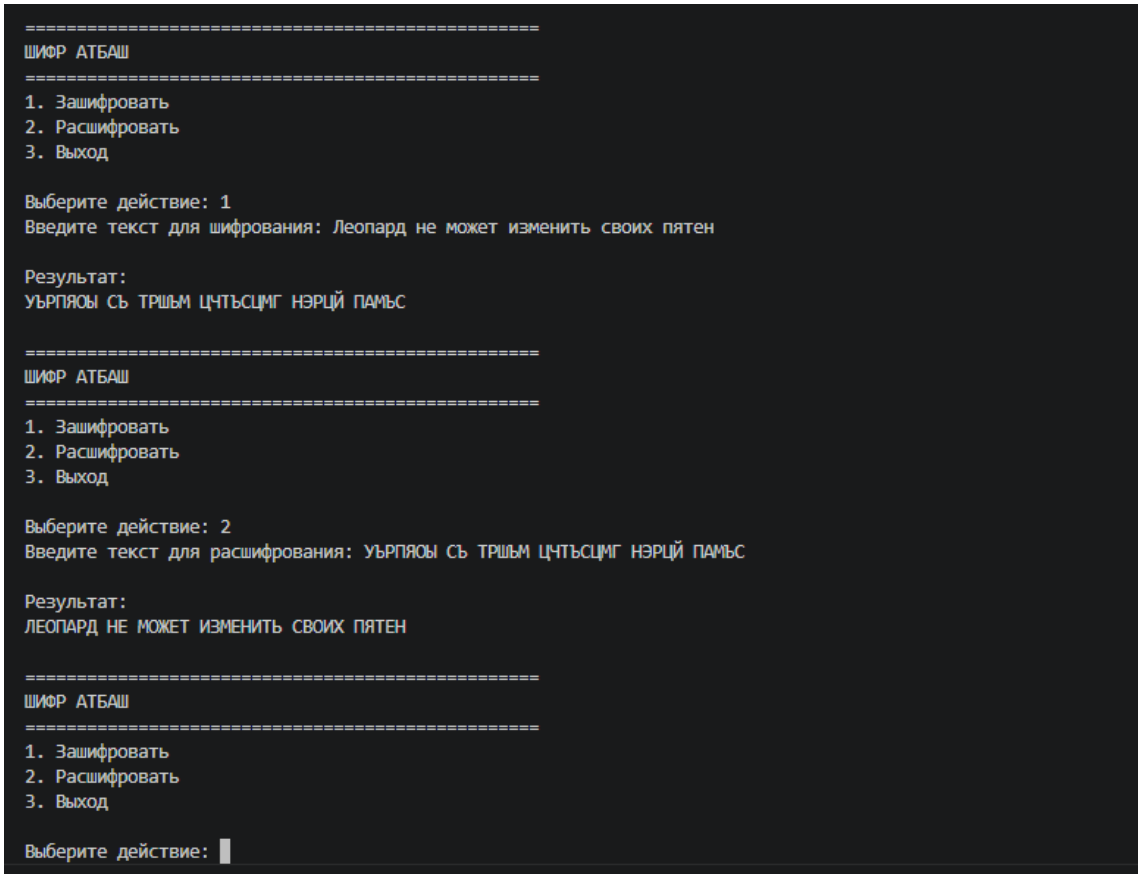


Рисунок 5. Результат выполнения шифра АТБАШ

Тест на 1000 символов:

```
================================================
ШИФР АТБАШ
================================================
1. Зашифровать
2. Расшифровать
3. Выход

Выберите действие: 1
Введите текст для шифрования: Ночь опускалась на город медленно, будто нехотя. Фонари зажигались один за другим, и их
 жёлтый свет ложился на мокрый асфальт длинными дрожащими полосами. Ветер гнал по улице обрывки газет и сухие листья,
 и они шелестели под ногами редких прохожих, спешивших домой. Анна стояла у окна и смотрела, как дождь чертит по стек
лу тонкие извилистые линии. Ей казалось, что где-то там, за этой пеленой, ещё живёт что-то важное, что она так и не у
спела понять за все эти годы. Внизу хлопнула дверь подъезда, и во двор вышел старик с собакой. Собака долго обнюхивал
а каждый куст, а старик терпеливо ждал, подняв воротник пальто и глядя в небо, где сквозь тучи пробивалась одинокая б
ледная звезда. Анна улыбнулась — впервые за весь день. Она налила себе чаю, села в кресло и раскрыла книгу, которую о
ткладывала уже целую неделю. За окном шумел дождь, в комнате тикали часы, и мир постепенно становился на место. В так
ие минуты кажется, что всё ещё можно исправить, что до утра далеко и что самое главное в жизни — это просто дождаться
 рассвета.

Результат:
СРЗГ РПЛНФЯУЯНГ СЯ ЬРОРЫ ТЪЫУЪССР, ЮЛЫМР СЪЙРМА. КРСЯОЦ ЧЯШЦЬЯУЦНГ РЫЦС ЧЯ ЫОЛЬЦТ, Ц ЦЙ ШЩУМДХ НЭЪМ УРШЦУНА СЯ ТРФОДХ
 ЯНКЯУГМ ЫУЦССДТЦ ЫОРШЯЁЦТЦ ПРУРНЯТЦ. ЭЪМЬО ЬСЯУ ПР ЛУЦИЬ РЮОДЭФЦ ЬЯЧЬМ Ц НЛЙЦЬ УЦНМГА, Ц РСЦ ЖЬУЪНМЬУЦ ПРЫ СРЬЯТЦ ОЬ
ЫФЦЙ ПОРЙРШЦЙ, НПЪЖЦЭЖЦЙ ЫРТРХ. ЯССЯ НМРАУЯ Л РФСЯ Ц НТРМОЬУЯ, ФЯФ ЫРШЫГ ЗЪОМЦМ ПР НМЬФУЛ МРСФЦЬ ЦЧЭЦУЦНМДЬ УЦСЦЦ. ЬХ
 ФЯЧЯУРНГ, ЗМР ЬЫЬ-МР МЯТ, ЧЯ ВМРХ ПЬУЪСРХ, ЬЁЩ ШЦЭЩМ ЗМР-МР ЭЯШСРЪ, ЗМР РСЯ МЯФ Ц СЬ ЛНПЬУЯ ПРСАМГ ЧЯ ЭНЬ ВМЦ ЬРЫД.
ЭСЦЧЛ ЙУРПСЛУЯ ЫЭЪОГ ПРЫЕЪЧЫЯ, Ц ЭР ЫЭРО ЭДЖЬУ НМЯОЦФ Н НРЮЯФРХ. НРЮЯФЯ ЫРУЬР РЮСБЙЦЭЯУЯ ФЯШЫДХ ФЛНМ, Я НМЯОЦФ МЬОПЪУ
ЦЭР ШЫЯУ, ПРЫСАЭ ЭРОРМСЦФ ПЯУГМР Ц ЬУАЫА Э СЪЮР, ЬЫЬ НФЭРЧГ МЛЗЦ ПОРЮЦЭЯУЯНГ РЫЦСРФЯА ЮУЪЫСЯА ЧЭЪЧЫЯ. ЯССЯ ЛУДЮСЛУЯНГ
 — ЭПЬОЭДЬ ЧЯ ЭЬНГ ЫЬСГ. РСЯ СЯУЦУЯ НЬЮЬ ЗЯБ, НЬУЯ Э ФОЬНУР Ц ОЯНФОДУЯ ФСЦЬЛ, ФРМРОЛБ РМФУЯЫДЭЯУЯ ЛШЬ ИЪУЛБ СЪЫЬУБ. Ч
Я РФСРТ ЖЛТЬУ ЫРШЫГ, Э ФРТСЯМЬ МЦФЯУЦ ЗЯНД, Ц ТЦО ПРНМЬПЬССР НМЯСРЭЦУНА СЯ ТЬНМР. Э МЯФЦЬ ТЦСЛМД ФЯШЬМНА, ЗМР ЭНЩ ЪЁЩ
 ТРШСР ЦНПОЯЭЦМГ, ЗМР ЫР ЛМОЯ ЫЯУЪФР Ц ЗМР НЯТРЪ ЬУЯЭСРЪ Э ШЦЧСЦ — ВМР ПОРНМР ЫРШЫЯМГНА ОЯННЭЬМЯ.
```

```
================================================
ШИФР АТБАШ
================================================
1. Зашифровать
2. Расшифровать
3. Выход

Выберите действие: 2
Введите текст для расшифрования: СРЗГ РПЛНФЯУЯНГ СЯ ЬРОРЫ ТЪЫУЪССР, ЮЛЫМР СЪЙРМА. КРСЯОЦ ЧЯШЦЬЯУЦНГ РЫЦС ЧЯ ЫОЛЬЦТ, Ц
 ЦЙ ШЩУМДХ НЭЪМ УРШЦУНА СЯ ТРФОДХ ЯНКЯУГМ ЫУЦССДТЦ ЫОРШЯЁЦТЦ ПРУРНЯТЦ. ЭЪМЬО ЬСЯУ ПР ЛУЦИЬ РЮОДЭФЦ ЬЯЧЬМ Ц НЛЙЦЬ УЦНМ
ГА, Ц РСЦ ЖЬУЪНМЬУЦ ПРЫ СРЬЯТЦ ОЬЫФЦЙ ПОРЙРШЦЙ, НПЪЖЦЭЖЦЙ ЫРТРХ. ЯССЯ НМРАУЯ Л РФСЯ Ц НТРМОЬУЯ, ФЯФ ЫРШЫГ ЗЪОМЦМ ПР Н
МЬФУЛ МРСФЦЬ ЦЧЭЦУЦНМДЬ УЦСЦЦ. ЬХ ФЯЧЯУРНГ, ЗМР ЬЫЬ-МР МЯТ, ЧЯ ВМРХ ПЬУЪСРХ, ЬЁЩ ШЦЭЩМ ЗМР-МР ЭЯШСРЪ, ЗМР РСЯ МЯФ Ц С
Ъ ЛНПЬУЯ ПРСАМГ ЧЯ ЭНЬ ВМЦ ЬРЫД. ЭСЦЧЛ ЙУРПСЛУЯ ЫЭЪОГ ПРЫЕЪЧЫЯ, Ц ЭР ЫЭРО ЭДЖЬУ НМЯОЦФ Н НРЮЯФРХ. НРЮЯФЯ ЫРУЬР РЮСБЙЦ
ЭЯУЯ ФЯШЫДХ ФЛНМ, Я НМЯОЦФ МЬОПЪУЦЭР ШЫЯУ, ПРЫСАЭ ЭРОРМСЦФ ПЯУГМР Ц ЬУАЫА Э СЪЮР, ЬЫЬ НФЭРЧГ МЛЗЦ ПОРЮЦЭЯУЯНГ РЫЦСРФЯ
А ЮУЪЫСЯА ЧЭЪЧЫЯ. ЯССЯ ЛУДЮСЛУЯНГ — ЭПЬОЭДЬ ЧЯ ЭЬНГ ЫЬСГ. РСЯ СЯУЦУЯ НЬЮЬ ЗЯБ, НЬУЯ Э ФОЬНУР Ц ОЯНФОДУЯ ФСЦЬЛ, ФРМРОЛ
Б РМФУЯЫДЭЯУЯ ЛШЬ ИЪУЛБ СЪЫЬУБ. ЧЯ РФСРТ ЖЛТЬУ ЫРШЫГ, Э ФРТСЯМЬ МЦФЯУЦ ЗЯНД, Ц ТЦО ПРНМЬПЬССР НМЯСРЭЦУНА СЯ ТЬНМР. Э
МЯФЦЬ ТЦСЛМД ФЯШЬМНА, ЗМР ЭНЩ ЪЁЩ ТРШСР ЦНПОЯЭЦМГ, ЗМР ЫР ЛМОЯ ЫЯУЪФР Ц ЗМР НЯТРЪ ЬУЯЭСРЪ Э ШЦЧСЦ — ВМР ПОРНМР ЫРШЫЯМ
ГНА ОЯННЭЬМЯ.

Результат:
НОЧЬ ОПУСКАЛАСЬ НА ГОРОД МЕДЛЕННО, БУДТО НЕХОТЯ. ФОНАРИ ЗАЖИГАЛИСЬ ОДИН ЗА ДРУГИМ, И ИХ ЖЁЛТЫЙ СВЕТ ЛОЖИЛСЯ НА МОКРЫЙ
 АСФАЛЬТ ДЛИННЫМИ ДРОЖАЩИМИ ПОЛОСАМИ. ВЕТЕР ГНАЛ ПО УЛИЦЕ ОБРЫВКИ ГАЗЕТ И СУХИЕ ЛИСТЬЯ, И ОНИ ШЕЛЕСТЕЛИ ПОД НОГАМИ РЕ
ДКИХ ПРОХОЖИХ, СПЕШИВШИХ ДОМОЙ. АННА СТОЯЛА У ОКНА И СМОТРЕЛА, КАК ДОЖДЬ ЧЕРТИТ ПО СТЕКЛУ ТОНКИЕ ИЗВИЛИСТЫЕ ЛИНИИ. ЕЙ
 КАЗАЛОСЬ, ЧТО ГДЕ-ТО ТАМ, ЗА ЭТОЙ ПЕЛЕНОЙ, ЕЩЁ ЖИВЁТ ЧТО-ТО ВАЖНОЕ, ЧТО ОНА ТАК И НЕ УСПЕЛА ПОНЯТЬ ЗА ВСЕ ЭТИ ГОДЫ.
ВНИЗУ ХЛОПНУЛА ДВЕРЬ ПОДЪЕЗДА, И ВО ДВОР ВЫШЕЛ СТАРИК С СОБАКОЙ. СОБАКА ДОЛГО ОБНЮХИВАЛА КАЖДЫЙ КУСТ, А СТАРИК ТЕРПЕЛ
ИВО ЖДАЛ, ПОДНЯВ ВОРОТНИК ПАЛЬТО И ГЛЯДЯ В НЕБО, ГДЕ СКВОЗЬ ТУЧИ ПРОБИВАЛАСЬ ОДИНОКАЯ БЛЕДНАЯ ЗВЕЗДА. АННА УЛЫБНУЛАСЬ
 — ВПЕРВЫЕ ЗА ВЕСЬ ДЕНЬ. ОНА НАЛИЛА СЕБЕ ЧАЮ, СЕЛА В КРЕСЛО И РАСКРЫЛА КНИГУ, КОТОРУЮ ОТКЛАДЫВАЛА УЖЕ ЦЕЛУЮ НЕДЕЛЮ. З
А ОКНОМ ШУМЕЛ ДОЖДЬ, В КОМНАТЕ ТИКАЛИ ЧАСЫ, И МИР ПОСТЕПЕННО СТАНОВИЛСЯ НА МЕСТО. В ТАКИЕ МИНУТЫ КАЖЕТСЯ, ЧТО ВСЁ ЕЩЁ
 МОЖНО ИСПРАВИТЬ, ЧТО ДО УТРА ДАЛЕКО И ЧТО САМОЕ ГЛАВНОЕ В ЖИЗНИ — ЭТО ПРОСТО ДОЖДАТЬСЯ РАССВЕТА.

================================================
ШИФР АТБАШ
================================================
1. Зашифровать
2. Расшифровать
3. Выход

Выберите действие:
```

Рисунок 7. Шифрование АТБАШ на 1000 символов

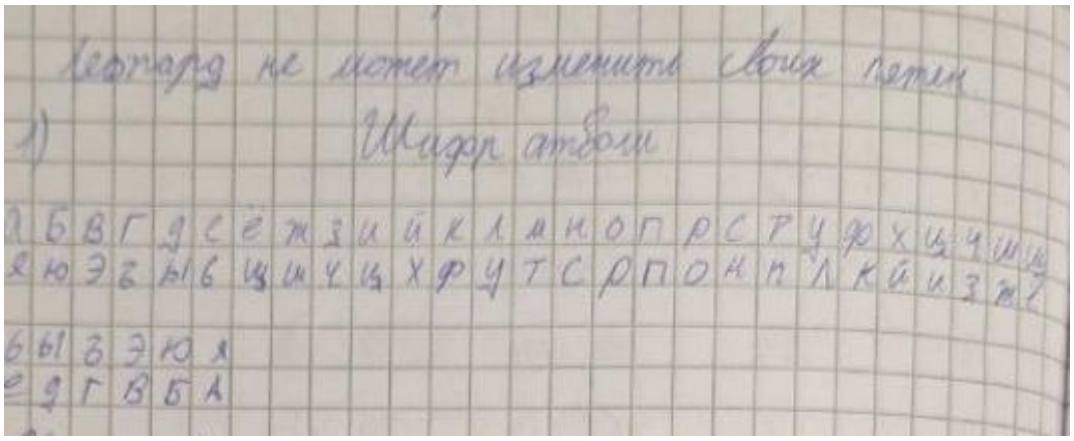


Рисунок 8. Ручной тест АТБАШ

Блок B: Шифры многозначной замены

5. Шифр Белазо

Шифр Белазо — это шифр подстановки. Он является усовершенствованием шифра Цезаря и предшественником шифра Виженера. В отличие от шифра Цезаря, где сдвиг постоянен для всего сообщения, шифр Белазо использует ключевое слово, которое определяет переменный сдвиг для каждого символа.

Обозначения:

P - открытый текст

C - зашифрованный текст

K - ключ

m - мощность алфавита

i - позиция символа в тексте (начало 0)

$C_i = (P_i + K \{i \bmod |K|\}) \bmod m$ - формула для шифра Белазо

Блок-схема:

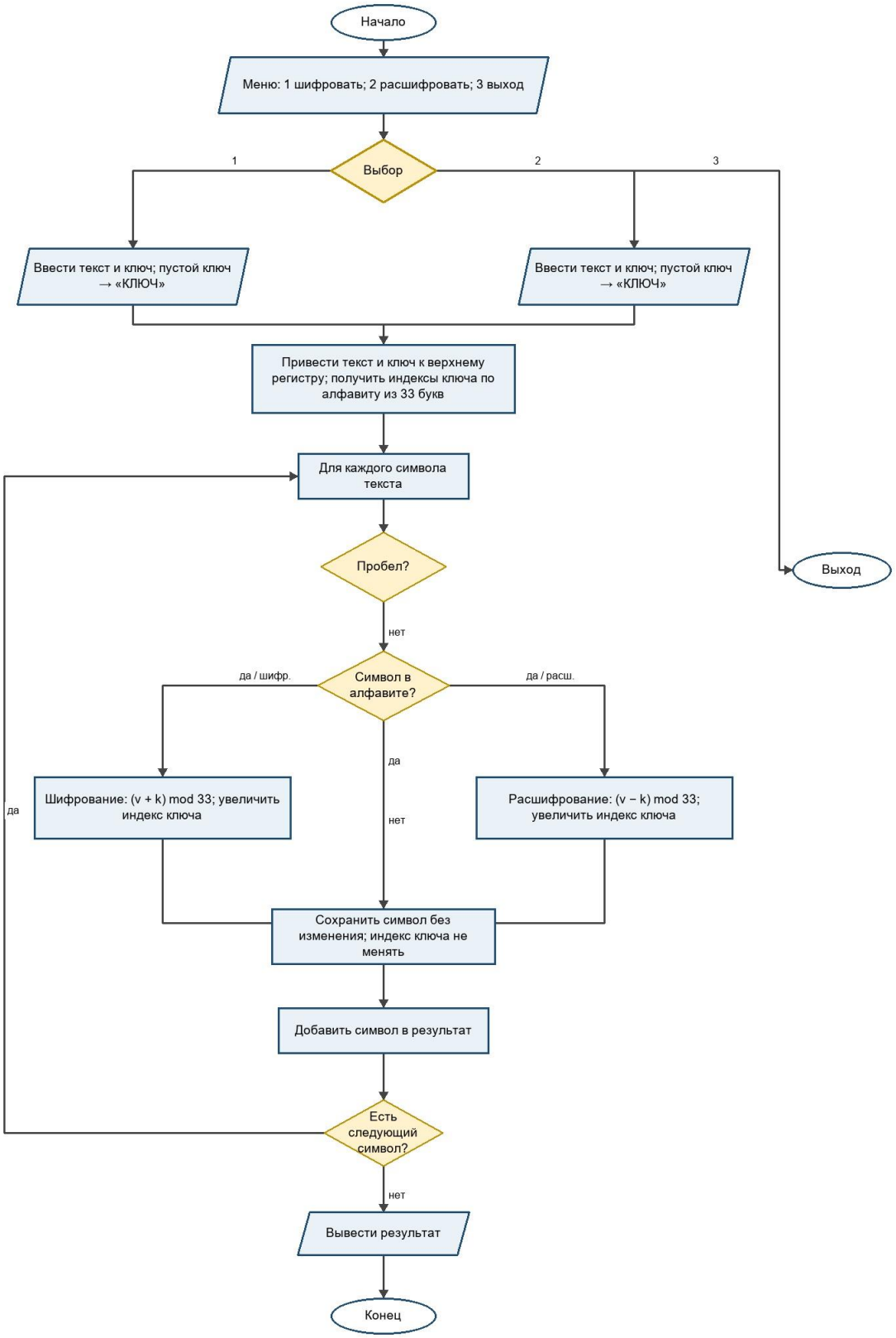


Рисунок 9. Блок-схема шифра Белазо.

Код программы:

```
# -*- coding: utf-8 -*-
"""
ПРОСТОЙ ШИФР БЕЛАЗО
"""

ALPHABET = 'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'
N = 33

char_to_num = {char: i for i, char in enumerate(ALPHABET)}
num_to_char = {i: char for i, char in enumerate(ALPHABET)}

KEYWORD = "КЛЮЧ"

def get_key_nums(keyword):
    return [char_to_num[char] for char in keyword.upper() if char in char_to_num]

def encrypt(text, keyword=KEYWORD):
    text = text.upper()
    key_nums = get_key_nums(keyword)
    key_length = len(key_nums)

    result = []
    key_idx = 0

    for char in text:
        if char == ' ':
            result.append(' ')
        elif char in char_to_num:
            vi = char_to_num[char]
            ki = key_nums[key_idx % key_length]
            wi = (vi + ki) % N
            result.append(num_to_char[wi])
            key_idx += 1
        else:
            result.append(char)

    return ''.join(result)

def decrypt(ciphertext, keyword=KEYWORD):
    ciphertext = ciphertext.upper()
    key_nums = get_key_nums(keyword)
    key_length = len(key_nums)

    result = []
    key_idx = 0

    for char in ciphertext:
        if char == ' ':
            result.append(' ')
        elif char in char_to_num:
            wi = char_to_num[char]
            ki = key_nums[key_idx % key_length]
            vi = (wi - ki) % N
            result.append(num_to_char[vi])
            key_idx += 1
        else:
            result.append(char)

    return ''.join(result)

while True:
    print("\n" + "=" * 50)
    print("ПРОСТОЙ ШИФР БЕЛАЗО")
    print("=" * 50)
    print("1. Зашифровать")
    print("2. Расшифровать")
    print("3. Выход")

    choice = input("\nВыберите действие: ")

    if choice == "1":
        text = input("Введите текст для шифрования: ")
        key = input("Введите ключ (Enter = КЛЮЧ): ").strip()

        if not key:
            key = KEYWORD

        encrypted = encrypt(text, key)

        print("\nРезультат шифрования:")
        print(encrypted)

    elif choice == "2":
        text = input("Введите текст для расшифрования: ")
        key = input("Введите ключ (Enter = КЛЮЧ): ").strip()

        if not key:
            key = KEYWORD

        decrypted = decrypt(text, key)

        print("\nРезультат расшифрования:")
        print(decrypted)

    elif choice == "3":
        print("Выход из программы...")
        break

    else:
        print("Неверный выбор!")
```

Результат работы программы:

```
 *  Выполнение задачи: C:\Users\NIKITA\AppData\Local\Microsoft\WindowsApps\python3.11.exe 'c:\Users\NIKITA\Desktop\Папочки\Лабы\КРИПТО\Лабы Тема 1\Лаб 1_2\5.Шифр Белазо.py'


 ================================================
 ПРОСТОЙ ШИФР БЕЛАЗО
 ================================================
 1. Зашифровать
 2. Расшифровать
 3. Выход

 Выберите действие: 1
 Введите текст для шифрования: Леопард не может изменить свохи пятен
 Введите ключ (Enter = КЛЮЧ):

 Результат шифрования:
 ЦРМЖКЬВ ЕП ШМЮПЮ ЖЯЧРЛАЭЭ ПЩЩБЖ ЖЙЮГЕ
```

```
================================================
ПРОСТОЙ ШИФР БЕЛАЗО
================================================
1. Зашифровать
2. Расшифровать
3. Выход

Выберите действие: 2
Введите текст для расшифрования: ЦРМЖКЬВ ЕП ШМЮПЮ ЖЯЧРЛАЭЭ ПЩЩБЖ ЖЙЮГЕ
Введите ключ (Enter = КЛЮЧ):

Результат расшифрования:
ЛЕОПАРД НЕ МОЖЕТ ИЗМЕНИТЬ СВОХИ ПЯТЕН

================================================
ПРОСТОЙ ШИФР БЕЛАЗО
================================================
1. Зашифровать
2. Расшифровать
3. Выход

Выберите действие:
```

Рисунок 10. Результат работы шифра Белазо.

Тест на 1000 знаков:

```
================================================
ПРОСТОЙ ШИФР БЕЛАЗО
================================================
1. Зашифровать
2. Расшифровать
3. Выход

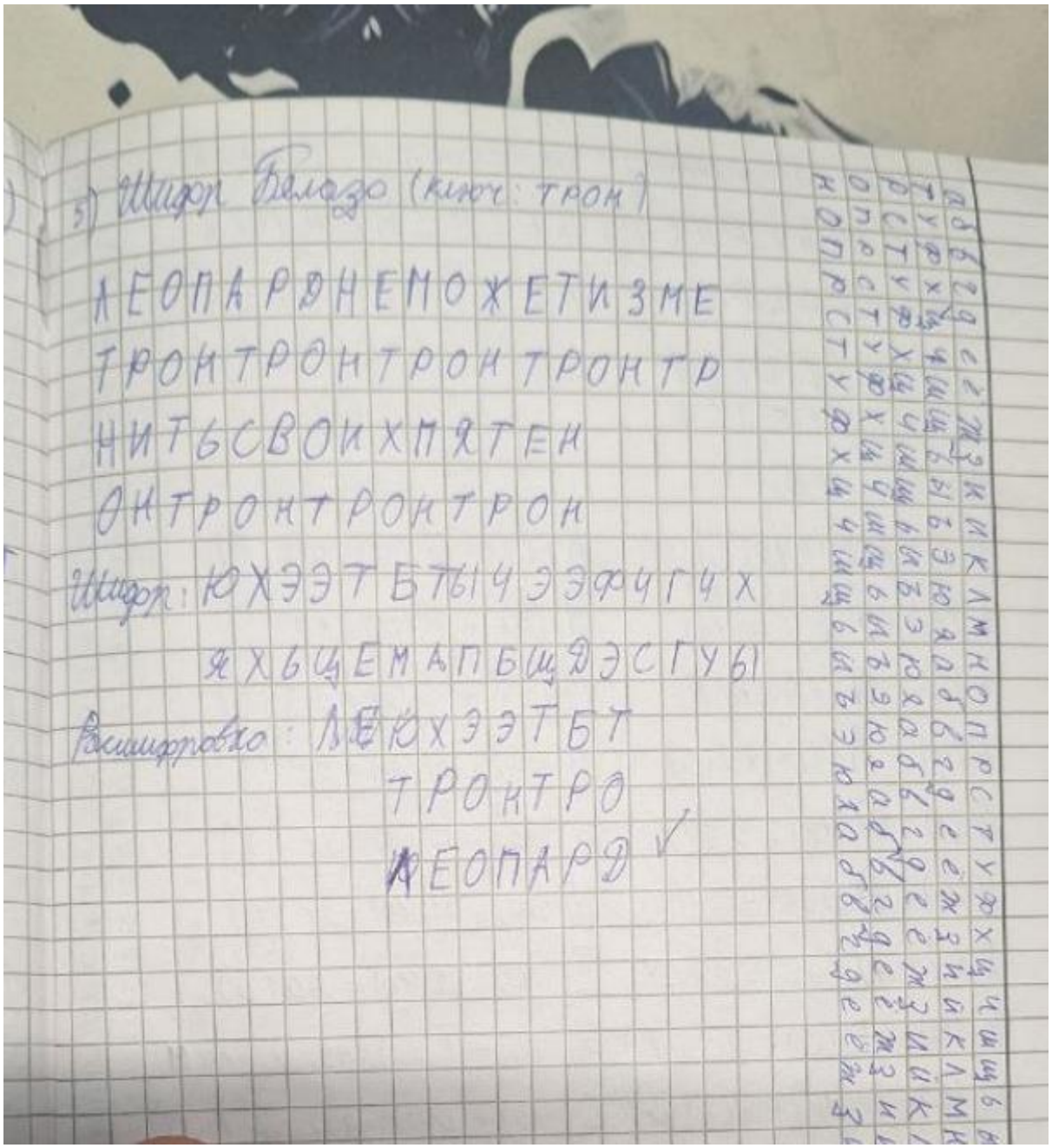
Выберите действие: 1
Введите текст для шифрования: Ночь опускалась на город медленно, будто нехотя. Фонари зажигались один за другим, и их
 жёлтый свет ложился на мокрый асфальт длинными дрожащими полосами. Ветер гнал по улице обрывки газет и сухие листья,
 и они шелестели под ногами редких прохожих, спешивших домой. Анна стояла у окна и смотрела, как дождь чертит по стек
лу тонкие извилистые линии. Ей казалось, что где-то там, за этой пеленой, ещё живёт что-то важное, что она так и не у
спела понять за все эти годы. Внизу хлопнула дверь подъезда, и во двор вышел старик с собакой. Собака долго обнюхивал
а каждый куст, а старик терпеливо ждал, подняв воротник пальто и глядя в небо, где сквозь тучи пробивалась одинокая б
ледная звезда. Анна улыбнулась — впервые за весь день. Она налила себе чаю, села в кресло и раскрыла книгу, которую о
ткладывала уже целую неделю. За окном шумел дождь, в комнате тикали часы, и мир постепенно становился на место. В так
ие минуты кажется, что всё ещё можно исправить, что до утра далеко и что самое главное в жизни — это просто дождаться
 рассвета.
Введите ключ (Enter = КЛЮЧ):

Результат шифрования:
ШЪХУ ЩЫСИХЛЙЧЬЗ ЛЧ НЬОЁО ШГЫЦРЛЕЩ, МСЫЭЪ ЛЬАЪРЦ. ЯЪЛЧЫФ ЁЧСФБЧЦФПУ ЩПЖЕ ТЛ ВЗЮОЖД, У ФУ ЮРЧРТФ ЭАЬЭ ЧМЮУЧПЦ ШЛ КЁХЬЩБ
 КЭТЧЦЗР ЫЦФЛЕЁШЖ ЫЫЪЕЧДФКА ЪЬЙЁЬЛКА. МРРЬЫ ОЛЧЦ ЫМ КЦФФЬ ЩМОТМЦЖ ЪКУГЙ У ЭСМУР ЙАЫЮЪЦ, У ЪЛА ГРЙЬЬЮГГУ ЫМЫ ШЪБЧЧФ ОЬ
ОЦЖМ ЪЬММЩТЖМ, ЫЫГПУНЦАА ПМДЩХ. ЮЕШЛ ПЙЩКЙЧ Ю ЪИЕК Ф ПДЩЮОЬЦЛ, ИЧХ ПМЮОЗ ХЬЫЮЖЙ ЪЪ ПЙПЦЙК ЭЪЛВУР ЖЯМФЙАЫЮЩЬ ЦФЛАУ. РЗ
 ВКУЮГЩЭЪ, ОЭЬ БЫП-ЮМ ЙКШ, ЁЧ ЗЮМБ ЪРЙЫШЪЗ, ЬДС ЕАМСР ОЭЬ-РЁ МЛЕЕЩР, ХЙЩ ЪЛЧ ЭЛИ А ШР СИЪРЙЧ ЪЪЛЦЭЗ ЁЧ МЭГ ФЭФ БЁОЖ.
АЕУУС МЦЪНЕЮЧЮ ЫМРОУ ЪЪВСПУВЧ, У НМ ЫМЬО ЩЁДГГ ЫЮЮЗУЦ П ИЩМЮВЩХ. ПЁЛЛИЧ ОЬЙЪЩ ЪЯЕИБЖЩКЧЮ ВКТВТФ ЦСИЭ, Л ПЙКЬЖВ ЭРОЖПЧ
ЖЩЩ ТВЧЦ, ЫМЫШКА ЩЦЬМЙШФИ ЖКЧЬЙЩ Ф БГЙПЭ Щ ШРЯЁ, НПГ ИХНМЯЖ ЮСОУ ЫЮЁЛФАЧЦЛПУ ЩПЖЕЩЦЮЦ ЛЧГЫШЛЭ ЯМРЁЫК. ЛЛЕК ЯЙТЛЩСГКЭЬ
 — ЩЪРОЩЁР ЁЧ МРПУ ОРЛУ. ЩЦЮ ЕКЧЖГК ЭГШП ГЮХ, ЬРЙЧ М ЦОЬЬЧМ А ЫЛПВЫЖЙЧ ХЩЖЬЮ, ЦМЙЩЬСХ ЩЮИГКПЩЦКЧЮ КСР ФЬЦЯЬ ЕППГГИ. У
Ю ЁХЩМД ГЯКЬЦ ПМЮОЗ, А ВЩШЛЧЭР РАХЛЙА ВЛПТ, У ШЖЗ ЪЪПЙПЫГЕШЬ ПЙКЩМЩУЧПЦ ШЛ КЬЬЮМ. Щ ЭЛИАП ШЖЕЮЮЩ ВКТГЙЬК, ХЙЩ НПЭ ПЕД
 ДЩТЛЁ УЭНЗКНЖЙЖ, ГРЁ ОЬ СЙЫЛ ВЧЦРИЁ У ГРЁ ЬЛКЁП ОЙЧМЩМЬ М ТЖЯШФ — ЫЙЩ ЫЮЁЬЮМ ЫЩТВЧЭЗПЦ ЫЛПИМРРЧ.
```

```
Выберите действие: 2
Введите текст для расшифрования: ШЪХУ ЩЫСИХЛЙЧЬЗ ЛЧ НЬОЁО ШГЫЦРЛЕЩ, МСЫЭЪ ЛЬАЪРЦ. ЯЪЛЧЫФ ЁЧСФБЧЦФПУ ЩПЖЕ ТЛ ВЗЮОЖД, У
 ФУ ЮРЧРТФ ЭАЬЭ ЧМЮУЧПЦ ШЛ КЁХЬЩБ КЭТЧЦЗР ЫЦФЛЕЁШЖ ЫЫЪЕЧДФКА ЪЬЙЁЬЛКА. МРРЬЫ ОЛЧЦ ЫМ КЦФФЬ ЩМОТМЦЖ ЪКУГЙ У ЭСМУР ЙАЫЮ
ЪЦ, У ЪЛА ГРЙЬЬЮГГУ ЫМЫ ШЪБЧЧФ ОЬОЦЖМ ЪЬММЩТЖМ, ЫЫГПУНЦАА ПМДЩХ. ЮЕШЛ ПЙЩКЙЧ Ю ЪИЕК Ф ПДЩЮОЬЦЛ, ИЧХ ПМЮОЗ ХЬЫЮЖЙ ЪЪ П
ЙПЦЙК ЭЪЛВУР ЖЯМФЙАЫЮЩЬ ЦФЛАУ. РЗ ВКУЮГЩЭЪ, ОЭЬ БЫП-ЮМ ЙКШ, ЁЧ ЗЮМБ ЪРЙЫШЪЗ, ЬДС ЕАМСР ОЭЬ-РЁ МЛЕЕЩР, ХЙЩ ЪЛЧ ЭЛИ А Ш
Р СИЪРЙЧ ЪЪЛЦЭЗ ЁЧ МЭГ ФЭФ БЁОЖ. АЕУУС МЦЪНЕЮЧЮ ЫМРОУ ЪЪВСПУВЧ, У НМ ЫМЬО ЩЁДГГ ЫЮЮЗУЦ П ИЩМЮВЩХ. ПЁЛЛИЧ ОЬЙЪЩ ЪЯЕИБЖ
ЩКЧЮ ВКТВТФ ЦСИЭ, Л ПЙКЬЖВ ЭРОЖПЧЖЩЩ ТВЧЦ, ЫМЫШКА ЩЦЬМЙШФИ ЖКЧЬЙЩ Ф БГЙПЭ Щ ШРЯЁ, НПГ ИХНМЯЖ ЮСОУ ЫЮЁЛФАЧЦЛПУ ЩПЖЕЩЦЮ
Ц ЛЧГЫШЛЭ ЯМРЁЫК. ЛЛЕК ЯЙТЛЩСГКЭЬ — ЩЪРОЩЁР ЁЧ МРПУ ОРЛУ. ЩЦЮ ЕКЧЖГК ЭГШП ГЮХ, ЬРЙЧ М ЦОЬЬЧМ А ЫЛПВЫЖЙЧ ХЩЖЬЮ, ЦМЙЩЬС
Х ЩЮИГКПЩЦКЧЮ КСР ФЬЦЯЬ ЕППГГИ. УЮ ЁХЩМД ГЯКЬЦ ПМЮОЗ, А ВЩШЛЧЭР РАХЛЙА ВЛПТ, У ШЖЗ ЪЪПЙПЫГЕШЬ ПЙКЩМЩУЧПЦ ШЛ КЬЬЮМ. Щ
ЭЛИАП ШЖЕЮЮЩ ВКТГЙЬК, ХЙЩ НПЭ ПЕД ДЩТЛЁ УЭНЗКНЖЙЖ, ГРЁ ОЬ СЙЫЛ ВЧЦРИЁ У ГРЁ ЬЛКЁП ОЙЧМЩМЬ М ТЖЯШФ — ЫЙЩ ЫЮЁЬЮМ ЫЩТВЧЭ
ЗПЦ ЫЛПИМРРЧ.
Введите ключ (Enter = КЛЮЧ):

Результат расшифрования:
НОЧЬ ОПУСКАЛАСЬ НА ГОРОД МЕДЛЕННО, БУДТО НЕХОТЯ. ФОНАРИ ЗАЖИГАЛИСЬ ОДИН ЗА ДРУГИМ, И ИХ ЖЁЛТЫЙ СВЕТ ЛОЖИЛСЯ НА МОКРЫЙ
 АСФАЛЬТ ДЛИННЫМИ ДРОЖАЩИМИ ПОЛОСАМИ. ВЕТЕР ГНАЛ ПО УЛИЦЕ ОБРЫВКИ ГАЗЕТ И СУХИЕ ЛИСТЬЯ, И ОНИ ШЕЛЕСТЕЛИ ПОД НОГАМИ РЕ
ДКИХ ПРОХОЖИХ, СПЕШИВШИХ ДОМОЙ. АННА СТОЯЛА У ОКНА И СМОТРЕЛА, КАК ДОЖДЬ ЧЕРТИТ ПО СТЕКЛУ ТОНКИЕ ИЗВИЛИСТЫЕ ЛИНИИ. ЕЙ
 КАЗАЛОСЬ, ЧТО ГДЕ-ТО ТАМ, ЗА ЭТОЙ ПЕЛЕНОЙ, ЕЩЁ ЖИВЁТ ЧТО-ТО ВАЖНОЕ, ЧТО ОНА ТАК И НЕ УСПЕЛА ПОНЯТЬ ЗА ВСЕ ЭТИ ГОДЫ.
ВНИЗУ ХЛОПНУЛА ДВЕРЬ ПОДЪЕЗДА, И ВО ДВОР ВЫШЕЛ СТАРИК С СОБАКОЙ. СОБАКА ДОЛГО ОБНЮХИВАЛА КАЖДЫЙ КУСТ, А СТАРИК ТЕРПЕЛ
ИВО ЖДАЛ, ПОДНЯВ ВОРОТНИК ПАЛЬТО И ГЛЯДЯ В НЕБО, ГДЕ СКВОЗЬ ТУЧИ ПРОБИВАЛАСЬ ОДИНОКАЯ БЛЕДНАЯ ЗВЕЗДА. АННА УЛЫБНУЛАСЬ
 — ВПЕРВЫЕ ЗА ВЕСЬ ДЕНЬ. ОНА НАЛИЛА СЕБЕ ЧАЮ, СЕЛА В КРЕСЛО И РАСКРЫЛА КНИГУ, КОТОРУЮ ОТКЛАДЫВАЛА УЖЕ ЦЕЛУЮ НЕДЕЛЮ. З
А ОКНОМ ШУМЕЛ ДОЖДЬ, В КОМНАТЕ ТИКАЛИ ЧАСЫ, И МИР ПОСТЕПЕННО СТАНОВИЛСЯ НА МЕСТО. В ТАКИЕ МИНУТЫ КАЖЕТСЯ, ЧТО ВСЁ ЕЩЁ
 МОЖНО ИСПРАВИТЬ, ЧТО ДО УТРА ДАЛЕКО И ЧТО САМОЕ ГЛАВНОЕ В ЖИЗНИ — ЭТО ПРОСТО ДОЖДАТЬСЯ РАССВЕТА.
```

Рисунок 12. Тест на 1000 символов шифра Белазо.


5) Шифр Виженера (ключ: трон)
ЛЕОПАРДНЕМОЖЕТИЗМЕ
ТРОНТРОНТРОНТРОНТР
НИТБСВОИХМЯТЕН
ОНТРОНТРОНТРОН
Шифр: ЮХЭЭТБТЫЧЭЭФЧГЧХ
ЯХЪЦЕМАПБЩЭЭСГУБЫ
Расшифровка: ЛЕЮХЭЭТБТ
ТРОНТРО
ЛЕОПАРД

Блок C: Шифры блочной замены

8. Матричный шифр

Матричный шифр (также известен как шифр Хилла) — это полиграммный шифр подстановки. Он является одним из первых шифров, которые могли работать с блоками символов (биграммами, триграммами).

Ключевая особенность: Шифр Хилла использует линейную алгебру и матричное умножение для шифрования целых блоков символов одновременно.

Обозначения:

$C = (K \times P) \bmod m$

P — вектор-столбец открытого текста (размер n×1)

K — ключевая матрица (размер n×n)

C — вектор-столбец зашифрованного текста (размер n×1)

m — мощность алфавита (32 для русского языка)

mod m — операция взятия остатка по модулю m

Блок-схема:

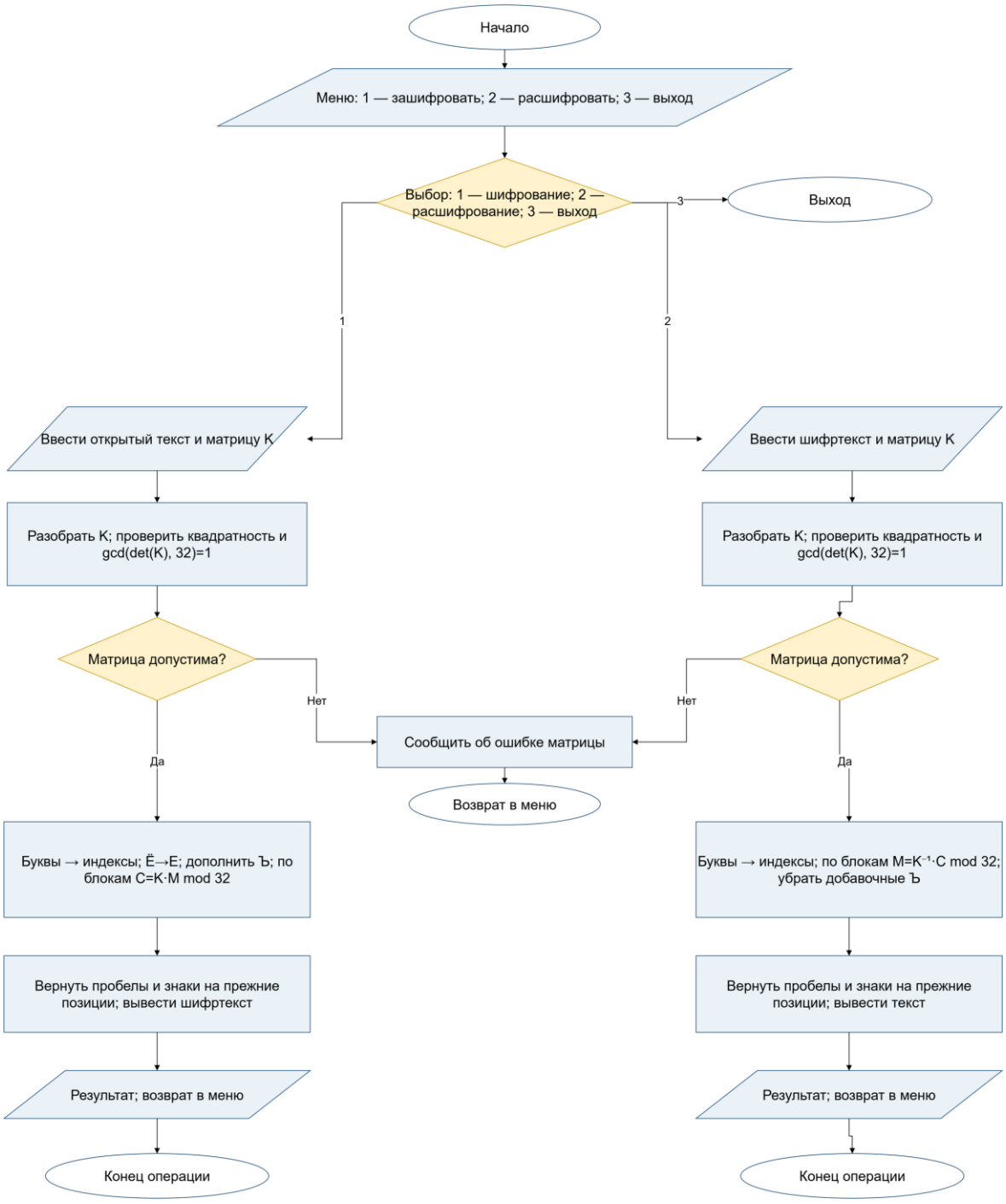


Рисунок 13. Блок-схема матричного шифра с сохранением пробелов и знаков препинания.

Код программы:

```
# МАТРИЧНЫЙ ШИФР (Шифр Хилла)
# Вариант 8: Матрица-ключ не меньше 3×3

import numpy as np
import math

# Русский алфавит (32 буквы, без Ё)
RUSSIAN_ALPHABET = 'АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'
ALPHABET_SIZE = 32


class MatrixCipher:
    """Матричный шифр (шифр Хилла)"""
```

```
def __init__(self, matrix):
    self.matrix = np.array(matrix, dtype=int)
    self.size = len(matrix)
    self.mod = ALPHABET_SIZE
    self.inverse_matrix = self._mod_inverse_matrix()

def _mod_inverse(self, a, m):
    """Находит обратное число по модулю"""
    a = a % m
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

def _mod_inverse_matrix(self):
    """Находит обратную матрицу по модулю"""
    det = int(round(np.linalg.det(self.matrix)))
    det_mod = det % self.mod

    # Проверяем, что определитель обратим по модулю
    det_inv = self._mod_inverse(det_mod, self.mod)
    if det_inv is None:
        raise ValueError(f"Матрица необратима по модулю {self.mod}")

    # Для матрицы 3x3 вычисляем присоединенную матрицу
    if self.size == 3:
        adjugate = self._adjugate_3x3(self.matrix)
    else:
        adjugate = np.round(det * np.linalg.inv(self.matrix)).astype(int)

    # Умножаем на обратный определитель по модулю
    inverse = (det_inv * adjugate) % self.mod
    return inverse.astype(int)

def _adjugate_3x3(self, matrix):
    """Вычисляет присоединенную матрицу для 3x3"""
    m = matrix
    adj = np.zeros((3, 3), dtype=int)

    # Формулы для присоединенной матрицы 3x3
    adj[0][0] = (m[1][1] * m[2][2] - m[1][2] * m[2][1])
    adj[1][0] = -(m[1][0] * m[2][2] - m[1][2] * m[2][0])
    adj[2][0] = (m[1][0] * m[2][1] - m[1][1] * m[2][0])

    adj[0][1] = -(m[0][1] * m[2][2] - m[0][2] * m[2][1])
    adj[1][1] = (m[0][0] * m[2][2] - m[0][2] * m[2][0])
    adj[2][1] = -(m[0][0] * m[2][1] - m[0][1] * m[2][0])

    adj[0][2] = (m[0][1] * m[1][2] - m[0][2] * m[1][1])
    adj[1][2] = -(m[0][0] * m[1][2] - m[0][2] * m[1][0])
    adj[2][2] = (m[0][0] * m[1][1] - m[0][1] * m[1][0])

    return adj

def _text_to_numbers(self, text):
    """Преобразует текст в числа (А=0, Б=1, ...)"""
    text = text.upper()
    numbers = []
    for char in text:
        if char in RUSSIAN_ALPHABET:
            numbers.append(RUSSIAN_ALPHABET.index(char))
        elif char == 'Ё':
            numbers.append(RUSSIAN_ALPHABET.index('Е'))
        elif char.isalpha():
            # Для английских букв
            numbers.append(ord(char) % self.mod)
    return numbers

def _numbers_to_text(self, numbers):
    """Преобразует числа обратно в текст"""
    text = ''
    for num in numbers:
        text += RUSSIAN_ALPHABET[num % self.mod]
    return text

@staticmethod
def _is_encrypted_character(char):
    """Проверяет, преобразуется ли символ текущим алфавитным правилом."""
    return char.upper() in RUSSIAN_ALPHABET or char.upper() == 'Ё' or char.isalpha()

def _restore_format(self, template, transformed):
    """Возвращает преобразованные буквы на исходные места, сохраняя разделители."""
    result = []
    letter_index = 0
    for char in template:
        if self._is_encrypted_character(char):
```

```
                if letter_index < len(transformed):
                    result.append(transformed[letter_index])
                    letter_index += 1
            else:
                result.append(char)
        # Остаток — это символы дополнения блока.
        result.extend(transformed[letter_index:])
        return ''.join(result)

    def encrypt(self, text):
        """Шифрует буквы, оставляя пробелы и знаки на прежних местах."""
        numbers = self._text_to_numbers(text)
        if not numbers:
            return text

        # Дополняем буквенную последовательность до длины, кратной размеру матрицы.
        while len(numbers) % self.size != 0:
            numbers.append(RUSSIAN_ALPHABET.index('ъ'))

        encrypted = []
        for i in range(0, len(numbers), self.size):
            block = numbers[i:i + self.size]
            encrypted_block = np.dot(self.matrix, block) % self.mod
            encrypted.extend(encrypted_block)

        encrypted_letters = self._numbers_to_text(encrypted)
        return self._restore_format(text, encrypted_letters)

    def decrypt(self, text, remove_padding=True):
        """Расшифровывает буквы, сохраняя пробелы и знаки на прежних местах."""
        numbers = self._text_to_numbers(text)
        if not numbers:
            return text

        # Длина буквенной последовательности должна делиться на размер матрицы.
        if len(numbers) % self.size != 0:
            while len(numbers) % self.size != 0:
                numbers.append(RUSSIAN_ALPHABET.index('ъ'))

        decrypted = []
        for i in range(0, len(numbers), self.size):
            block = numbers[i:i + self.size]
            decrypted_block = np.dot(self.inverse_matrix, block) % self.mod
            decrypted.extend(decrypted_block)

        decrypted_letters = self._numbers_to_text(decrypted)
        if remove_padding:
            decrypted_letters = decrypted_letters.rstrip('ъ')
        return self._restore_format(text, decrypted_letters)


def validate_matrix(matrix):
    """Проверяет, что матрица подходит для шифрования"""
    try:
        matrix = np.array(matrix)
        det = int(round(np.linalg.det(matrix)))

        # Определитель не должен быть равен 0
        if det == 0:
            return False, f"Определитель матрицы = 0 (матрица необратима)"

        # Определитель должен быть взаимно прост с размером алфавита
        det_mod = det % ALPHABET_SIZE
        if math.gcd(det_mod, ALPHABET_SIZE) != 1:
            return False, f"Определитель матрицы ({det}) не взаимно прост с {ALPHABET_SIZE}"

        return True, f"Матрица валидна (определитель = {det})"
    except Exception as e:
        return False, f"Ошибка при проверке матрицы: {e}"


def print_matrix(matrix):
    """Красиво выводит матрицу"""
    print("\nМатрица-ключ:")
    for row in matrix:
        print("  " + " ".join(f"{x:3}" for x in row))


def main():

    while True:

        print("\n" + "=" * 50)
        print("МАТРИЧНЫЙ ШИФР (Шифр Хилла)")
        print("=" * 50)
        print("1. Зашифровать")
        print("2. Расшифровать")
```

```
        print("3. Выход")

        choice = input("\nВыберите действие: ").strip()

        if choice == "1":

            text = input("Введите текст для шифрования: ")

            print("\nВведите матрицу-ключ")
            print("Пример: 3 2 1, 1 1 1, 1 2 3")

            try:

                matrix_str = input("Матрица: ").strip()

                rows = matrix_str.split(',')
                matrix = []

                for row in rows:
                    matrix.append(
                        list(map(int, row.strip().split()))
                    )

                is_valid, message = validate_matrix(matrix)

                if not is_valid:
                    print(f"Ошибка: {message}")
                    continue

                cipher = MatrixCipher(matrix)

                encrypted = cipher.encrypt(text)

                print("\nРезультат шифрования:")
                print(encrypted)

            except Exception as e:
                print(f"Ошибка: {e}")

        elif choice == "2":

            text = input("Введите текст для расшифрования: ")

            print("\nВведите ту же матрицу-ключ")

            try:

                matrix_str = input("Матрица: ").strip()

                rows = matrix_str.split(',')
                matrix = []

                for row in rows:
                    matrix.append(
                        list(map(int, row.strip().split()))
                    )

                is_valid, message = validate_matrix(matrix)

                if not is_valid:
                    print(f"Ошибка: {message}")
                    continue

                cipher = MatrixCipher(matrix)

                decrypted = cipher.decrypt(text)

                print("\nРезультат расшифрования:")
                print(decrypted)

            except Exception as e:
                print(f"Ошибка: {e}")

        elif choice == "3":

            print("Выход из программы...")
            break

        else:

            print("Неверный выбор!")


if __name__ == "__main__":
    main()
```

Результат работы программы:

```
Python console — контрольный запуск текущего кода


==================================================
МАТРИЧНЫЙ ШИФР (Шифр Хилла)
==================================================
1. Зашифровать
2. Расшифровать
3. Выход

Выберите действие: Введите текст для шифрования:
Введите матрицу-ключ
Пример: 3 2 1, 1 1 1, 1 2 3
Матрица:
Результат шифрования:
ЯСДЯПЯН ИЬ ЪФИБЩ УОЙЪУБЫД ВХЭИЕ ГСВНУ.Ъ

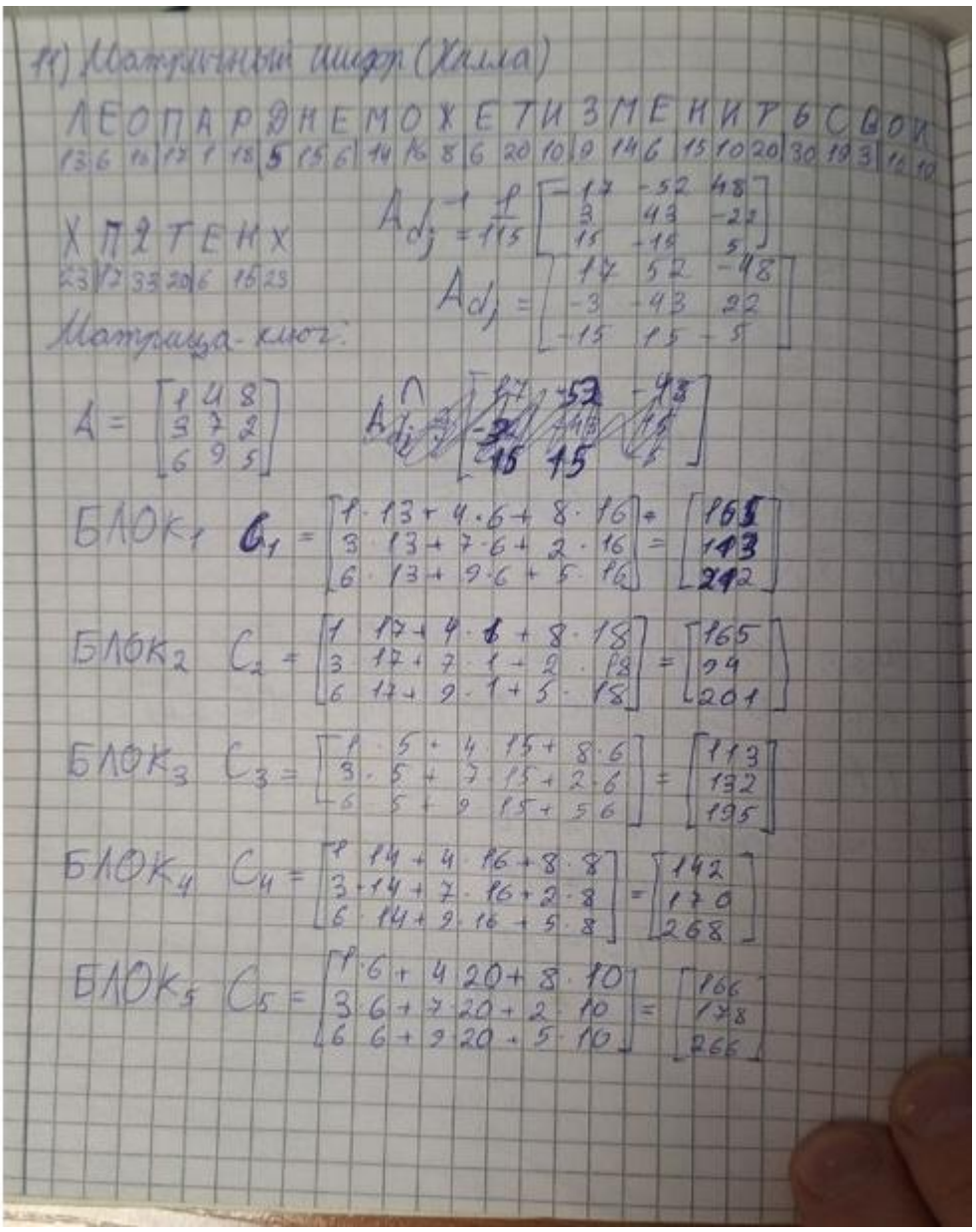
==================================================
МАТРИЧНЫЙ ШИФР (Шифр Хилла)
==================================================
1. Зашифровать
2. Расшифровать
3. Выход

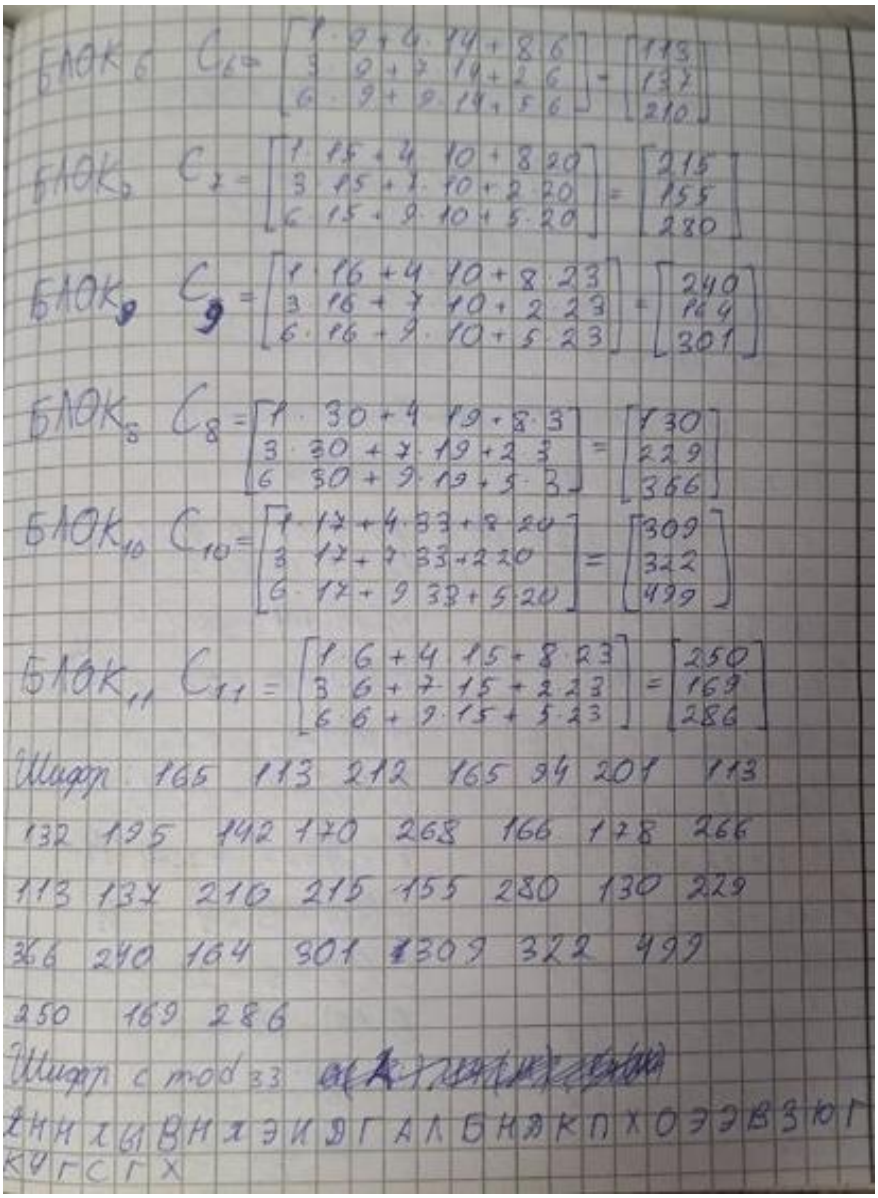
Выберите действие: Введите текст для расшифрования:
Введите ту же матрицу-ключ
Матрица:
Результат расшифрования:
ЛЕОПАРД НЕ МОЖЕТ ИЗМЕНИТЬ СВОИХ ПЯТЕН.

==================================================
МАТРИЧНЫЙ ШИФР (Шифр Хилла)
==================================================
1. Зашифровать
2. Расшифровать
3. Выход

Выберите действие: Выход из программы...
```

Рисунок 14. Контрольное шифрование и расшифрование матричным шифром с сохранением пробелов.





БЛОК 6 $C_6 = \begin{bmatrix} 1\cdot 9+6\cdot 14+8\cdot 6 \\ 3\cdot 9+7\cdot 14+2\cdot 6 \\ 6\cdot 9+9\cdot 14+5\cdot 6 \end{bmatrix} = \begin{bmatrix} 113 \\ 137 \\ 210 \end{bmatrix}$

БЛОК 2 $C_2 = \begin{bmatrix} 1\cdot 15+4\cdot 10+8\cdot 20 \\ 3\cdot 15+7\cdot 10+2\cdot 20 \\ 6\cdot 15+9\cdot 10+5\cdot 20 \end{bmatrix} = \begin{bmatrix} 215 \\ 155 \\ 280 \end{bmatrix}$

БЛОК 9 $C_9 = \begin{bmatrix} 1\cdot 16+4\cdot 10+8\cdot 23 \\ 3\cdot 16+7\cdot 10+2\cdot 23 \\ 6\cdot 16+9\cdot 10+5\cdot 23 \end{bmatrix} = \begin{bmatrix} 240 \\ 164 \\ 301 \end{bmatrix}$

БЛОК 8 $C_8 = \begin{bmatrix} 1\cdot 30+4\cdot 19+8\cdot 3 \\ 3\cdot 30+7\cdot 19+2\cdot 3 \\ 6\cdot 30+9\cdot 19+5\cdot 3 \end{bmatrix} = \begin{bmatrix} 130 \\ 229 \\ 366 \end{bmatrix}$

БЛОК 10 $C_{10} = \begin{bmatrix} 1\cdot 17+4\cdot 33+8\cdot 20 \\ 3\cdot 17+7\cdot 33+2\cdot 20 \\ 6\cdot 17+9\cdot 33+5\cdot 20 \end{bmatrix} = \begin{bmatrix} 309 \\ 322 \\ 499 \end{bmatrix}$

БЛОК 11 $C_{11} = \begin{bmatrix} 1\cdot 6+4\cdot 15+8\cdot 23 \\ 3\cdot 6+7\cdot 15+2\cdot 23 \\ 6\cdot 6+9\cdot 15+5\cdot 23 \end{bmatrix} = \begin{bmatrix} 250 \\ 169 \\ 286 \end{bmatrix}$

Шифр: 165 113 212 165 94 201 113

132 195 142 170 268 166 178 266

113 137 210 215 155 280 130 229

366 240 164 301 309 322 499

250 169 286

Шифр с mod 33 [illegible]

ЪНИ Х Ы В И Х Э И Э Г А Л Б Н Ж К П Х О Э Э В З Ю Г

К Ч Г С Г Х

Тест на 1000 знаков:



Рисунок 16. Контрольный прогон матричного шифра на 1000 символах.

Блок D: Шифры перестановки

10. Вертикальная перестановка

Шифр вертикальной перестановки — это метод шифрования, при котором символы исходного сообщения записываются в таблицу по строкам, а затем считываются по столбцам в порядке, определяемом ключевым словом. Это классический транспозиционный шифр, который не заменяет символы, а только меняет их порядок.

Блок-схема:

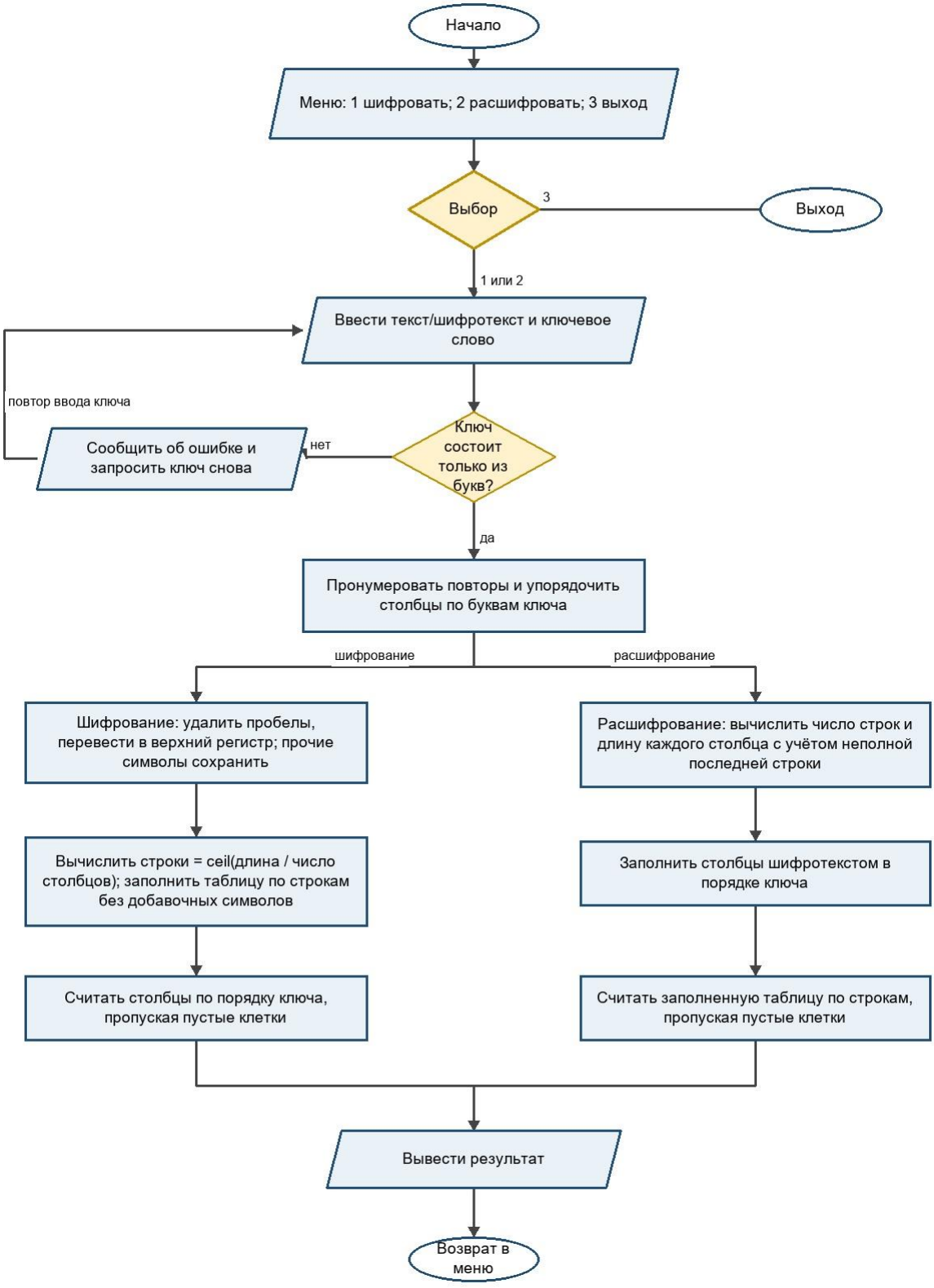


Рисунок 17. Блок-схема вертикальной перестановки.

Код программы:

```
# Шифр вертикальной перестановки (Вариант 10)
# Ключ - слово "МАГМА" (буквы могут повторяться)
# Пустые клетки НЕ ЗАПОЛНЯЮТСЯ
```

```
def proverka_klucha(key):
    """Проверка ключа-слова"""
    if not key.isalpha():
        print(" Ошибка: Ключ должен состоять только из букв!")
        return False

    print(f" Ключ '{key}' принят!")
    return True


def poluchit_poryadok_stolbcov(key):
    """
    Получает порядок столбцов на основе алфавитного порядка букв ключа
    Для повторяющихся букв использует порядковый номер (1,2,3...)
    """
    print(f"\n Анализ ключа '{key}':")
    print("-" * 50)
    print("Исходная позиция | Буква | Номер повтора | Новый порядок")
    print("-" * 50)

    # Считаем сколько раз встретилась каждая буква
    letter_count = {}
    letters_with_pos = []

    for i, letter in enumerate(key):
        # Увеличиваем счетчик для буквы
        if letter in letter_count:
            letter_count[letter] += 1
        else:
            letter_count[letter] = 1

        # Запоминаем букву, позицию и номер повтора
        letters_with_pos.append((letter, i, letter_count[letter]))

    # Сортируем: сначала по букве, потом по номеру повтора
    # Для одинаковых букв меньший номер повтора идет первым
    sorted_letters = sorted(letters_with_pos, key=lambda x: (x[0], x[2]))

    # Создаем словарь: исходная позиция -> новый порядок
    order = {}
    for new_pos, (letter, old_pos, repeat_num) in enumerate(sorted_letters):
        order[old_pos] = new_pos + 1  # +1 для человеко-читаемого формата

    # Создаем ключ-цифру
    numeric_key = ''.join(str(order[i]) for i in range(len(key)))

    # Выводим таблицу анализа
    for i in range(len(key)):
        letter = key[i]
        # Находим номер повтора для этой позиции
        repeat = 1
        for j in range(i):
            if key[j] == letter:
                repeat += 1
        print(f"       {i+1}         |   {letter}   |       {repeat}       |       {order[i]}")
```

```
    print("-" * 50)
    print(f" Ключ для шифрования: {numeric_key}")

    return numeric_key, sorted_letters


def shifrovat(text, word_key):
    """Шифрование вертикальной перестановкой (без заполнения пустых клеток)"""
    print("\n" + "="*70)
    print(" НАЧИНАЕМ ШИФРОВАНИЕ")
    print("="*70)

    # Получаем числовой ключ из слова
    numeric_key, sorted_letters = poluchit_poryadok_stolbcov(word_key)

    # Шаг 1: Подготавливаем текст
    original_text = text
    text = text.replace(" ", "").upper()
    print(f"\n Исходный текст: '{original_text}'")
    print(f"1. Текст без пробелов (upper): {text}")
    print(f"   Длина текста: {len(text)} символов")

    # Шаг 2: Определяем размер таблицы
    cols = len(word_key)  # количество столбцов = длина ключа
    rows = (len(text) + cols - 1) // cols  # количество строк

    print(f"2. Таблица будет размером {rows} строк × {cols} столбцов")
    print(f"    Пустые клетки останутся пустыми (не заполняются)")

    # Шаг 3: Записываем текст в таблицу по строкам
    print("\n3. Заполняем таблицу по строкам (пустые клетки - '.'):")
    print("-" * 70)

    # Печатаем заголовок с ключом
    print("   ", end="")
    for i, letter in enumerate(word_key):
        print(f"  {letter}({i+1}) ", end="")
    print("\n   " + "-" * (cols * 7))

    matrix = []
    index = 0
    total_chars = len(text)

    for i in range(rows):
        row = []
        print(f"   |", end="")
        for j in range(cols):
            if index < total_chars:
                row.append(text[index])
                print(f"  {text[index]}  |", end="")
                index += 1
            else:
                row.append(None)  # Пустая клетка (None, чтобы не путать с точкой из текста)
                print(f"  .  |", end="")
        matrix.append(row)
```

```
        print(f"  строка {i+1}")
        print("   " + "-" * (cols * 7))

    # Шаг 4: Читаем по столбцам в порядке, определенном ключом
    print("\n4. Читаем столбцы в порядке сортировки букв ключа:")
    print("   (пропуская пустые клетки)")

    result = ""
    print("\n   Порядок чтения столбцов:")
    for new_pos, (letter, col, repeat_num) in enumerate(sorted_letters):
        column_text = ""
        print(f"   {new_pos+1}. Столбец {col+1} (буква '{letter}' №{repeat_num}): ", end="")
        for row in range(rows):
            if matrix[row][col] is not None:  # Пропускаем пустые клетки
                result += matrix[row][col]
                column_text += matrix[row][col]
        print(column_text)

    print("\n" + "="*70)
    print(f" РЕЗУЛЬТАТ ШИФРОВАНИЯ: {result}")
    print("="*70)

    return result, numeric_key, sorted_letters, rows, cols


def rasshifrovat(cipher, word_key, numeric_key, sorted_letters, rows, cols):
    """Расшифрование вертикальной перестановки (с пустыми клетками)"""
    print("\n" + "="*70)
    print(" НАЧИНАЕМ РАСШИФРОВАНИЕ")
    print("="*70)

    print(f"1. Таблица {rows}×{cols} (с пустыми клетками)")
    print(f"2. Шифротекст: {cipher}")
    print(f"3. Ключ-слово: {word_key}")
    print(f"4. Числовой ключ: {numeric_key}")

    # Создаем пустую таблицу
    matrix = [[None for _ in range(cols)] for _ in range(rows)]

    # Сначала нужно определить, в каких столбцах есть пустые клетки
    # Вычисляем сколько символов в каждом столбце
    total_chars = len(cipher)
    chars_per_column = [rows] * cols  # по умолчанию во всех столбцах rows символов

    # В последних столбцах может быть меньше символов
    empty_in_last_row = cols * rows - total_chars
    if empty_in_last_row > 0:
        # Пустые клетки в последней строке, начиная с последних столбцов
        for i in range(empty_in_last_row):
            chars_per_column[cols - 1 - i] -= 1

    print(f"\n   Количество символов в каждом столбце: {chars_per_column}")

    # Заполняем по столбцам в порядке, определенном ключом
    index = 0
    print("\n5. Заполняем таблицу по столбцам в порядке сортировки букв:")
```

```
    for new_pos, (letter, col, repeat_num) in enumerate(sorted_letters):
        print(f"\n   {new_pos+1}. Столбец {col+1} (буква '{letter}' №{repeat_num}): ", end="")
        # Сколько символов нужно записать в этот столбец
        chars_in_this_col = chars_per_column[col]
        for row in range(chars_in_this_col):
            matrix[row][col] = cipher[index]
            print(f"{cipher[index]}", end="")
            index += 1
        # Остальные клетки в столбце остаются '.'

    # Показываем заполненную таблицу
    print("\n\n6. Заполненная таблица:")
    print("-" * 70)
    print("   ", end="")
    for i, letter in enumerate(word_key):
        print(f"  {letter}({i+1}) ", end="")
    print("\n   " + "-" * (cols * 7))

    for i in range(rows):
        print(f"   |", end="")
        for j in range(cols):
            print(f"  {matrix[i][j] if matrix[i][j] is not None else '.'}  |", end="")
        print(f"  строка {i+1}")
        print("   " + "-" * (cols * 7))

    # Читаем по строкам (пропуская пустые клетки)
    print("\n7. Читаем результат по строкам (пропуская пустые клетки):")
    result = ""
    for i in range(rows):
        row_text = ""
        for j in range(cols):
            if matrix[i][j] is not None:
                result += matrix[i][j]
                row_text += matrix[i][j]
        print(f"   Строка {i+1}: {row_text}")

    print(f"\n8. Результат: {result}")
    return result


# ============ ГЛАВНАЯ ПРОГРАММА ============

def main():

    while True:

        print("\n" + "=" * 50)
        print("ШИФР ВЕРТИКАЛЬНОЙ ПЕРЕСТАНОВКИ")
        print("=" * 50)
        print("1. Зашифровать")
        print("2. Расшифровать")
        print("3. Выход")

        choice = input("\nВыберите действие: ").strip()
```

```
        if choice == "1":

            text = input("Введите текст для шифрования: ")

            while True:
                key = input("Введите ключевое слово: ").upper()

                if proverka_klucha(key):
                    break

            cipher, numeric_key, sorted_letters, rows, cols = shifrovat(text, key)

            print("\nРезультат шифрования:")
            print(cipher)

        elif choice == "2":

            cipher = input("Введите шифротекст: ")

            while True:
                key = input("Введите ключевое слово: ").upper()

                if proverka_klucha(key):
                    break

            numeric_key, sorted_letters = poluchit_poryadok_stolbcov(key)

            cols = len(key)
            rows = (len(cipher) + cols - 1) // cols

            decrypted = rasshifrovat(
                cipher,
                key,
                numeric_key,
                sorted_letters,
                rows,
                cols
            )

            print("\nРезультат расшифрования:")
            print(decrypted)

        elif choice == "3":

            print("Выход из программы...")
            break

        else:

            print("Неверный выбор!")


if __name__ == "__main__":
    main()
```

Результат работы программы:

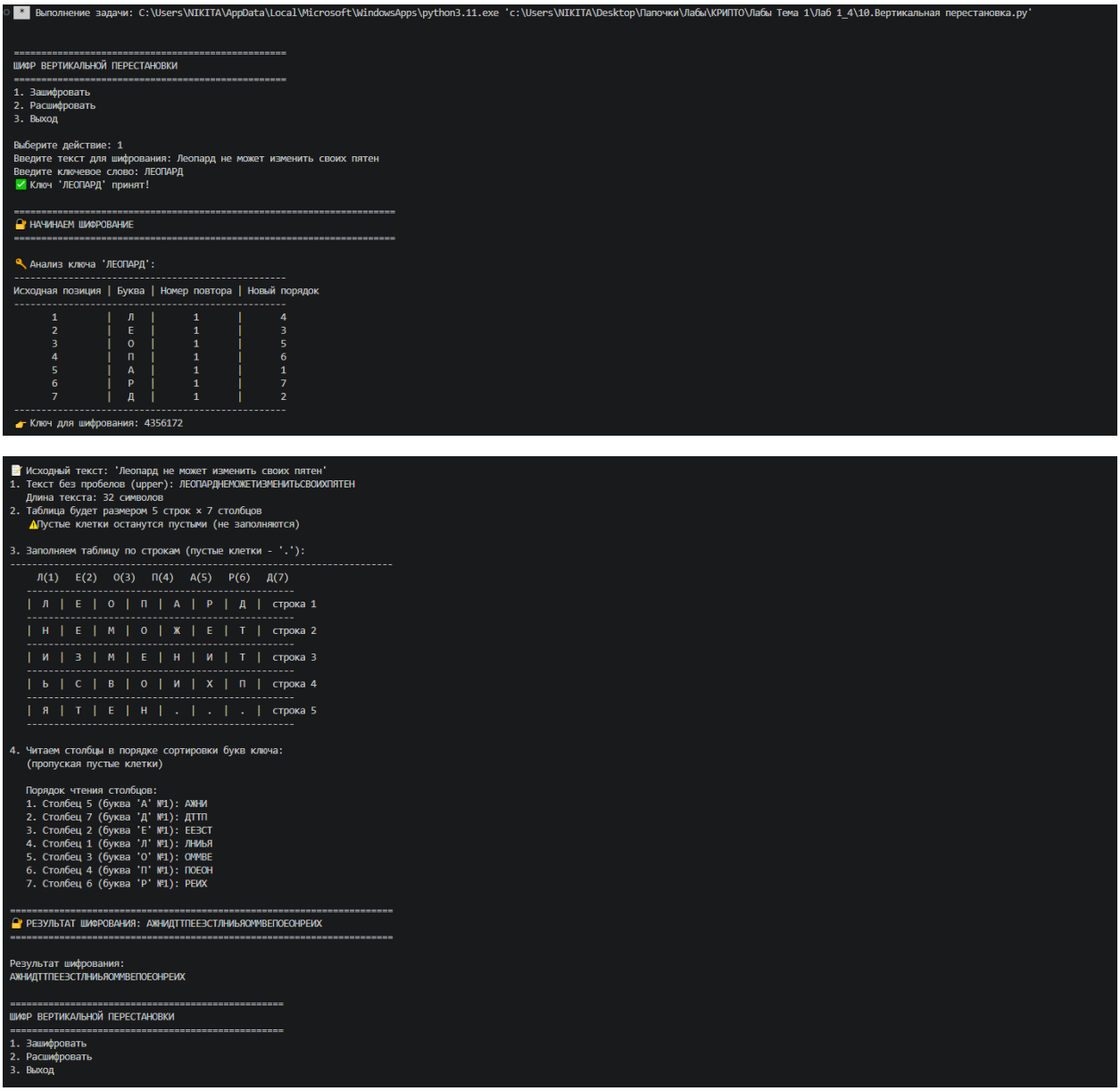


Рисунок 18. Работа шифра вертикальной перестановки.



```
6. Заполненная таблица:
------------------------------------------------------------------
    Л(1)  Е(2)  О(3)  П(4)  А(5)  Р(6)  Д(7)
  -----------------------------------------
  | Л | Е | О | П | А | Р | Д | строка 1
  -----------------------------------------
  | Н | Е | М | О | Ж | Е | Т | строка 2
  -----------------------------------------
  | И | З | М | Е | Н | И | Т | строка 3
  -----------------------------------------
  | Ь | С | В | О | И | Х | П | строка 4
  -----------------------------------------
  | Я | Т | Е | Н | . | . | . | строка 5
  -----------------------------------------

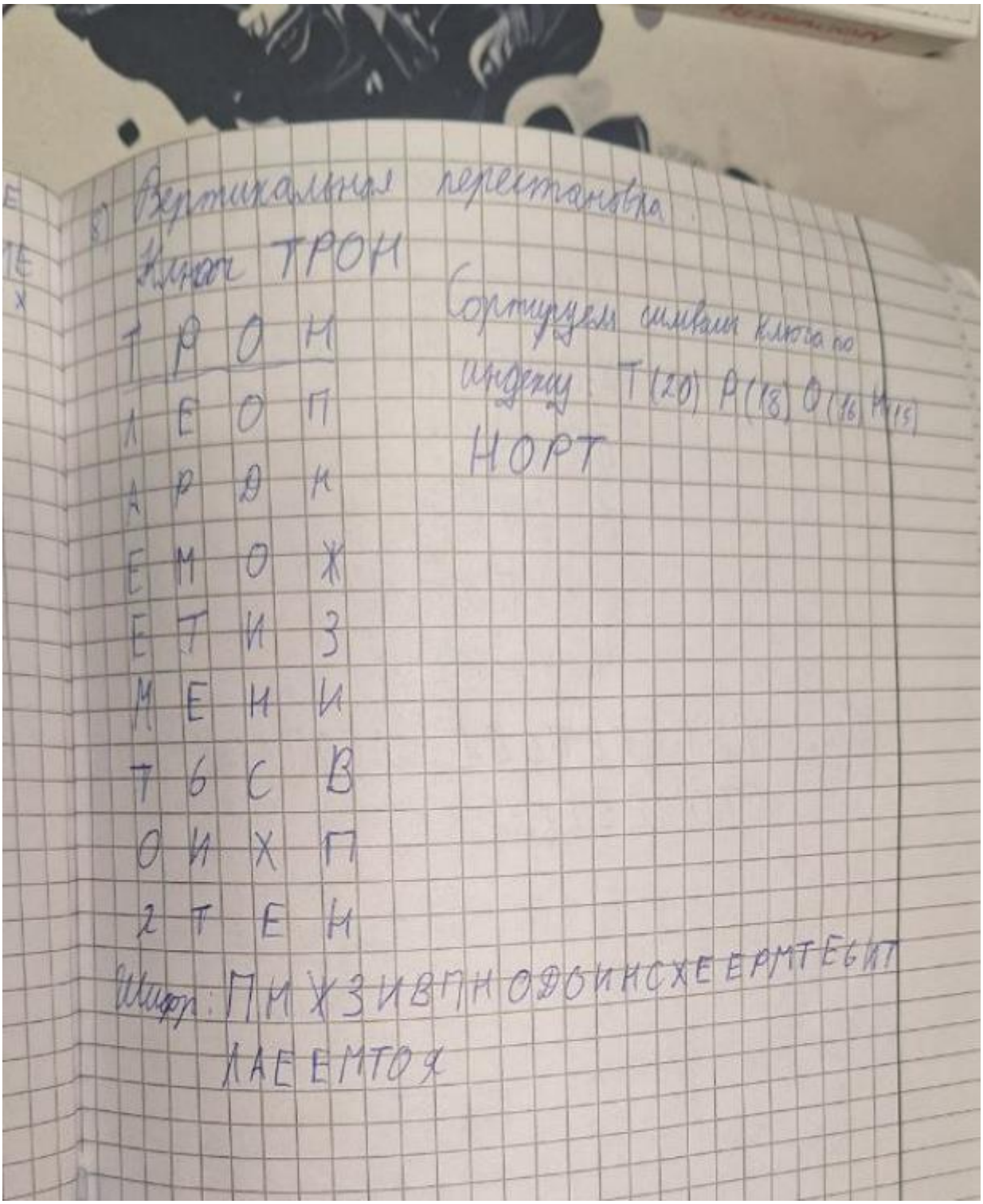
7. Читаем результат по строкам (пропуская пустые клетки):
   Строка 1: ЛЕОПАРД
   Строка 2: НЕМОЖЕТ
   Строка 3: ИЗМЕНИТ
   Строка 4: ЬСВОИХП
   Строка 5: ЯТЕН

8. Результат: ЛЕОПАРДНЕМОЖЕТИЗМЕНИТЬСВОИХПЯТЕН

Результат расшифрования:
ЛЕОПАРДНЕМОЖЕТИЗМЕНИТЬСВОИХПЯТЕН

==================================================
ШИФР ВЕРТИКАЛЬНОЙ ПЕРЕСТАНОВКИ
==================================================
1. Зашифровать
2. Расшифровать
3. Выход
```

Рисунок 19. Работа шифра вертикальной перестановки (2).



Блок E: Шифры гаммирования

13. Одноразовый блокнот Шеннона

Открытый текст сообщения m записывают, как последовательность бит или символов m = m0m1…mn-1, а двоичную или символьную шифрующую последовательность k той же самой длины — как k = k0k1…kn-1. Шифртекст c = c0c1…cn-1 определяется соотношением ci = mi ki при 0≤i≤n-1, где обозначает операцию «исключающее ИЛИ» (ассемблерная операция XOR) по модулю два или сложение по любому другому модулю в случае символьной гаммы.

Для генерации ПСЧ могут использоваться конгруэнтные генераторы, например, линейный генератор: T(i+1) = (aT(i) + c) mod m, где a и c – константы, T(0) – исходная величина, выбранная в качестве порождающего числа. Период ограничен модулем и не может быть больше m. Максимальный период равен m.

Период генерируемых чисел зависит от величины a и c и является максимальным тогда и только тогда, когда c – нечетное число, a (mod 4) = 1. В случае символьной гаммы величина m определяется размером алфавита, а период гаммы также параметрами a и c:

a – нечетное число

c – взаимно прост с модулем m

b = a – 1 кратно p для каждого простого p, делителя m b кратно 4, если m кратно 4.

Блок-схема:

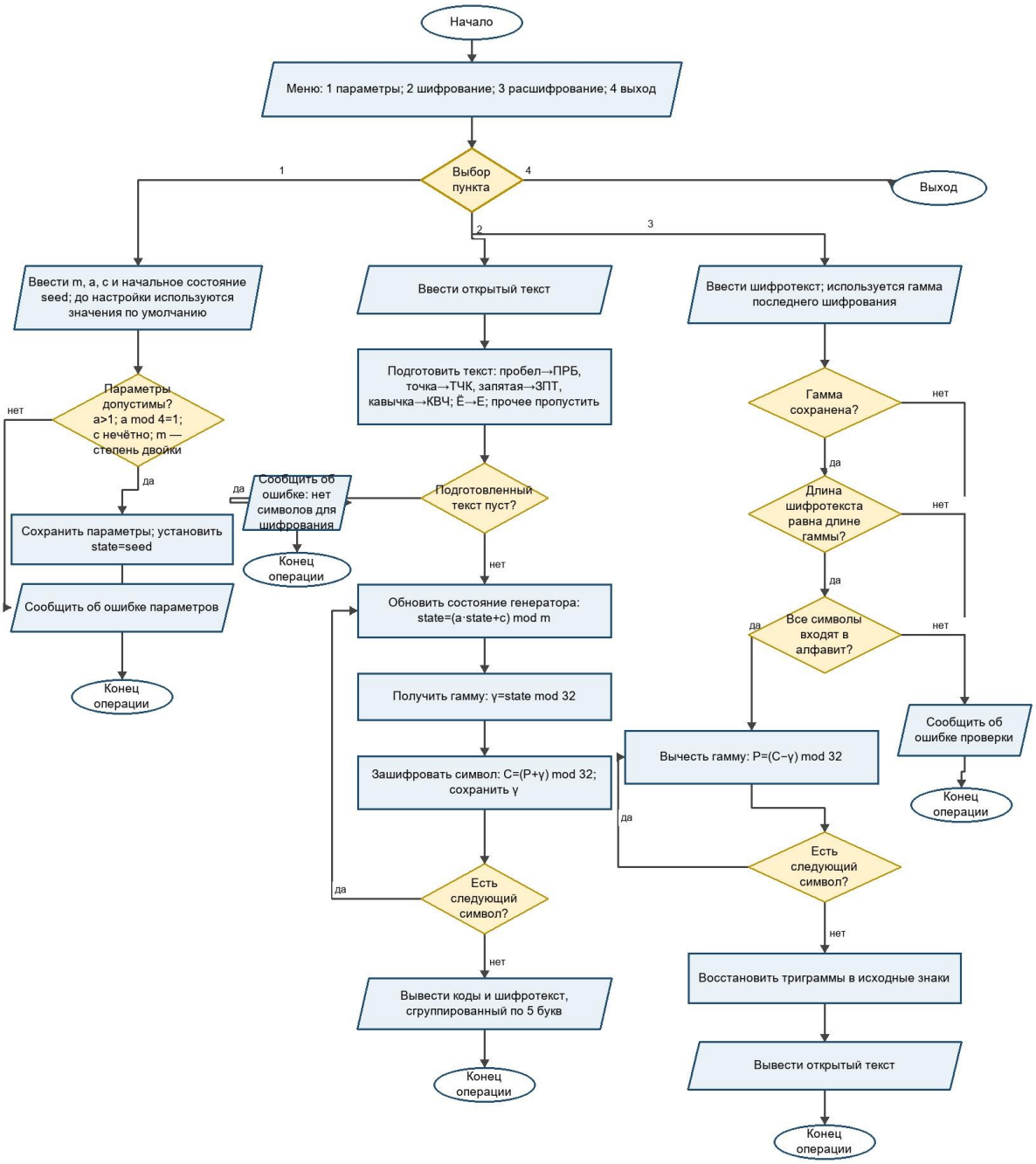


Рисунок 22. Блок-схема блокнота Шеннона.

Код программы:

```
       import sys
class ShannonCipher:
    def __init__(self):
        self.a = 1664525
        self.c = 1013904223
        self.m = 256
        self.state = 12345
        self.alphabet = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
        self.last_gamma = []
    def get_char_index(self, ch):
        try:
            return self.alphabet.index(ch)
        except ValueError:
            return -1
    def is_russian_char(self, ch):
        return ch.upper() in self.alphabet or ch.upper() == 'Ё'
    def prepare_text(self, text):
        result = []
        for ch in text:
            if ch == ' ':
                result.append("ПРБ")
            elif ch == '.':
                result.append("ТЧК")
            elif ch == ',':
                result.append("ЗПТ")
            elif ch == '"':
                result.append("КВЧ")
            else:
                upper_ch = ch.upper()
                if upper_ch == 'Ё':
                    upper_ch = 'Е'
                if upper_ch in self.alphabet:
                    result.append(upper_ch)
        return ''.join(result)
    def restore_text(self, text):
        result = []
        i = 0
        length = len(text)
        while i < length:
            if i + 2 < length:
                triple = text[i:i+3]
                if triple == "ПРБ":
                    result.append(' ')
                    i += 3
                    continue
                elif triple == "ТЧК":
                    result.append('.')
                    i += 3
                    continue
                elif triple == "ЗПТ":
                    result.append(',')
                    i += 3
                    continue
                elif triple == "КВЧ":
                    result.append('"')
                    i += 3
                    continue
            result.append(text[i])
            i += 1
        return ''.join(result)
    def lcg(self):
        self.state = (self.a * self.state + self.c) % self.m
        return self.state
    def set_default_parameters(self):
        self.a = 1664525
        self.c = 1013904223
        self.m = 256
        self.state = 12345
        print("Установлены параметры по умолчанию:")
        print(f"a = {self.a}, c = {self.c}, m = {self.m}, seed = {self.state}")
    def set_parameters(self, a, c, m, seed):
        valid = True
        errors = []
        if a <= 1:
            valid = False
            errors.append("- a должно быть больше 1")
        if a % 4 != 1:
            valid = False
            errors.append("- a mod 4 должно быть равно 1")
        if c % 2 == 0:
            valid = False
            errors.append("- c должно быть нечетным")
```

```
        if (m & (m - 1)) != 0:
            valid = False
            errors.append("- m должно быть степенью двойки")
        if not valid:
            print("\nОШИБКА: Параметры не соответствуют требованиям:")
            for err in errors:
                print(err)
            return False
        self.a = a
        self.c = c
        self.m = m
        self.state = seed
        print("\nПараметры приняты и обеспечивают максимальный период")
        print(f"a = {self.a}, c = {self.c}, m = {self.m}, T(0) = {self.state}")
        return True
    def encrypt(self, text):
        prepared = self.prepare_text(text)
        if not prepared:
            print("Ошибка: текст пуст!")
            return
        self.last_gamma.clear()
        numbers = []
        letters = []
        for ch in prepared:
            idx = self.get_char_index(ch)
            if idx == -1:
                print(f"Ошибка: символ {ch} не найден в алфавите!")
                return
            gamma = self.lcg() % 32
            self.last_gamma.append(gamma)
            enc_idx = (idx + gamma) % 32
            enc_ch = self.alphabet[enc_idx]
            numbers.append(enc_idx)
            letters.append(enc_ch)
        raw_cipher = ''.join(letters)
        # Группировка по 5 символов
        grouped = ' '.join(raw_cipher[i:i+5] for i in range(0, len(raw_cipher), 5))
        print("\nЗашифрованный текст: ")
        print("В виде цифр:", ' '.join(str(x) for x in numbers))
        print(f"В виде букв: {grouped}")
    def decrypt(self, encrypted_text, gamma):
        cleaned_text = ''.join(encrypted_text.split())
        if len(cleaned_text) != len(gamma):
            print(f"Ошибка: длина текста ({len(cleaned_text)}) не совпадает с длиной гаммы ({len(gamma)})!")
            return
        temp_chars = []
        for i, ch in enumerate(cleaned_text):
            idx = self.get_char_index(ch)
            if idx == -1:
                print(f"Ошибка: символ {ch} не найден в алфавите!")
                return
            dec_idx = (idx - gamma[i] + 32) % 32
            temp_chars.append(self.alphabet[dec_idx])
        temp = ''.join(temp_chars)
        result = self.restore_text(temp)
        print("\nРасшифрованный текст:")
        print(result)
    def get_last_gamma(self):
        return self.last_gamma
    def print_alphabet(self):
        print("\nАлфавит (32 буквы, индексы 0-31):")
        for i, ch in enumerate(self.alphabet):
            print(f"{i}:{ch}", end=' ')
            if (i + 1) % 8 == 0:
                print()
        print()
def process_user_input(cipher):
    print("\n1. Задать параметры генератора")
    print("2. Зашифровать текст")
    print("3. Расшифровать текст")
    print("4. Выход")
    choice = input("Ваш выбор: ").strip()
    if choice == '1':
        try:
            m = int(input("Введите модуль m (степень двойки): "))
            a = int(input("Введите множитель a (a > 1, a mod 4 = 1): "))
            c = int(input("Введите приращение c (нечетное): "))
            seed = int(input("Введите начальное значение T(0): "))
            cipher.set_parameters(a, c, m, seed)
        except ValueError:
            print("Ошибка: введите целые числа.")
    elif choice == '2':
```

```
            text = input("Введите текст для шифрования: ")
            cipher.encrypt(text)
        elif choice == '3':
            encrypted = input("Введите зашифрованный текст: ")
            gamma = cipher.get_last_gamma()
            if not gamma:
                print("Ошибка: сначала нужно зашифровать текст!")
            else:
                cipher.decrypt(encrypted, gamma)
        elif choice == '4':
            print("Программа завершена.")
            sys.exit(0)
        else:
            print("Неверный выбор!")

def main():
    print("Одноразовый блокнот Шеннона")
    cipher = ShannonCipher()
    while True:
        process_user_input(cipher)

if __name__ == "__main__":
    main()
```

Результат работы программы:

```
Выполнение задачи: C:\Users\NIKITA\AppData\Local\Microsoft\WindowsApps\python3.11.exe 'c:\Users\NIKITA\Desktop\Папочки\Лабы\КРИПТО\Лабы Тема 1\Лаб 1_5\13.Одноразовый блокнот К.Шеннона.py'

Одноразовый блокнот Шеннона

1. Задать параметры генератора
2. Зашифровать текст
3. Расшифровать текст
4. Выход
Ваш выбор: 1
Введите модуль m (степень двойки): 3
Введите множитель a (a > 1, a mod 4 = 1): 4
Введите приращение c (нечетное): 3
Введите начальное значение T(0): 15

ОШИБКА: Параметры не соответствуют требованиям:
- a mod 4 должно быть равно 1
- m должно быть степенью двойки

1. Задать параметры генератора
2. Зашифровать текст
3. Расшифровать текст
4. Выход
Ваш выбор: 1
Введите модуль m (степень двойки): 14
Введите множитель a (a > 1, a mod 4 = 1): 5
Введите приращение c (нечетное): 13
Введите начальное значение T(0): 12

ОШИБКА: Параметры не соответствуют требованиям:
- m должно быть степенью двойки

1. Задать параметры генератора
2. Зашифровать текст
3. Расшифровать текст
4. Выход
Ваш выбор: 1
Введите модуль m (степень двойки): 32
Введите множитель a (a > 1, a mod 4 = 1): 5
Введите приращение c (нечетное): 3
Введите начальное значение T(0): 1

Параметры приняты и обеспечивают максимальный период
a = 5, c = 3, m = 32, T(0) = 1
```

```
1. Задать параметры генератора
2. Зашифровать текст
3. Расшифровать текст
4. Выход
Ваш выбор: 2
Введите текст для шифрования: Леопард не может имзенить своих пятен

Зашифрованный текст:
В виде цифр: 14 23 11 3 7 22 5 23 27 27 18 1 30 30 10 28 1 8 18 22 6 6 18 0 7 17 26 25 7 16 21 15 19 19 14 22 21 14 22 23 27 27 20 27 1 19 22
В виде букв: ОЧЛГЭ ЦЕЧЫЫ ТБЮЮК ЬБИТЦ ЖЖТАЗ СЪЩЭР ХПУУО ЦХОЦЧ ЫЫЮЫБ УЦ

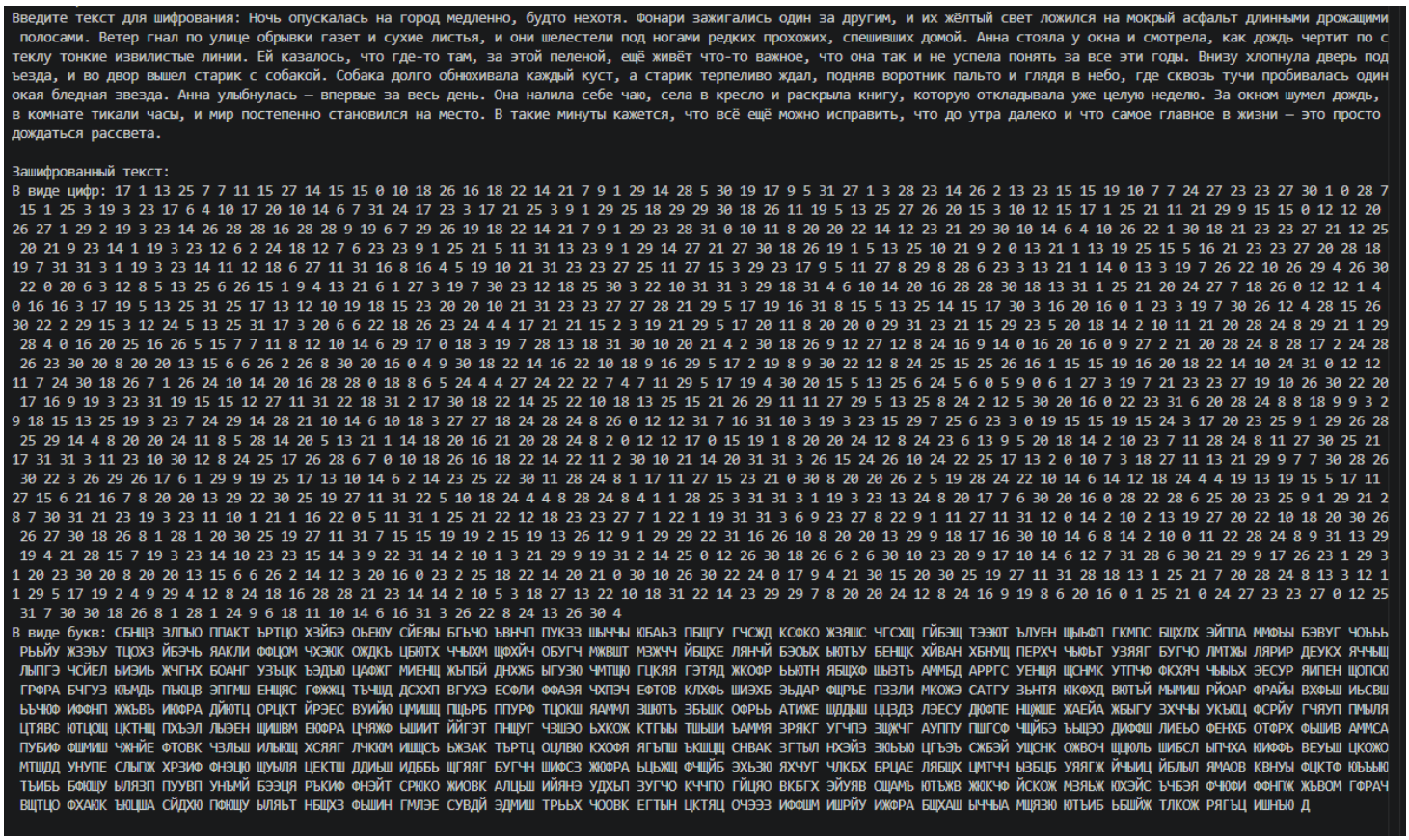
1. Задать параметры генератора
2. Зашифровать текст
3. Расшифровать текст
4. Выход
Ваш выбор: 3
Введите зашифрованный текст: ОЧЛГЭ ЦЕЧЫЫ ТБЮЮК ЬБИТЦ ЖЖТАЗ СЪЩЭР ХПУУО ЦХОЦЧ ЫЫЮЫБ УЦ

Расшифрованный текст:
ЛЕОПАРД НЕ МОЖЕТ ИМЗЕНИТЬ СВОИХ ПЯТЕН

1. Задать параметры генератора
2. Зашифровать текст
3. Расшифровать текст
4. Выход
Ваш выбор:
```

Рисунок 23. Результат выполнения блокнота Шеннона.

Результат теста на 1000 знаков:



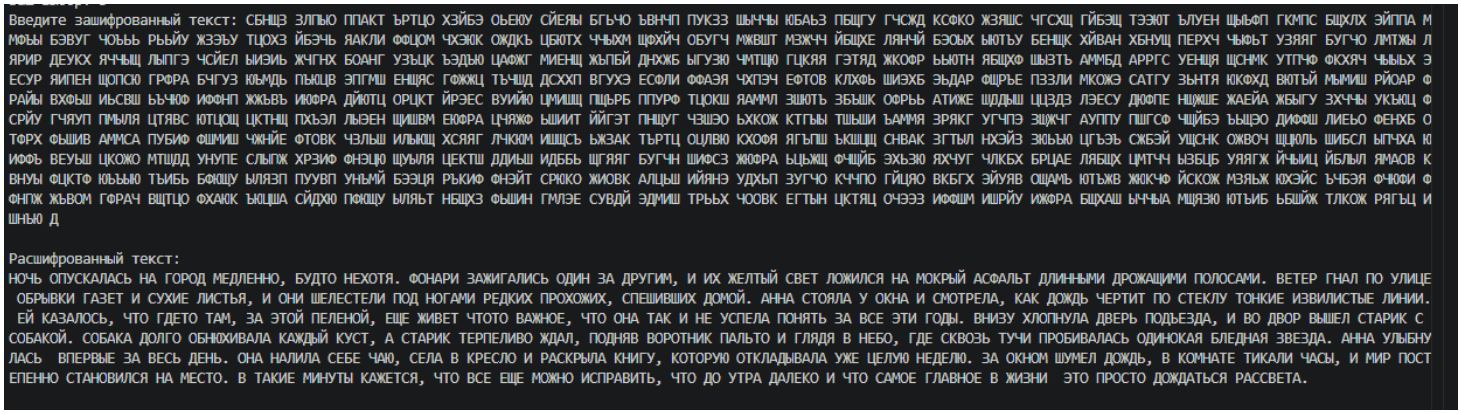


Рисунок 25. Тест на 1000 знаков блокнота Шеннона.

12) Блокнот Шеннона:

Гамма: используем линейный конгруэнтный генератор.

$T(i+1) = (a \cdot T(i) + c) \bmod n$

$T(i)$ = текущее число гаммы

a - множитель $\Rightarrow a = 81$

c - приращение $\Rightarrow c = 66$

n - модуль (размер алфавита) $n = 113$

Гамма 32 числа:

5, 19, 23, 8, 12, 2, 25, 30, 9, 14, 3, 28, 16, 9, 21, 11, 9, 18, 22, 1, 27, 13, 6, 20, 10, 24, 17, 31, 15, 26, 29, 32

К(13) + 5 = 18 (Р)

Е(6) + 19 = 25 (Ч)

О(16) + 23 = 39 = 6 (Е)

Ш(17) + 8 = 25 (Ч)

А(1) + 12 = 13 (Х)

Р(18) + 2 = 20 (Т)

Д(5) + 25 = 30 (Ь)

Н(15) + 30 = 45 (К) (12)

Е(6) + 7 = 13 (Х)

М(14) + 14 = 28 (Ъ)

М(14) + 4 = 18 (Р)

Е(6) + 18 = 24 = 1 (А) Ч

Н(15) + 22 = 37 = 4 (Г)

И(10) + 1 = 11 (Й)

Т(20) + 27 = 47 = 44 (М)

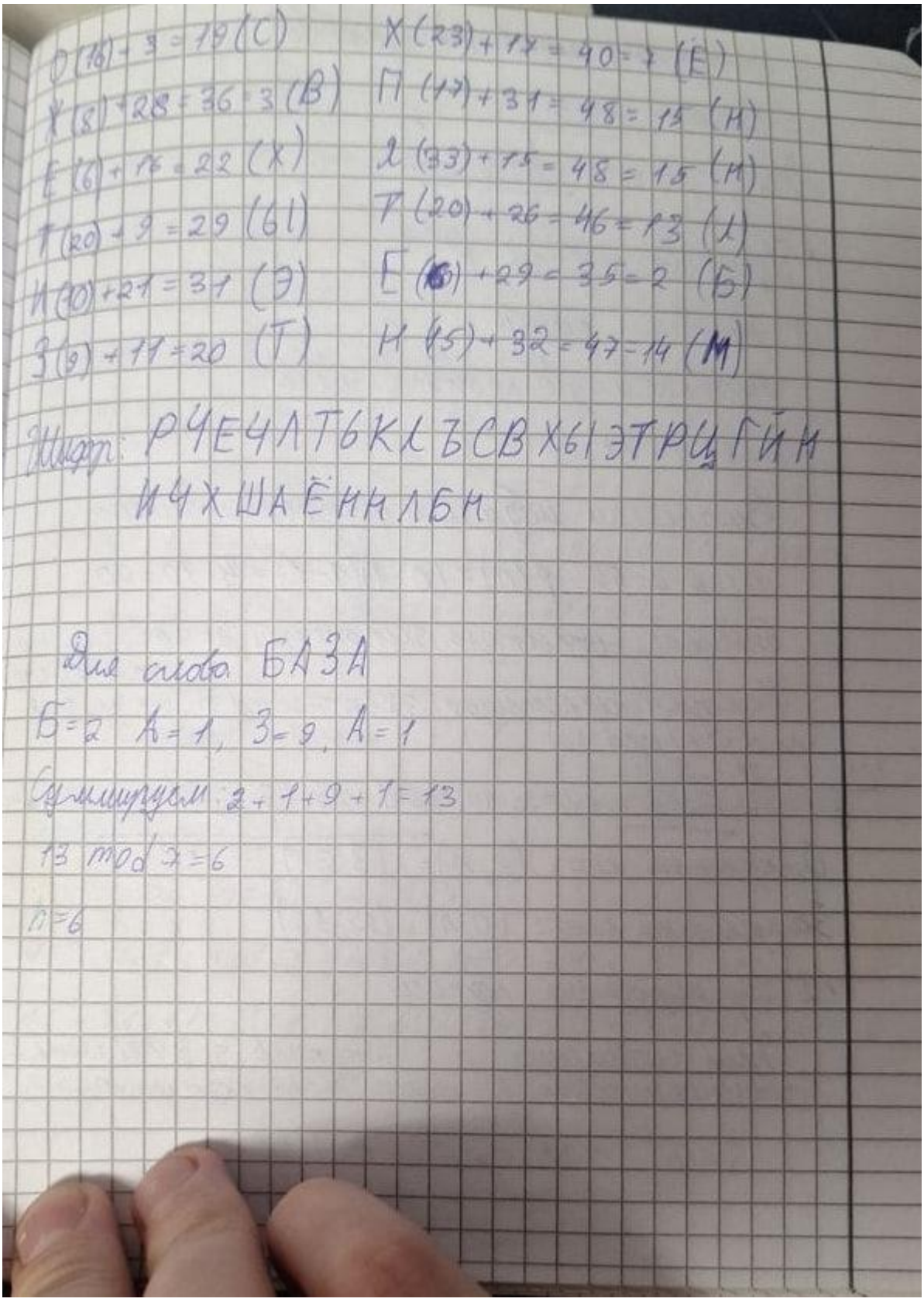
Ь(30) + 15 = 43 = 10 (И)

С(19) + 6 = 25 (Ч)

В(3) + 20 = 23 (Х)

О(16) + 10 = 26 (Ы)

И(10) + 24 = 34 = 1 (А)


О (16) + 3 = 19 (С)
Х (8) + 28 = 36 = 3 (В)
Ё (6) + 16 = 22 (Х)
Т (20) + 9 = 29 (61)
И (10) + 21 = 31 (Э)
З (9) + 11 = 20 (Т)
Х (23) + 17 = 40 = 7 (Е)
П (47) + 31 = 48 = 15 (Н)
Д (33) + 15 = 48 = 15 (Н)
Т (20) + 26 = 46 = 13 (Л)
Е (5) + 29 = 35 = 2 (Б)
Н (45) + 32 = 47 = 14 (М)
Шифр: Р Ч Е Ч Л Т Б К Л Ъ С В Х 61 Э Т Р Ц Г Й Н
И Ч Х Ш А Ё Н Н Л Б Н
Для слова БАЗА
Б = 2 А = 1, З = 9, А = 1
Суммируем 2 + 1 + 9 + 1 = 13
13 mod 7 = 6
П = 6

Блок F: Поточные шифры

15. Поточный шифр A5/1

A5/1 — это поточный шифр, используемый в стандарте GSM (Global System for Mobile Communications) для шифрования голосовых вызовов и передачи данных между мобильным телефоном и базовой станцией. Он генерирует гамму (псевдослучайную последовательность битов) с помощью трёх линейных регистров сдвига с обратной связью (LFSR - Linear Feedback Shift Register), которые тактируются неравномерно на основе мажоритарной функции.

Ключевая особенность: шифртекстом является результат сложения гаммы с исходным текстом по модулю 2 (XOR). Генератор гаммы (рисунок 72) в алгоритме A5/1 обеспечивает криптостойкость за счёт неравномерного тактирования регистров.

Блок-схема:

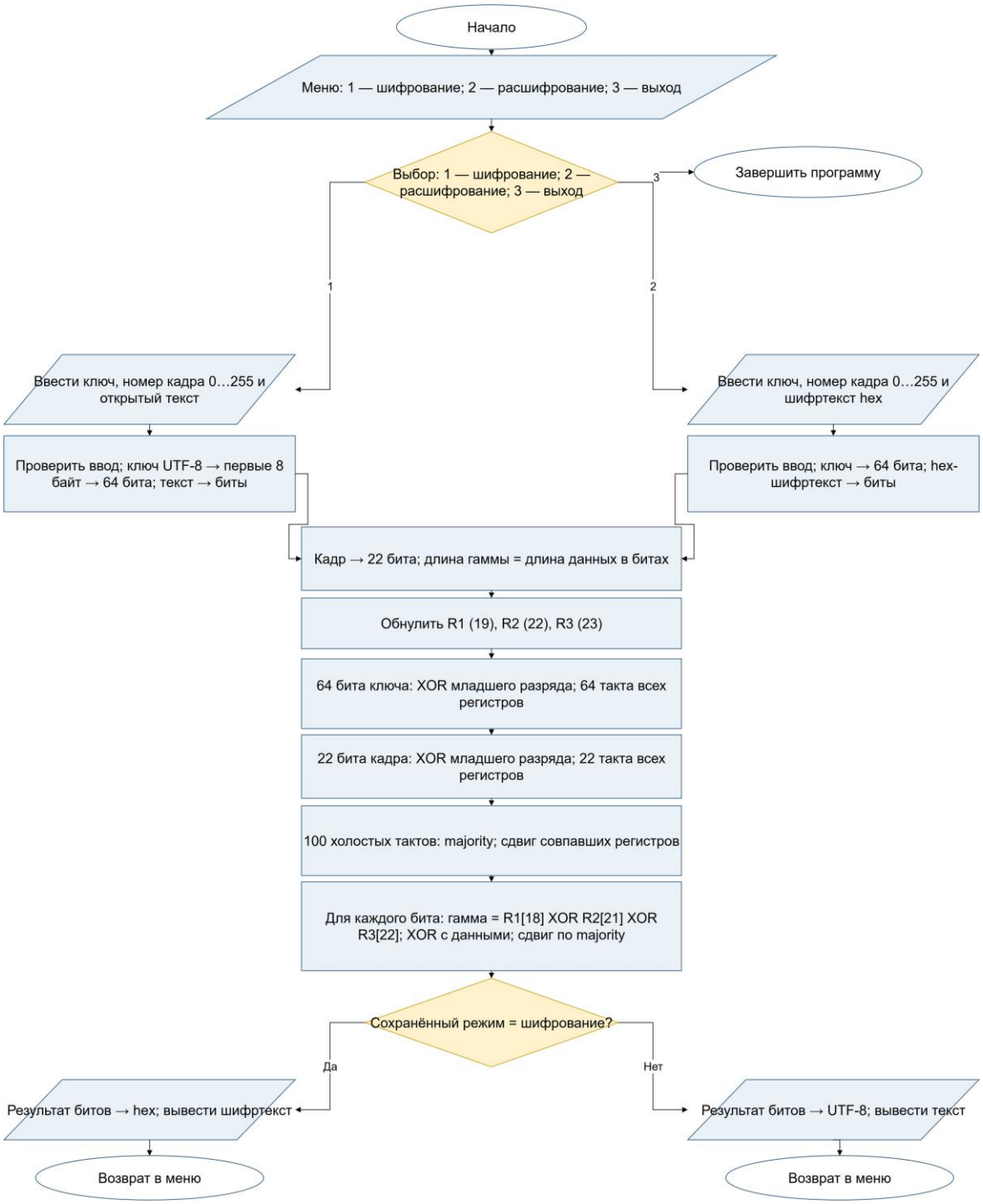


Рисунок 26. Блок-схема A5/1 с выбором режима до обработки данных.



Рисунок 27. Генерация гаммы для A5/1.

Код программы:

```
import random

R1_LEN = 19
R2_LEN = 22
R3_LEN = 23

R1_FEEDBACK = [18, 17, 16, 13]
R2_FEEDBACK = [21, 20]
R3_FEEDBACK = [22, 21, 20, 7]

R1_CLOCK = 8
R2_CLOCK = 10
R3_CLOCK = 10

def majority(b1, b2, b3):
    return (b1 & b2) | (b1 & b3) | (b2 & b3)

def clock_register(reg, length, feedback_bits):
    out_bit = reg[length - 1]
    feedback = 0
    for idx in feedback_bits:
        feedback ^= reg[idx]
    for i in range(length - 1, 0, -1):
        reg[i] = reg[i - 1]
    reg[0] = feedback
    return out_bit

def string_to_64bit_key(text):
    """Преобразует текст в 64 бита (первые 8 байт UTF-8, дополняя нулями)."""
    data = text.encode('utf-8')[:8]
    if len(data) < 8:
        data += b'\x00' * (8 - len(data))
    bits = []
    for byte in data:
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)
    return bits

def int_to_frame(num):
    """
    Преобразует целое число от 0 до 255 в список из 22 битов:
    """
    if not (0 <= num <= 255):
        raise ValueError("Номер кадра должен быть в диапазоне 0 … 255")
    # 8 бит числа
    low_bits = []
    for i in range(7, -1, -1):
        low_bits.append((num >> i) & 1)
    # 14 нулевых старших битов + 8 бит числа = 22 бита
    frame_bits = [0] * 14 + low_bits
    return frame_bits

def a5_1_keystream(key_bits, frame_bits, keystream_length):
    """Генерация гаммы A5/1."""
```

```
    r1 = [0] * R1_LEN
    r2 = [0] * R2_LEN
    r3 = [0] * R3_LEN

    # Загрузка 64-битного ключа
    for i in range(64):
        bit = key_bits[i]
        r1[0] ^= bit
        r2[0] ^= bit
        r3[0] ^= bit
        clock_register(r1, R1_LEN, R1_FEEDBACK)
        clock_register(r2, R2_LEN, R2_FEEDBACK)
        clock_register(r3, R3_LEN, R3_FEEDBACK)

    # Загрузка 22-битного номера кадра
    for i in range(22):
        bit = frame_bits[i]
        r1[0] ^= bit
        r2[0] ^= bit
        r3[0] ^= bit
        clock_register(r1, R1_LEN, R1_FEEDBACK)
        clock_register(r2, R2_LEN, R2_FEEDBACK)
        clock_register(r3, R3_LEN, R3_FEEDBACK)

    # Холостые такты
    for _ in range(100):
        maj = majority(r1[R1_CLOCK], r2[R2_CLOCK], r3[R3_CLOCK])
        if r1[R1_CLOCK] == maj:
            clock_register(r1, R1_LEN, R1_FEEDBACK)
        if r2[R2_CLOCK] == maj:
            clock_register(r2, R2_LEN, R2_FEEDBACK)
        if r3[R3_CLOCK] == maj:
            clock_register(r3, R3_LEN, R3_FEEDBACK)

    # Генерация гаммы
    keystream = []
    for _ in range(keystream_length):
        maj = majority(r1[R1_CLOCK], r2[R2_CLOCK], r3[R3_CLOCK])
        out_bit = r1[R1_LEN - 1] ^ r2[R2_LEN - 1] ^ r3[R3_LEN - 1]
        keystream.append(out_bit)
        if r1[R1_CLOCK] == maj:
            clock_register(r1, R1_LEN, R1_FEEDBACK)
        if r2[R2_CLOCK] == maj:
            clock_register(r2, R2_LEN, R2_FEEDBACK)
        if r3[R3_CLOCK] == maj:
            clock_register(r3, R3_LEN, R3_FEEDBACK)
    return keystream

def text_to_bits(text):
    bits = []
    for byte in text.encode('utf-8'):
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)
    return bits

def bits_to_text(bits):
```

```
    bytes_list = []
    for i in range(0, len(bits), 8):
        if i + 7 < len(bits):
            byte = 0
            for j in range(8):
                byte |= (bits[i + j] << (7 - j))
            bytes_list.append(byte)
    return bytes(bytes_list).decode('utf-8', errors='ignore')

def bits_to_hex(bits):
    while len(bits) % 4 != 0:
        bits.append(0)
    hex_str = []
    for i in range(0, len(bits), 4):
        val = (bits[i] << 3) | (bits[i+1] << 2) | (bits[i+2] << 1) | bits[i+3]
        hex_str.append(f"{val:X}")
    return ''.join(hex_str)

def hex_to_bits(hex_str):
    bits = []
    for ch in hex_str:
        val = int(ch, 16)
        bits.append((val >> 3) & 1)
        bits.append((val >> 2) & 1)
        bits.append((val >> 1) & 1)
        bits.append(val & 1)
    return bits

def encrypt():
    print("\nШИФРОВАНИЕ")
    key_text = input("Введите ключ (текст): ")
    try:
        frame_num = int(input("Введите номер кадра (целое число от 0 до 255): "))
    except ValueError:
        print("Ошибка: необходимо ввести целое число")
        return

    plaintext = input("Введите текст для шифрования: ")
    if not plaintext:
        print("Текст не введён")
        return

    key_bits = string_to_64bit_key(key_text)
    try:
        frame_bits = int_to_frame(frame_num)
    except ValueError as e:
        print(e)
        return

    plain_bits = text_to_bits(plaintext)
    keystream = a5_1_keystream(key_bits, frame_bits, len(plain_bits))
    cipher_bits = [plain_bits[i] ^ keystream[i] for i in range(len(plain_bits))]

    print("\nРЕЗУЛЬТАТ")
    print(f"Ключ (hex): {bits_to_hex(key_bits)}")
    print(f"Номер кадра: {frame_num}")
```

```
    print(f"Зашифрованный текст (hex): {bits_to_hex(cipher_bits)}")

def decrypt():
    print("\nРАСШИФРОВАНИЕ")
    key_text = input("Введите ключ (текст): ")
    try:
        frame_num = int(input("Введите номер кадра (целое число от 0 до 255): "))
    except ValueError:
        print("Ошибка: необходимо ввести целое число")
        return

    cipher_hex = input("Введите зашифрованный текст в hex: ").strip()
    if not cipher_hex:
        print("Текст не введён")
        return

    key_bits = string_to_64bit_key(key_text)
    try:
        frame_bits = int_to_frame(frame_num)
    except ValueError as e:
        print(e)
        return

    cipher_bits = hex_to_bits(cipher_hex)
    keystream = a5_1_keystream(key_bits, frame_bits, len(cipher_bits))
    plain_bits = [cipher_bits[i] ^ keystream[i] for i in range(len(cipher_bits))]
    plaintext = bits_to_text(plain_bits)

    print("")
    print(f"Ключ (hex): {bits_to_hex(key_bits)}")
    print(f"Номер кадра: {frame_num}")
    print(f"Расшифрованный текст: {plaintext}")

def main():
    while True:
        print("\ ШИФР A5/1")
        print("1. Зашифровать текст")
        print("2. Расшифровать текст")
        print("3. Выход")
        choice = input("Выберите действие: ")

        if choice == '1':
            encrypt()
        elif choice == '2':
            decrypt()
        elif choice == '3':
            print("Программа завершена.")
            break
        else:
            print("Неверный выбор, попробуйте снова.")

if __name__ == "__main__":
    main()
```

Результат работы программы:



Рисунок 28. Результат контрольного запуска шифра A5/1, вариант 20.

Контрольный пример пересчитан по текущему исходному коду A5/1.

Таблица 1. Параметры тестового запуска.

A5/1 — параметры короткого примера

Параметр	Значение
Ключ, введённый в программу	АВГУСТ
Ключ после UTF-8 (hex, первые 8 байт)	D090D092D093D0A3
Ключ (64 бита)	Биты: 1101000010010000110100001001001011010000100100111101000010100011
Номер кадра	155
Номер кадра (22 бита)	Биты: 0000000000000010011011
Открытый текст	ПЕРЕСДАЧА
Кодировка и длина	UTF-8: 18 байт = 144 бит
Двоичное представление текста	Биты: 110100001001111111010000100101011101000010100000110100001001010111010000101000011101000010010 1001101000010010000110100001010011111010000010010000
Гамма (hex)	D1842338C5696D732372388B7D231FE74044
Шифртекст (hex)	011BF3AD15C9BDE6F3D3E81FADB3CF4090D4

Примечание. string_to_64bit_key() берёт первые 8 байт UTF-8 и дополняет нулями, если их меньше 8. Для ключа «АВГУСТ» это байты первых четырёх букв. int_to_frame() ограничивает номер кадра диапазоном 0–255 и добавляет 14 нулевых старших битов.

Состояния записаны в порядке элементов массивов программы: reg[0] → reg[n−1], не в условной записи от старшего бита к младшему.

На каждом такте программа вычисляет z_i = R1[18] XOR R2[21] XOR R3[22], затем c_i = p_i XOR z_i. Сдвигаются только регистры, управляющий бит которых совпал с большинством.

Проверка: повторное расшифрование с тем же ключом и номером кадра возвращает исходное сообщение. Таблица воспроизводит поведение учебной программы и не заявляет о соответствии промышленной GSM-реализации A5/1.

Тест на 1000 знаков:

```
ШИФРОВАНИЕ
Введите ключ (текст): ключ
Введите номер кадра (целое число от 0 до 255): 15
Введите текст для шифрования: Ночь опускалась на город медленно, будто нехотя. Фонари зажигались один за другим, и их
 жёлтый свет ложился на мокрый асфальт длинными дрожащими полосами. Ветер гнал по улице обрывки газет и сухие листья,
 и они шелестели под ногами редких прохожих, спешивших домой. Анна стояла у окна и смотрела, как дождь чертит по стек
лу тонкие извилистые линии. Ей казалось, что где-то там, за этой пеленой, ещё живёт что-то важное, что она так и не у
спела понять за все эти годы. Внизу хлопнула дверь подъезда, и во двор вышел старик с собакой. Собака долго обнюхивал
а каждый куст, а старик терпеливо ждал, подняв воротник пальто и глядя в небо, где сквозь тучи пробивалась одинокая б
ledная звезда. Анна улыбнулась — впервые за весь день. Она налила себе чаю, села в кресло и раскрыла книгу, которую о
ткладывала уже целую неделю. За окном шумел дождь, в комнате тикали часы, и мир постепенно становился на место. В так
ие минуты кажется, что всё ещё можно исправить, что до утра далеко и что самое главное в жизни — это просто дождаться
 рассвета.

РЕЗУЛЬТАТ
Ключ (hex): D0BAD0BBD18ED187
Номер кадра: 15
Зашифрованный текст (hex): 4EE3568A982ED2D5254B3ECB107E3BCB87597D478297A13F61E78381CB52047A9C8DDB9AC6F8ED0BF6BEC5B18E
6A41E9F48517E2B5CC96EDDC75108AABDC8488143CF5C4ADF241A15AB57EA1D21450BD75A129D7F566848C983C18EC3D0AE1F40567F07E1337F04
B5D20B5685EFF0B99E4E9CE81FA438521987A9DF4A89A307F8BB9524D78D135EEEC830971CBDC3E6288E788680D0F79112BBDB8B408D046F8226C
8A6DEEAD2F7F9C7B247AF8E81282C8F75491B5E6975F720F1B1BEC17663809593B70723743B81112B8E1A453D9682E80C0CCEEB8FD98DFF767097
EBB726D96F1149F4BB3BFC24F03A1A0F1082B383451A5BBE1F73F20D6282448BDF7D4C795E0ED9849EFE4D798C3A6A6F9150BB7C207B255361719
69C413C35EB6ACB3EF09C8260AC2C246A85F2CBCCA54A859185C118D0F984E6D862B5B6B2C49474A48C9D611DD00F5D01CC9CD02A1077E48DCB5E
CD625C0AA27EF3025E3D7082C59617A638789814AE04ADECB544C63A4B3DC5C58BC1F15F1AAA2D32EC1CAB1954F2B2FD7855D791489FAA9209299
8DC032B36431931F799F6BD274E6A6F0CCE4BC5A7C84765D46E480A45B122165B978E2C30435734C64554B8E851BA715CD8D32B9550941A589BF1
D312F6251CCB57C5AD6845CD924C9B66A2CD8072EFEF330E1FB897CA86CB60F92A6C4A8C664CDCD51A8B02932BC6AEFF836D03FFD6C6A2E745EE2
2CC4DFF6FAF37C0DE4A8616A76E3D7F2D6A1BD4473004E944FE2BA92D3A8D9E4D1001E787CA2D8C5BDC750EBDAE283F3CDC1D8402F1F9B1FB06BC
7BAA3E187FCBB7A11CFDBC3DBA5FB232D45C67581147941D2A6E2D50A01B8E034F1BD26D43F0898D5264C384283F437F93E6FFE48D4FEE93AAD1B
902A5D8CF8CD3531A62EBA8980A693930B5EEF011BB3971742A7447A07C28AB1EFF3AF8AF328C6FDA1CD474149EA53301CFA9A28A302537BA862B
22DFA07F2611FD0BEF64F75DBBAAC85F2E2BCC58D505D1B490541F7BDE6F5A724639D8AAF7449163B6C5F7FE60E3F812FA932561FF3916AAC3DE8
809BD23C5CD1AC4A109D8288E98203C9057C88963EC5D4367E1E0ABA1C19CD6935DF27398E6AC417726D524BC378771B920A87D0006B60ECEE3B6
159B5329CDE59DC12071102C931129E28E3BE647A8951EC6E375F04FB73D9F739C31509905F25CBD4A05B322E8B7CA1F9851C6EC80A7C315DE8E9
BB56A0C31B81CD79DD5AC83E8F2DD8A221129E101E6195152CEBAA1339D9E9B60C9B1EC891345B428B174BB19777CC86FEB13B77DF34E9A428125
23D177F148CADF8AA762C488F83928504E0C395424D969B5A9A60723D069C5125C350B58014FA0F4582AAAEFB854F6D7240059281A40ECD42A44F
3C37EE54453E37A63C4665783AD1BDAFFD02BD07DCF170BA39988E2898CF9F3B932BD536C94A5B3156362AD90E975E60339780FE09A218139A18E
```

```
E48DCB5ECD625C0AA27EF3025E3D7082C59617A638789814AE04ADECB544C63A4B3DC5C58BC1F15F1AAA2D32EC1CAB1954F2B2FD7855D791489FA
A92092998DC032B36431931F799F6BD274E6A6F0CCE4BC5A7C84765D46E480A45B122165B978E2C30435734C64554B8E851BA715CD8D32B955094
1A589BF1D312F6251CCB57C5AD6845CD924C9B66A2CD8072EFEF330E1FB897CA86CB60F92A6C4A8C664CDCD51A8B02932BC6AEFF836D03FFD6C6A
2E745EE22CC4DFF6FAF37C0DE4A8616A76E3D7F2D6A1BD4473004E944FE2BA92D3A8D9E4D1001E787CA2D8C5BDC750EBDAE283F3CDC1D8402F1F9
B1FB06BC7BAA3E187FCBB7A11CFDBC3DBA5FB232D45C67581147941D2A6E2D50A01B8E034F1BD26D43F0898D5264C384283F437F93E6FFE48D4FE
E93AAD1B902A5D8CF8CD3531A62EBA8980A693930B5EEF011BB3971742A7447A07C28AB1EFF3AF8AF328C6FDA1CD474149EA53301CFA9A28A3025
37BA862B22DFA07F2611FD0BEF64F75DBBAAC85F2E2BCC58D505D1B490541F7BDE6F5A724639D8AAF7449163B6C5F7FE60E3F812FA932561FF391
6AAC3DE8809BD23C5CD1AC4A109D8288E98203C9057C88963EC5D4367E1E0ABA1C19CD6935DF27398E6AC417726D524BC378771B920A87D0006B6
0ECEE3B6159B5329CDE59DC12071102C931129E28E3BE647A8951EC6E375F04FB73D9F739C31509905F25CBD4A05B322E8B7CA1F9851C6EC80A7C
315DE8E9BB56A0C31B81CD79DD5AC83E8F2DD8A221129E101E6195152CEBAA1339D9E9B60C9B1EC891345B428B174BB19777CC86FEB13B77DF34E
9A42812523D177F148CADF8AA762C488F83928504E0C395424D969B5A9A60723D069C5125C350B58014FA0F4582AAAEFB854F6D7240059281A40E
CD42A44F3C37EE54453E37A63C4665783AD1BDAFFD02BD07DCF170BA39988E2898CF9F3B932BD536C94A5B3156362AD90E975E60339780FE09A21
8139A18E828B1731EB2CB86EED8EB4B901EAFD4A49F6B3DC41879818926C846D980F765B4A56679D1F873ED37A89F743ACA7D0566E3FF92C7B38F
C2D4D23B70779C3AD514CD41254BE7C5E7DF0BEE60A86C56433E8C1272053E079232FBB02DF2F4C1ACC636B9C0DF52A67F8088FB185F0D53FF9BB
24966BB117153B938C569155234792AAA306084042C132EA63ADAA182EAC94E217C28E814AB9FB01DF7BA335640A68121EDB4615F52804EE70A9B
370DBD6F3AD5BD79E544A4A7F40641678A0FF9D296CEF47DEC0DAFD9F9E25572E5EB5E4FB0EBE0BCEF7B597095B1389CCE383957A468F9EC45B02
6C82974ABE5AF9A7557CF3F4E5AB9EEDE4EBB3B6FB6500A821FAFD2699728C033163ABD1F9E88FD251D1E5BDC45FE7B1265A1BB10D1AE863B07A1
82C265D7ED358A974E47171E638E3041C5B9B912EA643C4E1B4A7CC0FBDF2FCA56A61FBABE1E95F4E08EEACD0D643E597200C251A41F334784DA7
05DD5095182255170ED8895A3DF9689FC53542FB86B8EFB959EE601A266A50F3814FC12769297EC0F0FB7C68B9F3D1BD55E2FBE1FF7D19539E48C
67D58DAA5D3891234C391B250A155B83913DCD447991CB6C8464EA681AE3B24804F179F2EA1F6BCC66AC6BCAEEFA8337A88C411F154AD7CC3D69D
8F4F3B3DD792D7AB85A2F7767375F14B9F75195C8A4C44B417311817F35BD3830D101BD68959D22C9491D1B0D44725CFD743710BBCE92F2D21332
8ADBF2412E585F22E7A2BC776C161C2C303216A47B527B95DB8BD0E3C27C79CD73124F3860CF032AAA1EEA3B6F3C63BB5F55C86C0B4ECCFB55FF5
270CF9A308E6AB660207B09BE526F8F18A6E4E0FC273455E92E04E2C19DCCFA6F15044DB1D6B1EFEF965E0681A1BD0E97C3B14991D5166CB83066
D6793C84EF84D883F025FB70FE85DD420395F6B40050C613CA32BBAABB040EB90EEFE84DC06F39CEA6246060BFFB1ACF7089EC152A69488959AA3
321119C631D82921C8D8F9809D5BA19E24BC40BEE1C98454831401D10B9715506C7BFDFE94F342A91643B583BDB83762B0E6F3E74ACBFBD0D8930
8BBA3B9F9D0DF1D9C5A3B5149AA57CBAAC9A38E50A1DDCFD36C2B0F054D74A7D7D397753621C6E2A66099E5EB2604E123CD2F59FBA02031461D78
E230AEC08FAD25BD6CF5E

Ключ (hex): D0BAD0BBD18ED187
Номер кадра: 15
Расшифрованный текст: Ночь опускалась на город медленно, будто нехотя. Фонари зажигались один за другим, и их жёлтый
свет ложился на мокрый асфальт длинными дрожащими полосами. Ветер гнал по улице обрывки газет и сухие листья, и они ш
елестели под ногами редких прохожих, спешивших домой. Анна стояла у окна и смотрела, как дождь чертит по стеклу тонки
е извилистые линии. Ей казалось, что где-то там, за этой пеленой, ещё живёт что-то важное, что она так и не успела по
нять за все эти годы. Внизу хлопнула дверь подъезда, и во двор вышел старик с собакой. Собака долго обнюхивала каждый
 куст, а старик терпеливо ждал, подняв воротник пальто и глядя в небо, где сквозь тучи пробивалась одинокая бледная з
везда. Анна улыбнулась — впервые за весь день. Она налила себе чаю, села в кресло и раскрыла книгу, которую откладыва
ла уже целую неделю. За окном шумел дождь, в комнате тикали часы, и мир постепенно становился на место. В такие минут
ы кажется, что всё ещё можно исправить, что до утра далеко и что самое главное в жизни — это просто дождаться рассвет
а.
\ ШИФР A5/1
```

Рисунок 29. Тест на 1000 знаков шифра A5/1.

Блок G: Комбинационные шифры

19. Шифр AES

AES (Advanced Encryption Standard) — это симметричный блочный шифр, принятый в качестве стандарта шифрования в США в 2001 году. Он пришёл на смену устаревшему DES (Data Encryption Standard). AES является одним из наиболее распространённых алгоритмов симметричного шифрования в мире, используется в протоколах SSL/TLS, Wi-Fi (WPA2), файловых архиваторах, банковских системах и во многих других приложениях.

Ключевая особенность: AES работает с блоками данных фиксированного размера (128 бит = 16 байт) и поддерживает ключи длиной 128, 192 или 256 бит.

Блок-схема:

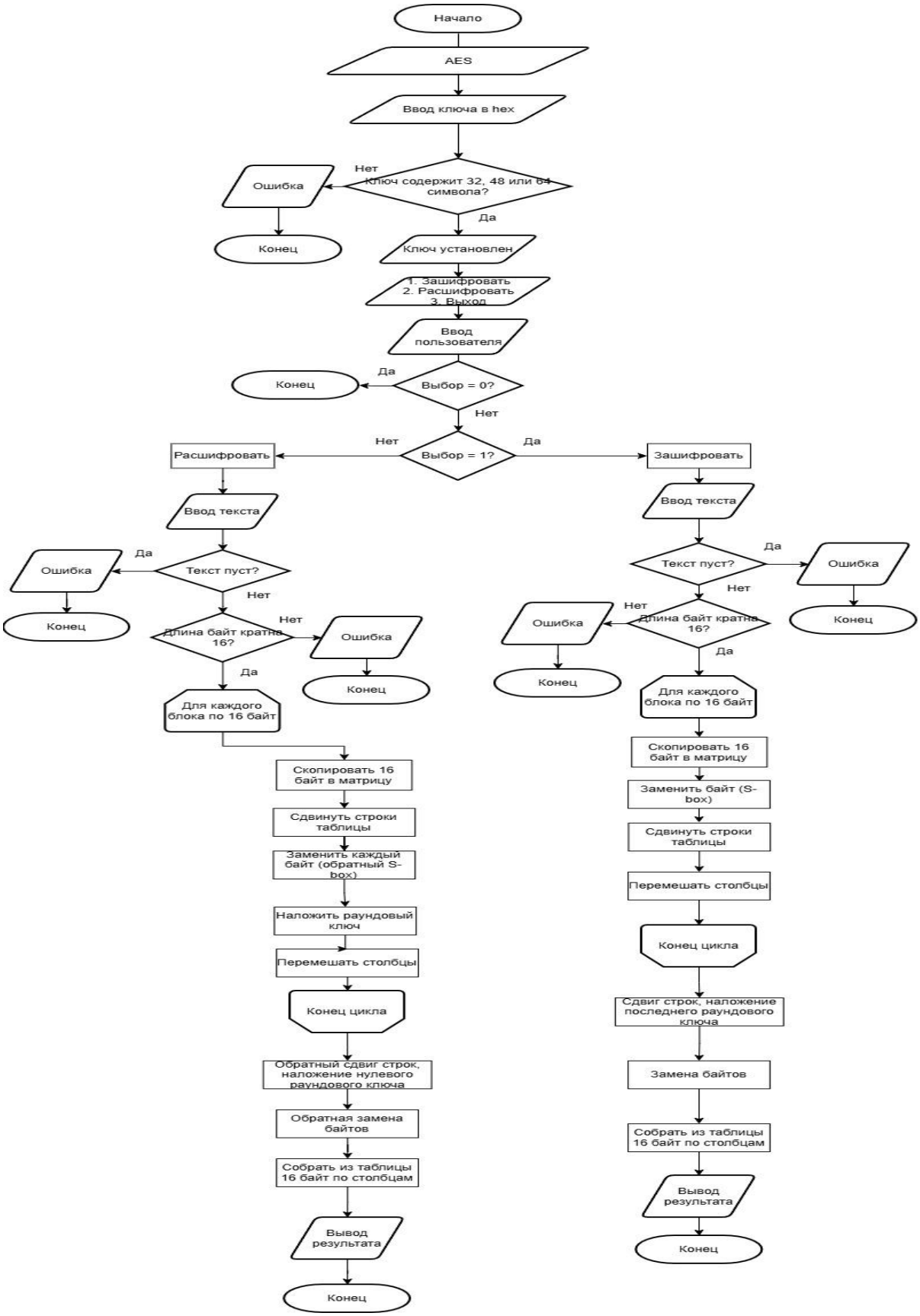


Рисунок 30. Блок-схема шифра AES.

Код программы:

```
import sys

# Таблицы S-box, обратный S-box и Rcon (как в оригинале)
SBOX = [
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16
]

INV_SBOX = [
    0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf, 0x40, 0xa3, 0x9e, 0x81, 0xf3, 0xd7, 0xfb,
    0x7c, 0xe3, 0x39, 0x82, 0x9b, 0x2f, 0xff, 0x87, 0x34, 0x8e, 0x43, 0x44, 0xc4, 0xde, 0xe9, 0xcb,
    0x54, 0x7b, 0x94, 0x32, 0xa6, 0xc2, 0x23, 0x3d, 0xee, 0x4c, 0x95, 0x0b, 0x42, 0xfa, 0xc3, 0x4e,
    0x08, 0x2e, 0xa1, 0x66, 0x28, 0xd9, 0x24, 0xb2, 0x76, 0x5b, 0xa2, 0x49, 0x6d, 0x8b, 0xd1, 0x25,
    0x72, 0xf8, 0xf6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xd4, 0xa4, 0x5c, 0xcc, 0x5d, 0x65, 0xb6, 0x92,
    0x6c, 0x70, 0x48, 0x50, 0xfd, 0xed, 0xb9, 0xda, 0x5e, 0x15, 0x46, 0x57, 0xa7, 0x8d, 0x9d, 0x84,
    0x90, 0xd8, 0xab, 0x00, 0x8c, 0xbc, 0xd3, 0x0a, 0xf7, 0xe4, 0x58, 0x05, 0xb8, 0xb3, 0x45, 0x06,
    0xd0, 0x2c, 0x1e, 0x8f, 0xca, 0x3f, 0x0f, 0x02, 0xc1, 0xaf, 0xbd, 0x03, 0x01, 0x13, 0x8a, 0x6b,
    0x3a, 0x91, 0x11, 0x41, 0x4f, 0x67, 0xdc, 0xea, 0x97, 0xf2, 0xcf, 0xce, 0xf0, 0xb4, 0xe6, 0x73,
    0x96, 0xac, 0x74, 0x22, 0xe7, 0xad, 0x35, 0x85, 0xe2, 0xf9, 0x37, 0xe8, 0x1c, 0x75, 0xdf, 0x6e,
    0x47, 0xf1, 0x1a, 0x71, 0x1d, 0x29, 0xc5, 0x89, 0x6f, 0xb7, 0x62, 0x0e, 0xaa, 0x18, 0xbe, 0x1b,
    0xfc, 0x56, 0x3e, 0x4b, 0xc6, 0xd2, 0x79, 0x20, 0x9a, 0xdb, 0xc0, 0xfe, 0x78, 0xcd, 0x5a, 0xf4,
    0x1f, 0xdd, 0xa8, 0x33, 0x88, 0x07, 0xc7, 0x31, 0xb1, 0x12, 0x10, 0x59, 0x27, 0x80, 0xec, 0x5f,
    0x60, 0x51, 0x7f, 0xa9, 0x19, 0xb5, 0x4a, 0x0d, 0x2d, 0xe5, 0x7a, 0x9f, 0x93, 0xc9, 0x9c, 0xef,
    0xa0, 0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5, 0xb0, 0xc8, 0xeb, 0xbb, 0x3c, 0x83, 0x53, 0x99, 0x61,
    0x17, 0x2b, 0x04, 0x7e, 0xba, 0x77, 0xd6, 0x26, 0xe1, 0x69, 0x14, 0x63, 0x55, 0x21, 0x0c, 0x7d
]

RCON = [0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1b, 0x36]

def xtime(x):
    return ((x << 1) ^ (0x1b if (x & 0x80) else 0x00)) & 0xff

class AES:
    def __init__(self):
        self.Nk = 0
        self.Nr = 0
        self.w = [0] * (60 * 4)

    def set_key(self, key):
        key_len = len(key)
```

```
    self.Nk = key_len // 4
    self.Nr = {4: 10, 6: 12, 8: 14}[self.Nk]
    for i in range(self.Nk):
        self.w[4*i + 0] = key[4*i + 0]
        self.w[4*i + 1] = key[4*i + 1]
        self.w[4*i + 2] = key[4*i + 2]
        self.w[4*i + 3] = key[4*i + 3]
    for i in range(self.Nk, 4 * (self.Nr + 1)):
        temp = [self.w[4*(i-1)+0], self.w[4*(i-1)+1], self.w[4*(i-1)+2], self.w[4*(i-1)+3]]
        if i % self.Nk == 0:
            temp = [temp[1], temp[2], temp[3], temp[0]]
            temp = [SBOX[b] for b in temp]
            temp[0] ^= RCON[i // self.Nk]
        elif self.Nk > 6 and i % self.Nk == 4:
            temp = [SBOX[b] for b in temp]
        self.w[4*i + 0] = self.w[4*(i - self.Nk) + 0] ^ temp[0]
        self.w[4*i + 1] = self.w[4*(i - self.Nk) + 1] ^ temp[1]
        self.w[4*i + 2] = self.w[4*(i - self.Nk) + 2] ^ temp[2]
        self.w[4*i + 3] = self.w[4*(i - self.Nk) + 3] ^ temp[3]

def add_round_key(self, state, rnd):
    for i in range(4):
        for j in range(4):
            state[j][i] ^= self.w[4 * (rnd * 4 + i) + j]

def sub_bytes(self, state):
    for i in range(4):
        for j in range(4):
            state[i][j] = SBOX[state[i][j]]

def inv_sub_bytes(self, state):
    for i in range(4):
        for j in range(4):
            state[i][j] = INV_SBOX[state[i][j]]

def shift_rows(self, state):
    state[1] = state[1][1:] + state[1][:1]
    state[2] = state[2][2:] + state[2][:2]
    state[3] = state[3][3:] + state[3][:3]

def inv_shift_rows(self, state):
    state[1] = state[1][-1:] + state[1][:-1]
    state[2] = state[2][-2:] + state[2][:-2]
    state[3] = state[3][-3:] + state[3][:-3]

def mix_columns(self, state):
    for j in range(4):
        a = state[0][j]
        b = state[1][j]
        c = state[2][j]
        d = state[3][j]
        state[0][j] = xtime(a) ^ xtime(b) ^ b ^ c ^ d
        state[1][j] = a ^ xtime(b) ^ xtime(c) ^ c ^ d
        state[2][j] = a ^ b ^ xtime(c) ^ xtime(d) ^ d
        state[3][j] = xtime(a) ^ a ^ b ^ c ^ xtime(d)
```

```
def inv_mix_columns(self, state):
    """Обратное преобразование столбцов AES по матрице 14/11/13/9."""
    def gf_mul(value, factor):
        result = 0
        while factor:
            if factor & 1:
                result ^= value
            value = xtime(value)
            factor >>= 1
        return result

    for col in range(4):
        a, b, c, d = (state[row][col] for row in range(4))
        state[0][col] = gf_mul(a, 14) ^ gf_mul(b, 11) ^ gf_mul(c, 13) ^ gf_mul(d, 9)
        state[1][col] = gf_mul(a, 9) ^ gf_mul(b, 14) ^ gf_mul(c, 11) ^ gf_mul(d, 13)
        state[2][col] = gf_mul(a, 13) ^ gf_mul(b, 9) ^ gf_mul(c, 14) ^ gf_mul(d, 11)
        state[3][col] = gf_mul(a, 11) ^ gf_mul(b, 13) ^ gf_mul(c, 9) ^ gf_mul(d, 14)

def encrypt_block(self, plain):
    state = [[0]*4 for _ in range(4)]
    for i in range(4):
        for j in range(4):
            state[j][i] = plain[i*4 + j]
    self.add_round_key(state, 0)
    for rnd in range(1, self.Nr):
        self.sub_bytes(state)
        self.shift_rows(state)
        self.mix_columns(state)
        self.add_round_key(state, rnd)
    self.sub_bytes(state)
    self.shift_rows(state)
    self.add_round_key(state, self.Nr)
    out = bytearray(16)
    for i in range(4):
        for j in range(4):
            out[i*4 + j] = state[j][i]
    return bytes(out)

def decrypt_block(self, cipher):
    state = [[0]*4 for _ in range(4)]
    for i in range(4):
        for j in range(4):
            state[j][i] = cipher[i*4 + j]
    self.add_round_key(state, self.Nr)
    for rnd in range(self.Nr - 1, 0, -1):
        self.inv_shift_rows(state)
        self.inv_sub_bytes(state)
        self.add_round_key(state, rnd)
        self.inv_mix_columns(state)
    self.inv_shift_rows(state)
    self.inv_sub_bytes(state)
    self.add_round_key(state, 0)
    out = bytearray(16)
    for i in range(4):
        for j in range(4):
            out[i*4 + j] = state[j][i]
```

```
        return bytes(out)

class AESCipher:
    def __init__(self):
        self.aes = AES()
        self.key_set = False

    def set_key_from_hex(self, key_hex):
        key_hex = key_hex.strip().lower()
        if len(key_hex) not in (32, 48, 64):
            print("Ошибка: длина ключа должна быть 32, 48 или 64 hex-символа (16, 24 или 32 байта).")
            return False
        try:
            key_bytes = bytes.fromhex(key_hex)
        except ValueError:
            print("Ошибка: неверный hex-формат ключа.")
            return False
        self.aes.set_key(key_bytes)
        self.key_set = True
        bits = len(key_bytes) * 8
        print(f"Ключ установлен ({bits} бит): {key_hex.upper()}")
        return True

    def encrypt_hex(self, plain_hex):
        if not self.key_set:
            print("Ошибка: ключ не установлен!")
            return ""
        try:
            data = bytes.fromhex(plain_hex)
        except ValueError:
            print("Ошибка: неверный hex-формат открытого текста.")
            return ""
        # Требуем, чтобы длина данных была кратна 16 байтам
        if len(data) % 16 != 0:
            print("Ошибка: длина открытого текста должна быть кратна 16 байтам (32 hex-символа).")
            return ""
        cipher = bytearray()
        for i in range(0, len(data), 16):
            block = data[i:i+16]
            enc_block = self.aes.encrypt_block(block)
            cipher.extend(enc_block)
        return cipher.hex().upper()

    def decrypt_hex(self, cipher_hex):
        if not self.key_set:
            print("Ошибка: ключ не установлен!")
            return ""
        try:
            cipher = bytes.fromhex(cipher_hex)
        except ValueError:
            print("Ошибка: неверный hex-формат зашифрованного текста.")
            return ""
        if len(cipher) % 16 != 0:
            print("Ошибка: длина зашифрованного текста должна быть кратна 16 байтам.")
            return ""
        plain = bytearray()
```

```
        for i in range(0, len(cipher), 16):
            block = cipher[i:i+16]
            dec_block = self.aes.decrypt_block(block)
            plain.extend(dec_block)
        return plain.hex().upper()

def main():
    cipher = AESCipher()
    while True:
        print("\nAES")
        print("1. Установить ключ (hex)")
        print("2. Зашифровать hex")
        print("3. Расшифровать hex")
        print("4. Выход")
        choice = input("Ваш выбор: ").strip()
        if choice == '1':
            key_hex = input("Введите ключ в hex: ").strip()
            cipher.set_key_from_hex(key_hex)
        elif choice == '2':
            plain_hex = input("Введите открытый текст в hex: ").strip()
            enc = cipher.encrypt_hex(plain_hex)
            if enc:
                print(f"Зашифрованный текст: {enc}")
        elif choice == '3':
            cipher_hex = input("Введите зашифрованный текст в hex: ").strip()
            dec = cipher.decrypt_hex(cipher_hex)
            if dec:
                print(f"Расшифрованный текст: {dec}")
        elif choice == '4':
            print("Программа завершена.")
            break
        else:
            print("Неверный выбор.")

if __name__ == "__main__":
    main()
```

Результат работы программы:

```
Python console test

AES — фактический вывод контрольного теста
Ключ AES-128: 000102030405060708090A0B0C0D0E0F
Вариант 20: Леопард не может изменить своих пятен.
Открытый блок после дополнения до 16 байт: D09BD0B5D0BED0BFD0B0D180D0B420D0BDD0B520D0BCD0BED0B6D0B5D18220D0B8D0B
    7D0BCD0B5D0BDD0B8D182D18C20D181D0B2D0BED0B8D18520D0BFD18FD182D0B5D0BD2E20202020202020202020
Шифртекст: 64CDBFD97621FD76329A1764104D09CEACEF78E0F00ED60E639CCCD66FF50DC4E7846C721989129BAFEB28FC69A0D3E1D8302
    7F21121B36B5AB9A49917C10E9080248B86D96DD5B390797971DBCB917F
После обратного преобразования: Леопард не может изменить своих пятен.
Итог: текст варианта восстановлен
Контрольный вектор AES-128: 69C4E0D86A7B0430D8CDB78070B4C55A
Расшифрование вектора совпало: True
```

Рисунок 31. Результат контрольного запуска AES, вариант 20.

Блок H: Ассиметричные шифры

21. Шифр RSA

RSA — это один из первых и наиболее известных асимметричных криптографических алгоритмов, названный в честь его создателей: Рона Ривеста, Ади Шамира и Леонарда Адлемана (1977 год). Алгоритм основан на вычислительной сложности факторизации больших целых чисел (разложения числа на простые множители).

Ключевая особенность: RSA использует два ключа: открытый (для шифрования) и закрытый (для расшифрования). Невозможно вычислить закрытый ключ из открытого за разумное время при достаточно больших размерах ключа.

Блок-схема:

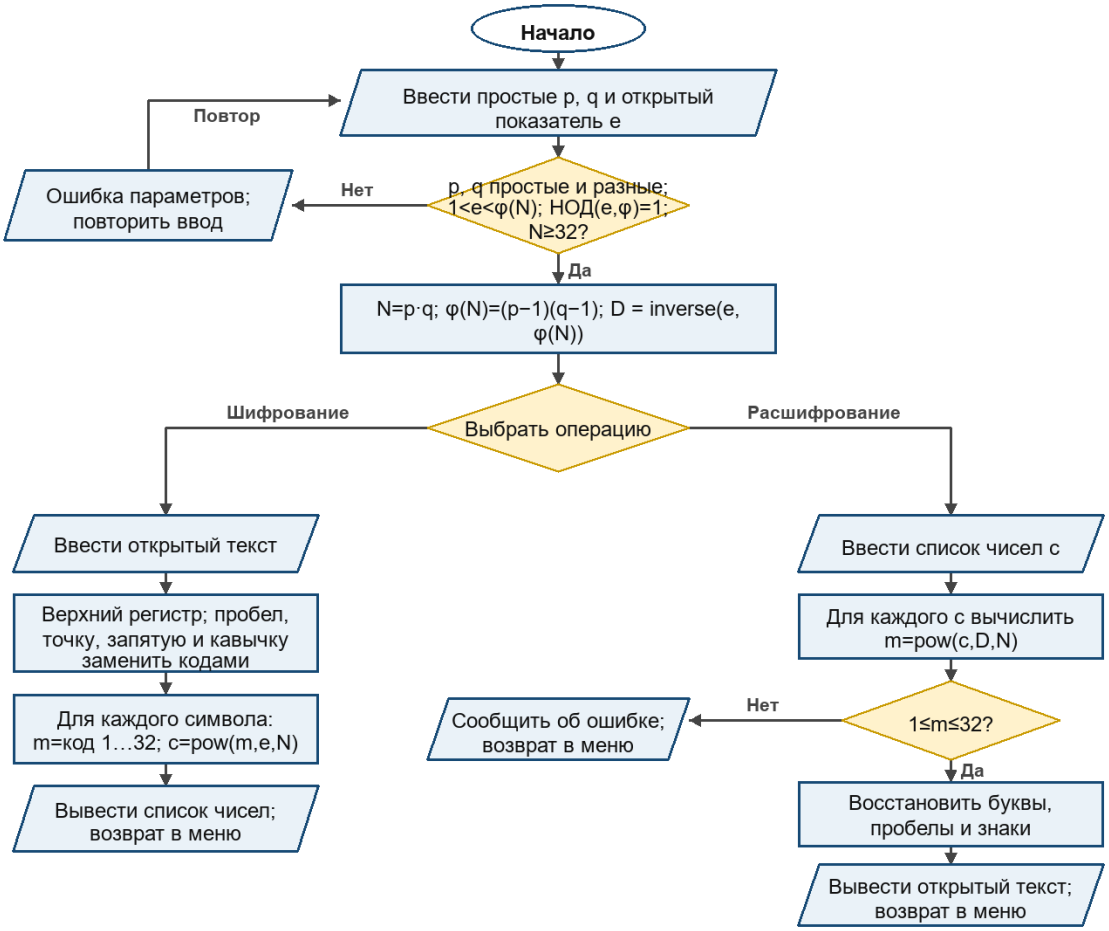


Рисунок 32. Блок-схема шифра RSA.

Код программы:

```
import sys
import math

# ---- Вспомогательные математические функции ----
def mod_pow(a, exp, mod):
    """Быстрое возведение в степень по модулю"""
    result = 1
    a %= mod
    while exp > 0:
        if exp & 1:
            result = (result * a) % mod
        a = (a * a) % mod
        exp >>= 1
    return result

def mod_inverse(a, m):
    """Расширенный алгоритм Евклида для нахождения обратного элемента"""
    m0, x0, x1 = m, 0, 1
    if m == 1:
        return 0
    while a > 1:
        q = a // m
        a, m = m, a % m
        x0, x1 = x1 - q * x0, x0
    return x1 + m0 if x1 < 0 else x1

def is_prime(n):
    """Проверка простоты (для небольших чисел)"""
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    i = 3
    while i * i <= n:
        if n % i == 0:
            return False
        i += 2
    return True

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

class RSACipher:
    def __init__(self):
        self.p = self.q = self.N = self.phi = self.E = self.D = 0
        self.key_set = False
        self.alphabet = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"   # 32 буквы

    def to_upper_russian(self, c):
        """Приведение русской буквы к верхнему регистру, Ё -> Е"""
        if 'а' <= c <= 'я' or 'А' <= c <= 'Я':
            return c.upper()
        if c == 'ё' or c == 'Ё':
            return 'Е'
        return c

    def is_russian_char(self, c):
        """Проверка, является ли символ русской буквой (включая Ё)"""
        return ('А' <= c <= 'Я') or ('а' <= c <= 'я') or c in ('Ё', 'ё')

    def prepare_text(self, text):
        """Подготовка текста: пробел -> ПРБ, точка -> ТЧК, запятая -> ЗПТ, кавычка -> КВЧ"""
        result = []
        for ch in text:
            if ch == ' ':
                result.append("ПРБ")
            elif ch == '.':
                result.append("ТЧК")
            elif ch == ',':
                result.append("ЗПТ")
            elif ch == '"':
                result.append("КВЧ")
            else:
                upper = self.to_upper_russian(ch)
                if self.is_russian_char(upper):
                    result.append(upper)
        return ''.join(result)

    def restore_text(self, text):
        """Восстановление пробелов, точек, запятых и кавычек после расшифровки"""
        result = []
        i = 0
        n = len(text)
        while i < n:
            if i + 2 < n:
                triple = text[i:i+3]
                if triple == "ПРБ":
```

```
                result.append(' ')
                i += 3
                continue
            elif triple == "ТЧК":
                result.append('.')
                i += 3
                continue
            elif triple == "ЗПТ":
                result.append(',')
                i += 3
                continue
            elif triple == "КВЧ":
                result.append('"')
                i += 3
                continue
        result.append(text[i])
        i += 1
    return ''.join(result)
def char_to_int(self, c):
    """Буква -> число 1..32 (нумерация с 1)"""
    try:
        return self.alphabet.index(c) + 1
    except ValueError:
        return -1
def int_to_char(self, n):
    """Число 1..32 -> буква"""
    if 1 <= n <= len(self.alphabet):
        return self.alphabet[n-1]
    return '?'
# ---- Установка ключа ----
def set_key(self, prime_p, prime_q, exp_e):
    if not is_prime(prime_p):
        raise ValueError("p не является простым числом.")
    if not is_prime(prime_q):
        raise ValueError("q не является простым числом.")
    if prime_p == prime_q:
        raise ValueError("p и q должны быть разными.")
    N = prime_p * prime_q
    phi = (prime_p - 1) * (prime_q - 1)
    if exp_e <= 1 or exp_e >= phi:
        raise ValueError("E должно быть в диапазоне 1 < E < ϕ(N).")
    if gcd(exp_e, phi) != 1:
        raise ValueError("E и ϕ(N) должны быть взаимно просты.")
    if N < 32:
        raise ValueError(f"Модуль N = {N} слишком мал для кодирования алфавита.")
    private_exponent = mod_inverse(exp_e, phi)
    if exp_e == private_exponent:
        raise ValueError("Открытый и секретный показатели совпадают; выберите другое E.")
    self.p = prime_p
    self.q = prime_q
    self.N = N
    self.phi = phi
    self.E = exp_e
    self.D = private_exponent
    self.key_set = True
    print("\nКЛЮЧИ УСТАНОВЛЕНЫ")
    print(f"p = {self.p}, q = {self.q}")
    print(f"N = {self.N}")
    print(f"ϕ(N) = {self.phi}")
    print(f"Открытый ключ (E, N) = ({self.E}, {self.N})")
    print(f"Секретный ключ D = {self.D}")
    return True
# ---- Шифрование ----
def encrypt(self, plain_text):
    if not self.key_set:
        print("Ошибка: ключ не установлен!")
        return ""
    prepared = self.prepare_text(plain_text)
    if not prepared:
        print("Ошибка: текст пуст!")
        return ""
    print(f"\nТекст после обработки: {prepared}")
    cipher_nums = []
    print("\n--- ШИФРОВАНИЕ ---")
    print("Символ -> Код -> Шифр")
    print("-------------------------------")
    for ch in prepared:
        m = self.char_to_int(ch)
        if m == -1:
            print(f"Ошибка: символ {ch} не найден в алфавите!")
            return ""
        ciph = mod_pow(m, self.E, self.N)
        cipher_nums.append(ciph)
```

```
            print(f"   {ch}    ->    {m:2d}     -> {ciph}")
        # Формируем строку с числами через пробел
        result = ' '.join(str(x) for x in cipher_nums)
        print("\nЗАШИФРОВАННЫЙ ТЕКСТ")
        print(result)
        return result
    # ---- Расшифровка ----
    def decrypt(self, cipher_text):
        if not self.key_set:
            print("Ошибка: ключ не установлен!")
            return ""
        # Разбираем строку с числами, разделёнными пробелами
        parts = cipher_text.strip().split()
        if not parts:
            print("Ошибка: не удалось разобрать зашифрованный текст!")
            return ""
        try:
            cipher_nums = [int(x) for x in parts]
        except ValueError:
            print("Ошибка: зашифрованный текст должен содержать только целые числа!")
            return ""
        print("\n--- РАСШИФРОВАНИЕ ---")
        print("Шифр -> Код -> Символ")
        print("------------------------------")
        decrypted_chars = []
        for ciph in cipher_nums:
            m = mod_pow(ciph, self.D, self.N)
            if m < 1 or m > len(self.alphabet):
                print(f"Ошибка: расшифрованное число {m} вне диапазона 1..32!")
                return ""
            ch = self.int_to_char(m)
            decrypted_chars.append(ch)
            print(f"{ciph} ->    {m:2d}     -> {ch}")
        decrypted_str = ''.join(decrypted_chars)
        result = self.restore_text(decrypted_str)
        print("\nРАСШИФРОВАННЫЙ ТЕКСТ")
        print(result)
        return result

def process_user_input(cipher):
    print("\nRSA")
    print("1. Установить ключ (p, q, e)")
    print("2. Зашифровать текст")
    print("3. Расшифровать текст")
    print("4. Выход")
    choice = input("Ваш выбор: ").strip()
    if choice == '1':
        try:
            p = int(input("Введите простое число p (например, 23): "))
            q = int(input("Введите простое число q (например, 29): "))
            e = int(input("Введите открытый ключ e (например, 3 или 65537): "))
            cipher.set_key(p, q, e)
        except ValueError:
            print("Ошибка: введите целые числа.")
    elif choice == '2':
        text = input("Введите текст для шифрования: ")
        cipher.encrypt(text)
    elif choice == '3':
        text = input("Введите зашифрованный текст (числа через пробел): ")
        cipher.decrypt(text)
    elif choice == '4':
        print("Программа завершена.")
        sys.exit(0)
    else:
        print("Неверный выбор!")

def main():
    cipher = RSACipher()
    while True:
        process_user_input(cipher)

if __name__ == "__main__":
    main()
```

Результат работы программы:

```
Python — контрольный запуск

RSA — фактический вывод реализации из репозитория
Вариант 20: текстовая проверка
Исходный текст: Леопард не может изменить своих пятен.
Параметры: p=61, q=53, e=17, N=3233, ϕ(N)=3120
Вычисленный закрытый показатель: D=2753
Открытый ключ (E, N): (17, 3233)
Выполнено шифрование и расшифрование по коду RSA.
Результат: ЛЕОПАРД НЕ МОЖЕТ ИЗМЕНИТЬ СВОИХ ПЯТЕН.
Проверка: результат совпадает с исходным текстом
```

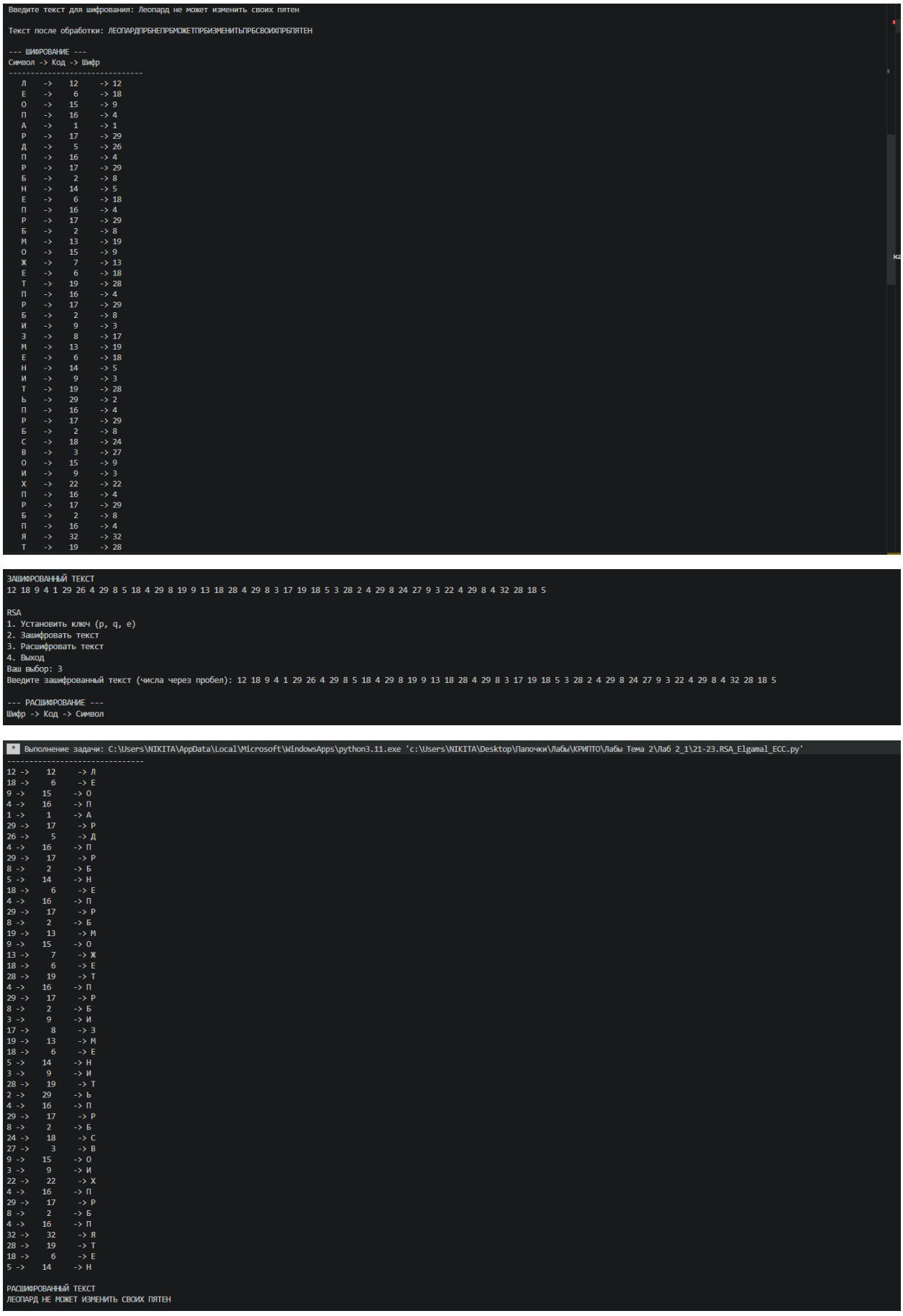


Рисунок 33. Результат контрольного запуска RSA, вариант 20.

Результат шифра в тетради:

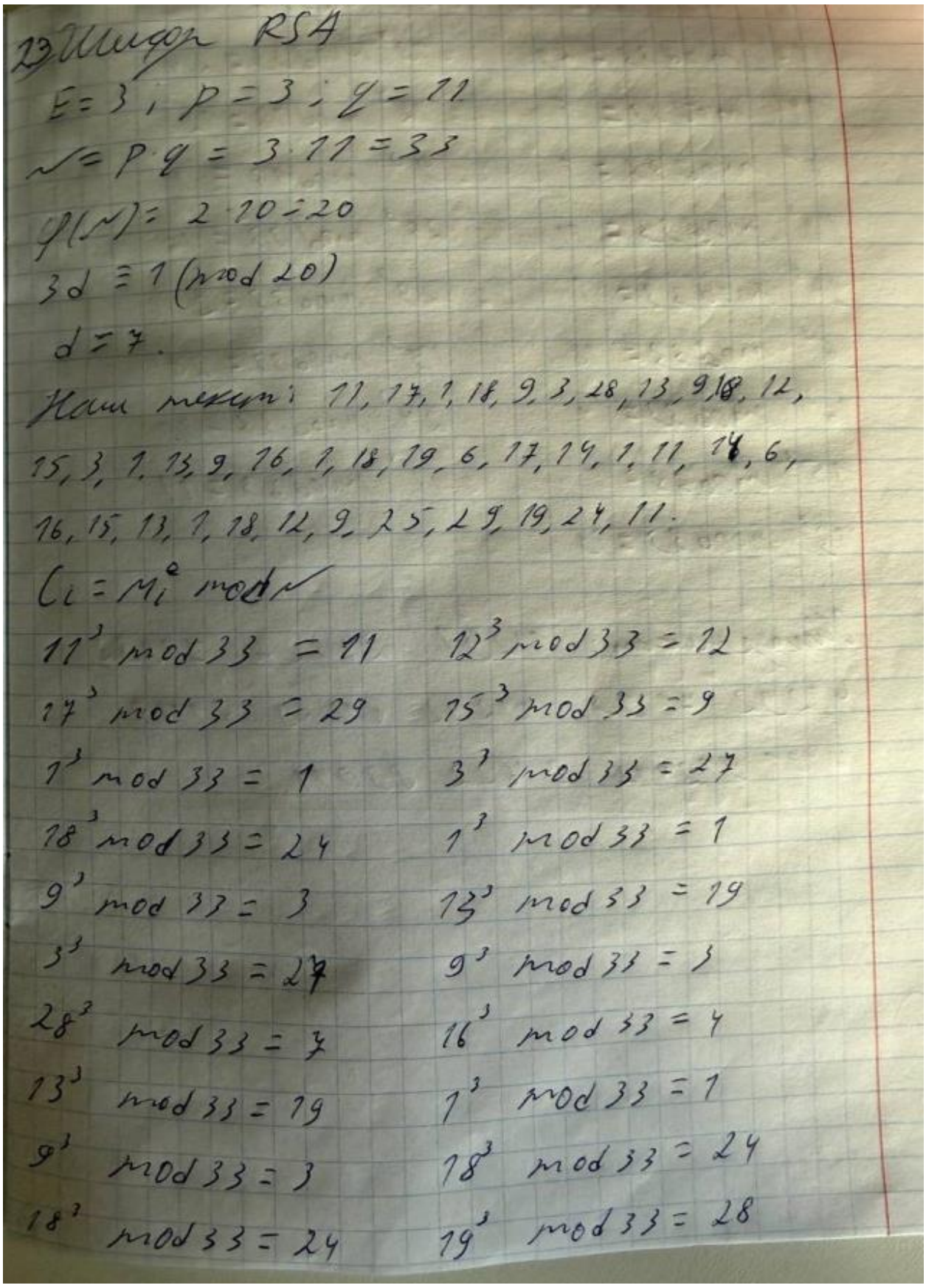


Рисунок 34. Шифр RSA в тетради.

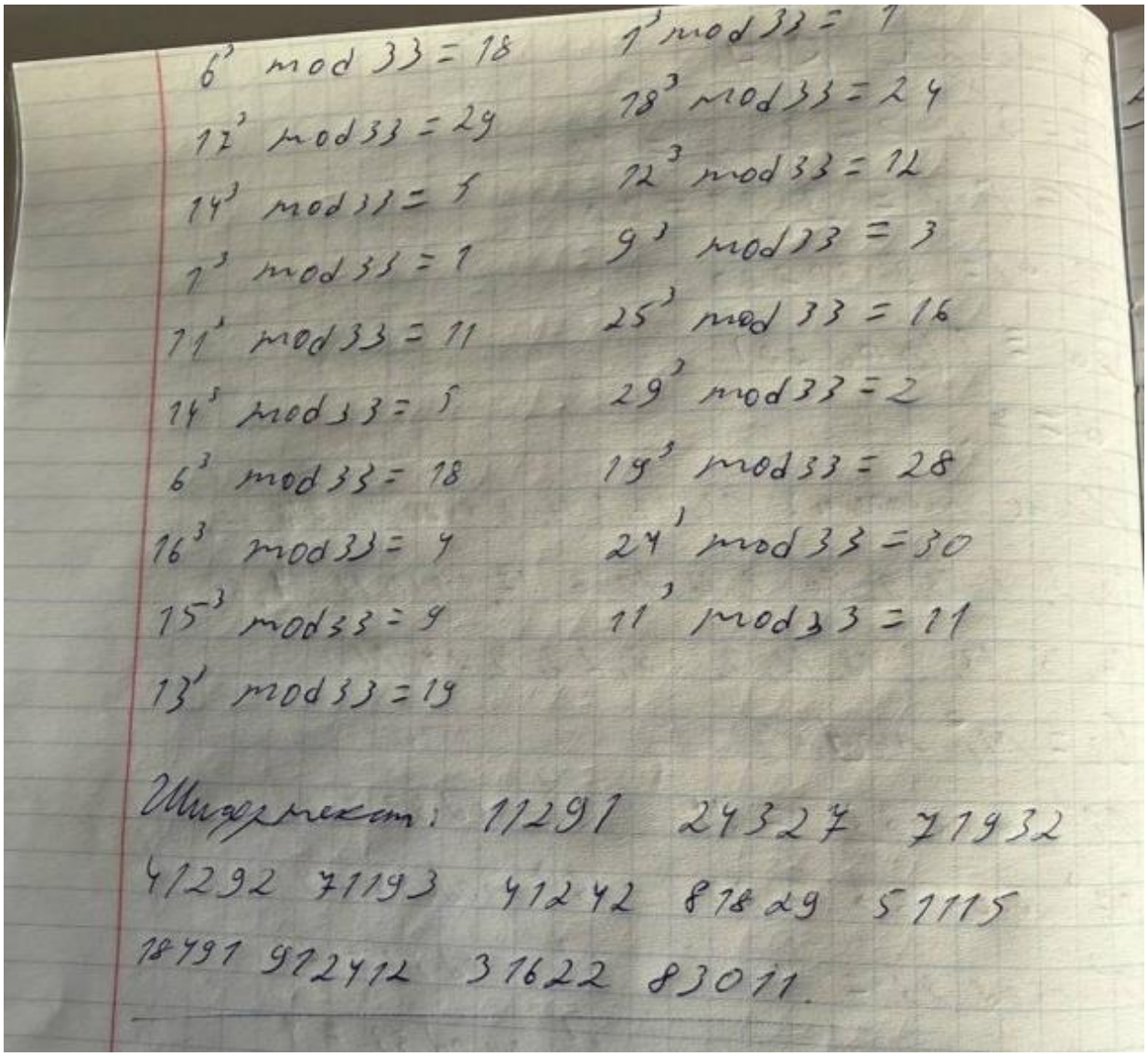


Рисунок 35. Шифр RSA в тетради (2).

Блок I: алгоритмы цифровых подписей

24. Цифровая подпись RSA

RSA - первый алгоритм цифровой подписи, который был разработан в 1977 году в Массачусетском технологическом институт и назван по первым буквам фамилий ее разработчиков (Ronald Rivest, Adi Shamir и Leonard Adleman). RSA основывается на сложности разложения большого числа n на простые множители.

1. Берутся два очень больших простых числа P и Q и находятся произведение простых чисел $N=P\times Q$ и функция Эйлера от этого произведения $\varphi(N)=(P-1)\times(Q-1)$.

2. Выбирается случайное целое число E: $1 < E < \varphi(N)$ и взаимно простое с $\varphi(N)$, и вычисляется D = 1 MOD $\varphi(N))/E$.

3. Потом E и N публикуются как открытый ключ, D сохраняется в тайне.

4. Если M — сообщение, а h(M) = m – хеш-код сообщения, длина которого должна быть в интервале (1,N), то электронная цифровая подпись S получается шифрованием хеш-кода сообщения m возведением в степень D по модулю N: S = mD MOD N.

5. Получателю отправляется сообщение M и подпись S.

6. Получатель сообщения хеширует M, получает хеш-код m’.

7. Проверяет подпись S: расшифровывает хеш-код, возведя S в степень E (число E ему известно) по модулю N: m = SE MOD N.

8. Сравнивает m и m’: если m = m’ - подпись верна.

Блок – схема алгоритма:

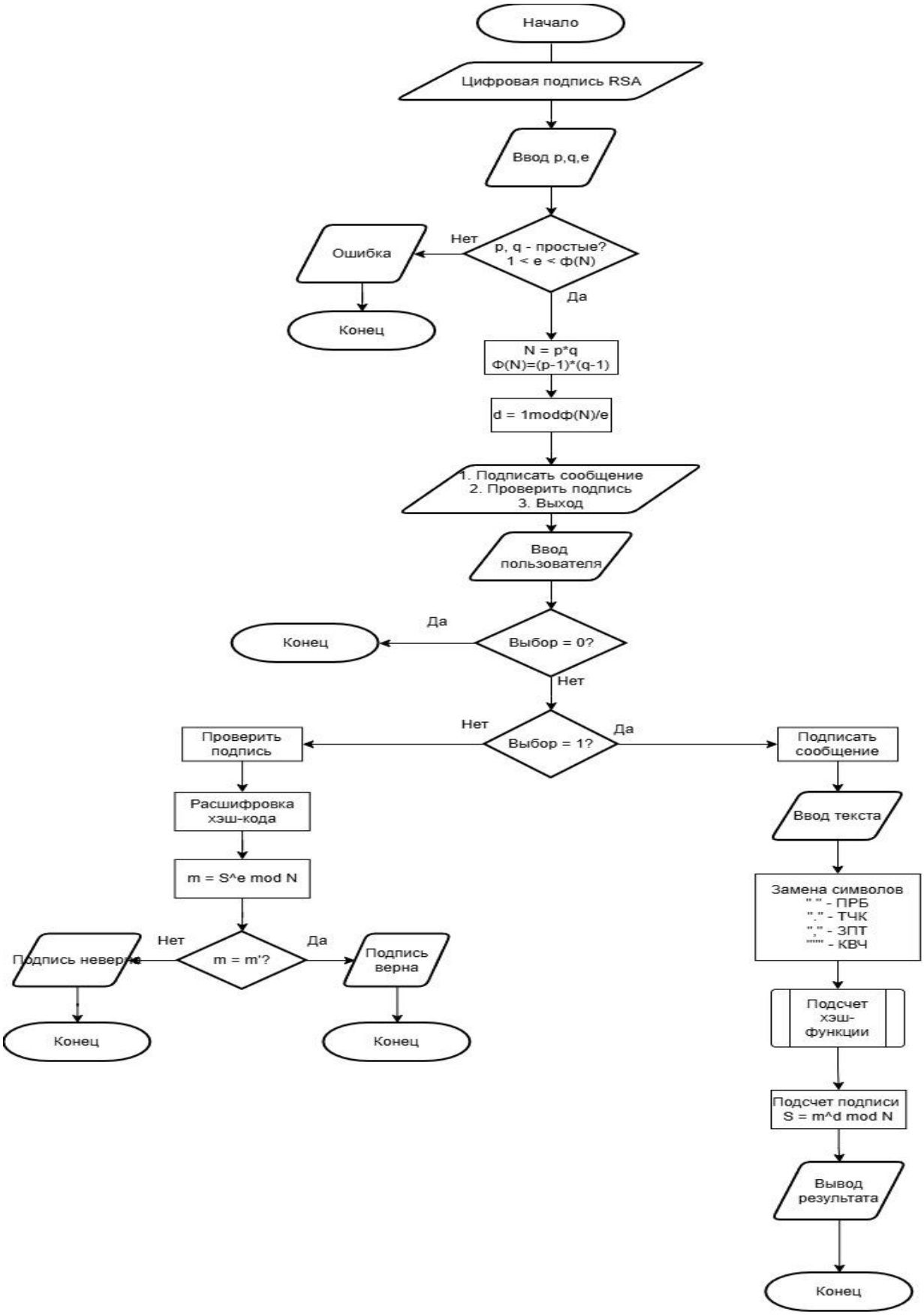


Рисунок 37. Блок-схема цифровой подписи RSA.

Код программы:

```
import random
import os
def is_prime(n: int, k: int = 10) -> bool:
    if n < 2:
```

```
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2
    for _ in range(k):
        a = random.randint(2, n - 2)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
def mod_inverse(a, m):
    m0, x0, x1 = m, 0, 1
    if m == 1:
        return 0
    while a > 1:
        q = a // m
        a, m = m, a % m
        x0, x1 = x1 - q * x0, x0
    return x1 + m0 if x1 < 0 else x1
def prepare_text(message: str) -> str:
    result = ""
    for ch in message.upper():
        if ch == ' ':
            result += "ПРБ"
        elif ch == '.':
            result += "ТЧК"
        else:
            result += ch
    return result
def square_hash(message: str, N: int) -> int:
    alphabet = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
    h = 0
    print(f"Начальное значение h0 = {h}")
    for ch in message:
        if ch in alphabet:
            code = alphabet.index(ch) + 1
        else:
            continue
        h = (h + code) % N
        h = (h * h) % N
    print(f"Итоговый хеш: {h}")
    return h
class RSASignature:
    def __init__(self):
        self.p = None
        self.q = None
        self.N = None
        self.phi = None
        self.e = None
        self.d = None
        self.keys_ready = False
        self.last_message = ""
        self.last_hash = 0
        self.last_signature = 0
    def set_keys(self, p, q, e):
        if not is_prime(p):
            raise ValueError("p не является простым числом")
        if not is_prime(q):
            raise ValueError("q не является простым числом")
        if p == q:
            raise ValueError("p и q должны быть разными")
        self.p = p
        self.q = q
        self.N = p * q
        self.phi = (p - 1) * (q - 1)
        if not (1 < e < self.phi):
            raise ValueError("e должен быть в диапазоне 1 < e < ϕ(N)")
        if gcd(e, self.phi) != 1:
            raise ValueError("e и ϕ(N) не взаимно просты")
        self.e = e
```

```
        self.d = mod_inverse(self.e, self.phi)
        if self.e == self.d:
            raise ValueError("Открытый и секретный ключи совпадают")
        if self.N <= 32:
            raise ValueError(f"Модуль N = {self.N} должен быть больше 32")
        self.keys_ready = True
        print("\nКлючи установлены")
        print(f"p = {p}, q = {q}")
        print(f"N = {self.N}")
        print(f"ϕ(N) = {self.phi}")
        print(f"Открытый ключ e = {e}")
        print(f"Секретный ключ d = {self.d}")
    def sign(self, message):
        if not self.keys_ready:
            raise RuntimeError("Ключи не установлены")
        prepared = prepare_text(message)
        h = square_hash(prepared, self.N)
        signature = pow(h, self.d, self.N)
        self.last_message = message
        self.last_hash = h
        self.last_signature = signature
        print(f"\nПодпись = {h}^{self.d} mod {self.N} = {signature}")
        return signature
    def verify(self):
        if not self.keys_ready:
            raise RuntimeError("Ключи не установлены")
        if not self.last_message:
            raise RuntimeError("Нет подписанного сообщения")
        prepared = prepare_text(self.last_message)
        h = square_hash(prepared, self.N)
        decrypted = pow(self.last_signature, self.e, self.N)
        print(f"\nПроверка подписи")
        print(f"Сообщение: {self.last_message}")
        print(f"Хэш сообщения (число): {h}")
        print(f"Подпись: {self.last_signature}")
        print(f"Вычисляем (подпись)^e mod N = {decrypted}")
        print(f"Сравниваем с хэшем: {h} == {decrypted} => {h == decrypted}")
        return h == decrypted
def clear_screen():
    os.system('cls' if os.name == 'nt' else 'clear')
def main():
    rsa = RSASignature()
    while True:
        clear_screen()
        print("\nЦИФРОВАЯ ПОДПИСЬ RSA")
        print("1. Ввести ключи (p, q, e)")
        print("2. Подписать сообщение")
        print("3. Проверить подпись")
        print("4. Выход")
        choice = input("Ваш выбор: ").strip()
        if choice == '1':
            try:
                p = int(input("Введите простое число p: "))
                q = int(input("Введите простое число q: "))
                e = int(input("Введите открытый ключ e: "))
                rsa.set_keys(p, q, e)
            except Exception as ex:
                print(f"Ошибка: {ex}")
        elif choice == '2':
            if not rsa.keys_ready:
                print("Сначала установите ключи (пункт 1).")
            else:
                msg = input("Введите сообщение для подписи: ")
                try:
                    rsa.sign(msg)
                except Exception as ex:
                    print(f"Ошибка: {ex}")
        elif choice == '3':
            if not rsa.keys_ready:
                print("Сначала установите ключи (пункт 1).")
            elif not rsa.last_message:
                print("Сначала подпишите сообщение (пункт 2).")
            else:
                try:
                    valid = rsa.verify()
                    if valid:
                        print("\nПОДПИСЬ ВЕРНА")
                    else:
                        print("\nПОДПИСЬ НЕВЕРНА")
                except Exception as ex:
                    print(f"Ошибка: {ex}")
        elif choice == '4':
            print("Программа завершена.")
            break
        else:
            print("Неверный выбор")
```

```
if __name__ == "__main__":
    main()
```

Результат работы программы:

```
* Выполнение задачи: C:\Users\NIKITA\AppData\Local\Microsoft\WindowsApps\python3.11.exe c:\Users\NIKITA\Desktop\Папочки\Лабы\КРИПТО\main.py
2. Подписать сообщение
3. Проверить подпись
4. Выход
Ваш выбор: 1
Введите простое число p: 3
Введите простое число q: 11
Введите открытый ключ e: 3

Ключи установлены
p = 3, q = 11
N = 33
ϕ(N) = 20
Открытый ключ e = 3
Секретный ключ d = 7

Нажмите Enter, чтобы вернуться в меню...

ЦИФРОВАЯ ПОДПИСЬ RSA
1. Ввести ключи (p, q, e)
2. Подписать сообщение
3. Проверить подпись
4. Выход
Ваш выбор: 2
Введите сообщение для подписи: Леопард не может изменить своих пятен
Начальное значение h0 = 0
Итоговый хеш: 9

Подпись = 9^7 mod 33 = 15

Нажмите Enter, чтобы вернуться в меню...

ЦИФРОВАЯ ПОДПИСЬ RSA
1. Ввести ключи (p, q, e)
2. Подписать сообщение
3. Проверить подпись
4. Выход
Ваш выбор: 3
Начальное значение h0 = 0
Итоговый хеш: 9

Проверка подписи
Сообщение: Леопард не может изменить своих пятен
Хэш сообщения (число): 9
Подпись: 15
Вычисляем (подпись)^e mod N = 9
Сравниваем с хэшем: 9 == 9 => True
```

Рисунок 38. Результат подписи RSA.

Результат подписи в тетради:

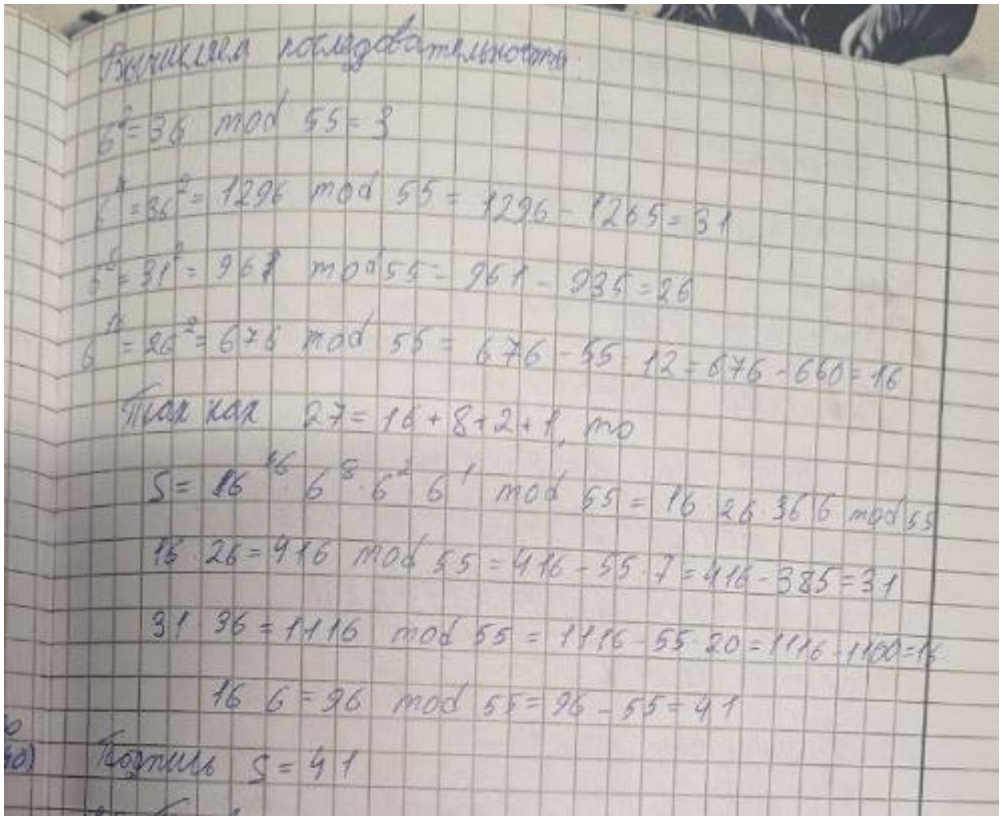


Рисунок 39. Работа в тетради подписи RSA.

Блок J: Стандарты цифровых подписей

27. Цифровая подпись ГОСТ Р 34.10-2012

Алгоритм реализует формирование и проверку электронной цифровой подписи на эллиптической кривой. В программе параметры кривой вводятся пользователем; точка G проверяется, а её порядок q вычисляется по точке.

Открытый ключ Y_A=x_A·G. Для сообщения вычисляется хеш e по учебному правилу из методички, после чего по случайному k формируется подпись (r,s). При проверке вычисляется точка C′ и сравниваются r′ и r.

Параметры контрольного примера: p=11, a=b=1, G=(0,1), q=7. Для основного запуска программа позволяет задать параметры вручную.

Блок-схема:

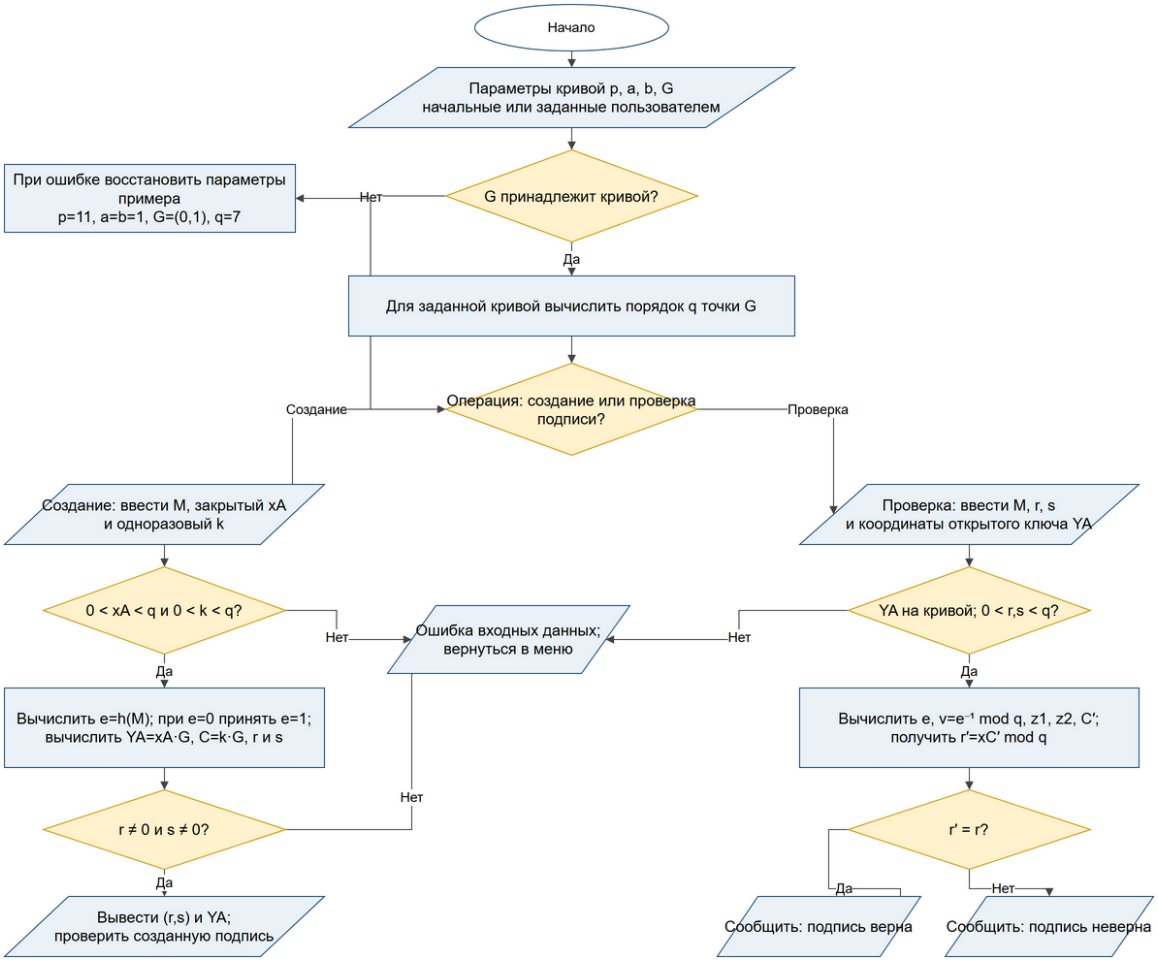


Рисунок 40. Блок-схема цифровой подписи ГОСТ Р 34.10-2012.

Код программы:

```
"""
Лабораторная работа 3_2
Алгоритм 27: ЭЦП по ГОСТ Р 34.10-2012
Вариант: "Леопард не может изменить своих пятен"

Источник: ЛАБКРИПТ_2022, Блок J, стр. 37-39
```

Задание: "PDF/Задание по ГОСТ Р 34.10-2012..pdf"

Параметры из методички (пример):

Кривая $E_p(a,b)$: $y^2 \equiv x^3 + x + 1 \pmod{11}$

$a = 1$, $b = 1$, $p = 11$

Порядок подгруппы $q = 7$

Генератор $G = (0, 1)$

Хэш-функция квадратичной свёртки ($h_0 = 0$):

$h_i = (h_{i-1} + M_i)^2 \bmod q$

где M_i — коды (ord) символов сообщения

Подпись (отправитель):

1. Выбрать секретный ключ $0 < x_A < q$
2. Вычислить $Y_A = x_A * G$ (открытый ключ)
3. Вычислить $e = h(\text{сообщение}) \bmod q$, если $e=0 \to e=1$
4. Выбрать случайное $0 < k < q$
5. Вычислить $C = k * G$, $r = x_C \bmod q$
6. $s = (k * e + r * x_A) \bmod q$
7. Подпись: (r, s)

Проверка (получатель):

1. Вычислить $e = h(\text{сообщение}) \bmod q$, если $e=0 \to e=1$
2. Проверить $0 < r < q$ и $0 < s < q$
3. $v = e^{-1} \bmod q$
4. $z_1 = s * v \bmod q$
5. $z_2 = -r * v \bmod q$
6. $C' = z_1*G + z_2*Y_A$
7. $r' = x_{C'} \bmod q$; если $r' = r$ — подпись верна

```
"""
import random
from math import isqrt

#  Эллиптическая кривая

class EllipticCurve:
    def __init__(self, p, a, b):
        self.p = p
        self.a = a
        self.b = b

    def is_on_curve(self, point):
        if point is None:
            return True
        x, y = point
        return (y * y - x ** 3 - self.a * x - self.b) % self.p == 0

    def neg(self, P):
        if P is None:
            return None
        return P[0], (-P[1]) % self.p

    def add(self, P, Q):
        if P is None:
            return Q
        if Q is None:
            return P
        x1, y1 = P
        x2, y2 = Q
        p = self.p
        if x1 == x2:
            if (y1 + y2) % p == 0:
                return None  # точка на бесконечности
            lam = (3 * x1 * x1 + self.a) * pow(2 * y1, p - 2, p) % p
        else:
            lam = (y2 - y1) * pow(x2 - x1, p - 2, p) % p
        x3 = (lam * lam - x1 - x2) % p
        y3 = (lam * (x1 - x3) - y1) % p
        return x3, y3

    def mul(self, k, P):
```

```
        """Скалярное умножение методом двоичного разложения."""
        result = None
        addend = P
        while k > 0:
            if k & 1:
                result = self.add(result, addend)
            addend = self.add(addend, addend)
            k >>= 1
        return result


def point_order(curve, P):
    """Вычисляет порядок точки P перебором кратных P.

    Для кривой над простым полем порядок группы точек не превосходит
    p + 1 + 2*sqrt(p) (граница Хассе), поэтому перебор имеет верхнюю границу.
    """
    if curve.p <= 1:
        raise ValueError("Модуль p должен быть больше 1.")
    if P is None or not curve.is_on_curve(P):
        raise ValueError("Точка G должна лежать на заданной кривой.")

    max_order = curve.p + 1 + isqrt(4 * curve.p)
    multiple = None
    for order in range(1, max_order + 1):
        multiple = curve.add(multiple, P)
        if multiple is None:
            return order
    raise ValueError("Не удалось вычислить порядок G; проверьте параметры кривой.")


#  Хэш-функция квадратичной свёртки (ЛАБКРИПТ_2022, стр. 1)
#  h_0 = 0
#  h_i = (h_{i-1} + M_i)² mod p

def hash_func(message: str, q: int) -> int:
    """
    Упрощённая хэш-функция квадратичной свёртки по методичке:
      h_0 = 0
      h_i = (h_{i-1} + ord(M_i))² mod q
    Если h = 0 — возвращаем 1 (защита от нулевого хэша).
    """
    h = 0
    for ch in message:
        h = (h + ord(ch)) ** 2 % q
    return h if h != 0 else 1


#  Генерация ключей

def keygen(curve, P, q):
    d = random.randint(1, q - 1)   # закрытый ключ
    Q = curve.mul(d, P)             # открытый ключ Q = d·P
    return d, Q


#  Создание подписи
#

def sign(message, curve, P, q, d, k=None):
    e = hash_func(message, q)
    if not 0 < d < q:
        raise ValueError(f"Закрытый ключ d должен быть в диапазоне 1..{q - 1}.")
    if k is not None:
        if not 0 < k < q:
            raise ValueError(f"Одноразовый ключ k должен быть в диапазоне 1..{q - 1}.")
        C = curve.mul(k, P)
        if C is None:
            raise ValueError("Выбранный k дал точку на бесконечности; введите другое k.")
        r = C[0] % q
        s = (k * e + r * d) % q
        if r == 0 or s == 0:
            raise ValueError("Для выбранного k получено r=0 или s=0; введите другое k.")
        return r, s, e, k
    while True:
        k = random.randint(1, q - 1)
        C = curve.mul(k, P)
        if C is None:
            continue
        r = C[0] % q
        if r == 0:
            continue
        s = (k * e + r * d) % q
        if s == 0:
            continue
        return r, s, e, k
```

```
#  Проверка подписи

def verify(message, r, s, curve, P, Q_pub, q):
    if not (0 < r < q and 0 < s < q):
        return False
    e = hash_func(message, q)
    v = pow(e, q - 2, q)       # e^{-1} mod q
    z1 = s * v % q
    z2 = (-r * v) % q
    C = curve.add(curve.mul(z1, P), curve.mul(z2, Q_pub))
    if C is None:
        return False
    return C[0] % q == r


#  Главное меню

# Параметры по умолчанию из примера методички (ЛАБКРИПТ_2022, стр. 37)
#   E_p(a,b): y² ≡ x³ + x + 1 (mod 11)
#   a=1, b=1, p=11, q=7, G=(0,1)
DEFAULT_P = 11
DEFAULT_A = 1
DEFAULT_B = 1
DEFAULT_G = (0, 1)
DEFAULT_Q = 7


def print_params(curve, G, q):
    print(f"  Кривая     : y² ≡ x³ + {curve.a}x + {curve.b}  (mod {curve.p})")
    print(f"  Генератор  : G = {G}")
    print(f"  Порядок    : q = {q}")


def main():
    print("=" * 60)
    print("ГОСТ Р 34.10-2012  —  Цифровая подпись (эллиптические кривые)")
    print("=" * 60)
    print("Параметры из методички (ЛАБКРИПТ_2022, Блок J, задание 27):")
    print(f"  Кривая E_p(a,b): y² ≡ x³ + ax + b  (mod p)")
    print(f"  Хэш: h_0=0, h_i=(h_{{i-1}}+ord(M_i))² mod q")
    print()

    # Загружаем параметры по умолчанию
    curve = EllipticCurve(p=DEFAULT_P, a=DEFAULT_A, b=DEFAULT_B)
    P_BASE = DEFAULT_G
    Q_ORDER = DEFAULT_Q

    while True:
        print("\n" + "-" * 60)
        print("Текущие параметры кривой:")
        print_params(curve, P_BASE, Q_ORDER)
        print("-" * 60)
        print("1 — Задать параметры кривой вручную")
        print("2 — Создать подпись")
        print("3 — Проверить подпись")
        print("4 — Показать пример из методички")
        print("0 — Выход")

        choice = input("Ваш выбор: ").strip()

        if choice == "0":
            break

        try:
            if choice == "1":
                print(f"\nВведите параметры кривой y² ≡ x³ + ax + b (mod p)")
                p = int(input("  p (простое): "))
                a = int(input("  a: "))
                b = int(input("  b: "))
                gx = int(input("  Gx (x-координата генератора): "))
                gy = int(input("  Gy (y-координата генератора): "))
                curve = EllipticCurve(p=p, a=a, b=b)
                P_BASE = (gx, gy)
                if not curve.is_on_curve(P_BASE):
                    print("  ОШИБКА: точка G не лежит на заданной кривой!")
                    # Возвращаем умолчания
                    curve = EllipticCurve(p=DEFAULT_P, a=DEFAULT_A, b=DEFAULT_B)
                    P_BASE = DEFAULT_G
                    Q_ORDER = DEFAULT_Q
                else:
                    Q_ORDER = point_order(curve, P_BASE)
                    print(f"  Порядок точки G вычислен автоматически: q = {Q_ORDER}.")
                    print("  Параметры установлены.")

            elif choice == "2":
                print("\n--- Создание подписи ---")
```

```
                message = input("Введите сообщение: ")
                if not message:
                    raise ValueError("Сообщение не должно быть пустым.")
                d = int(input(f"Введите закрытый ключ x_A (1..{Q_ORDER - 1}): "))
                k = int(input(f"Введите одноразовый ключ k (1..{Q_ORDER - 1}): "))
                if not (0 < d < Q_ORDER):
                    raise ValueError(f"x_A должен быть в диапазоне 1..{Q_ORDER - 1}.")
                Q_pub = curve.mul(d, P_BASE)
                r, s, e, _ = sign(message, curve, P_BASE, Q_ORDER, d, k)
                print(f"\nХэш e = {e}")
                print(f"Открытый ключ Y_A = {Q_pub}")
                print(f"Подпись (r, s) = ({r}, {s})")
                ok = verify(message, r, s, curve, P_BASE, Q_pub, Q_ORDER)
                print(f"Проверка созданной подписи: {'ВЕРНА ✓' if ok else 'НЕ ВЕРНА ✗'}")

            elif choice == "3":
                print("\n--- Проверка подписи ---")
                message = input("Введите исходное сообщение: ")
                r = int(input("Введите r: "))
                s = int(input("Введите s: "))
                qx = int(input("Введите координату Y_Ax: "))
                qy = int(input("Введите координату Y_Ay: "))
                Q_pub = (qx, qy)
                if not curve.is_on_curve(Q_pub):
                    raise ValueError("Открытый ключ Y_A не принадлежит заданной эллиптической кривой.")
                valid = verify(message, r, s, curve, P_BASE, Q_pub, Q_ORDER)
                print(f"Результат проверки: {'ПОДПИСЬ ВЕРНА ✓' if valid else 'ПОДПИСЬ НЕВЕРНА ✗'}")

            elif choice == "4":
                print("\n" + "=" * 60)
                print("ПРИМЕР ИЗ МЕТОДИЧКИ (ЛАБКРИПТ_2022, стр. 37)")
                print("=" * 60)
                print("Кривая E_p(a,b): y² ≡ x³ + x + 1 (mod 11)")
                print("a = 1, b = 1, p = 11, q = 7, G = (0, 1)")
                print()
                demo_curve = EllipticCurve(p=11, a=1, b=1)
                demo_G = (0, 1)
                demo_q = 7
                # Пример из PDF: x_A=4, k=5
                demo_xA = 4
                demo_k = 5
                demo_msg = "ДЕВА"  # Из таблицы вариантов для примера
                # Считаем хэш
                demo_e = hash_func(demo_msg, demo_q)
                demo_YA = demo_curve.mul(demo_xA, demo_G)
                demo_C = demo_curve.mul(demo_k, demo_G)
                demo_r = demo_C[0] % demo_q
                demo_s = (demo_k * demo_e + demo_r * demo_xA) % demo_q
                print(f"Сообщение   : {demo_msg!r}")
                print(f"x_A (секрет): {demo_xA}")
                print(f"k (разовый) : {demo_k}")
                print(f"Хэш e       : {demo_e}")
                print(f"Y_A = {demo_xA}·G = {demo_YA}")
                print(f"C = {demo_k}·G = {demo_C}")
                print(f"r = {demo_r}, s = {demo_s}")
                print(f"Подпись (r, s) = ({demo_r}, {demo_s})")
                ok = verify(demo_msg, demo_r, demo_s, demo_curve, demo_G, demo_YA, demo_q)
                print(f"Проверка: {'ВЕРНА ✓' if ok else 'НЕ ВЕРНА ✗'}")
                print("=" * 60)

            else:
                print("Выберите 0, 1, 2, 3 или 4.")

        except ValueError as exc:
            print(f"Ошибка ввода: {exc}")
        except Exception as exc:
            print(f"Ошибка: {exc}")


if __name__ == "__main__":
    main()
```

Контрольный пример по параметрам методички: кривая $y^2=x^3+x+1 \pmod{11}$, $G=(0,1)$, $q=7$. Программа вычисляет порядок G, формирует подпись для сообщения и проверяет её по открытому ключу.

Тест на длинном сообщении: сформирована строка длиной 1000 символов из текста варианта 20; подпись проверена программой. В результате сравнения получено «подпись верна».

Проблемы Выходные данные Консоль отладки Терминал Порты

```
Введите параметры кривой y² ≡ x³ + ax + b (mod p)
  p (простое): 11
  a: 2
  b: 6
  Gx (x-координата генератора): 10
  Gy (y-координата генератора): 5
  Порядок точки G вычислен автоматически: q = 7.
  Параметры установлены.

------------------------------------------------------------
Текущие параметры кривой:
  Кривая     : y² ≡ x³ + 2x + 6  (mod 11)
  Генератор  : G = (10, 5)
  Порядок    : q = 7
------------------------------------------------------------
1 — Задать параметры кривой вручную
2 — Создать подпись
3 — Проверить подпись
4 — Показать пример из методички
0 — Выход
Ваш выбор: 2

--- Создание подписи ---
Введите сообщение: ПЕРЕСДАЧА
Введите закрытый ключ x_A (1..6): 5
Введите одноразовый ключ k (1..6): 4

Хэш e = 2
Открытый ключ Y_A = (5, 8)
Подпись (r, s) = (1, 6)
Проверка созданной подписи: ВЕРНА ✓

------------------------------------------------------------
Текущие параметры кривой:
  Кривая     : y² ≡ x³ + 2x + 6  (mod 11)
  Генератор  : G = (10, 5)
  Порядок    : q = 7
------------------------------------------------------------
```

```
------------------------------------------------------------
1 — Задать параметры кривой вручную
2 — Создать подпись
3 — Проверить подпись
4 — Показать пример из методички
0 — Выход
Ваш выбор: 3

--- Проверка подписи ---
Введите исходное сообщение: ПЕРЕСДАЧА
Введите r: 1
Введите s: 6
Введите координату Y_Ax: 5
Введите координату Y_Ay: 8
Результат проверки: ПОДПИСЬ ВЕРНА ✓

------------------------------------------------------------
Текущие параметры кривой:
  Кривая     : y² ≡ x³ + 2x + 6  (mod 11)
  Генератор  : G = (10, 5)
  Порядок    : q = 7
------------------------------------------------------------
1 — Задать параметры кривой вручную
2 — Создать подпись
3 — Проверить подпись
4 — Показать пример из методички
0 — Выход
Ваш выбор:
```

Блок К: обмен ключами

28. Обмен ключами по алгоритму Диффи-Хеллмана

Протокол Диффи-Хеллмана — это криптографический протокол, позволяющий двум и более сторонам получить общий секретный ключ по незащищённому каналу связи. Протокол был предложен Уитфилдом Диффи и Мартином Хеллманом в 1976 году и стал первым практическим методом асимметричного распределения ключей.

Ключевая особенность: Протокол не выполняет шифрование напрямую, а позволяет сторонам договориться о секретном ключе, который затем можно использовать для симметричного шифрования. Алгоритм основан на вычислительной сложности дискретного логарифмирования.

Блок-схема алгоритма:

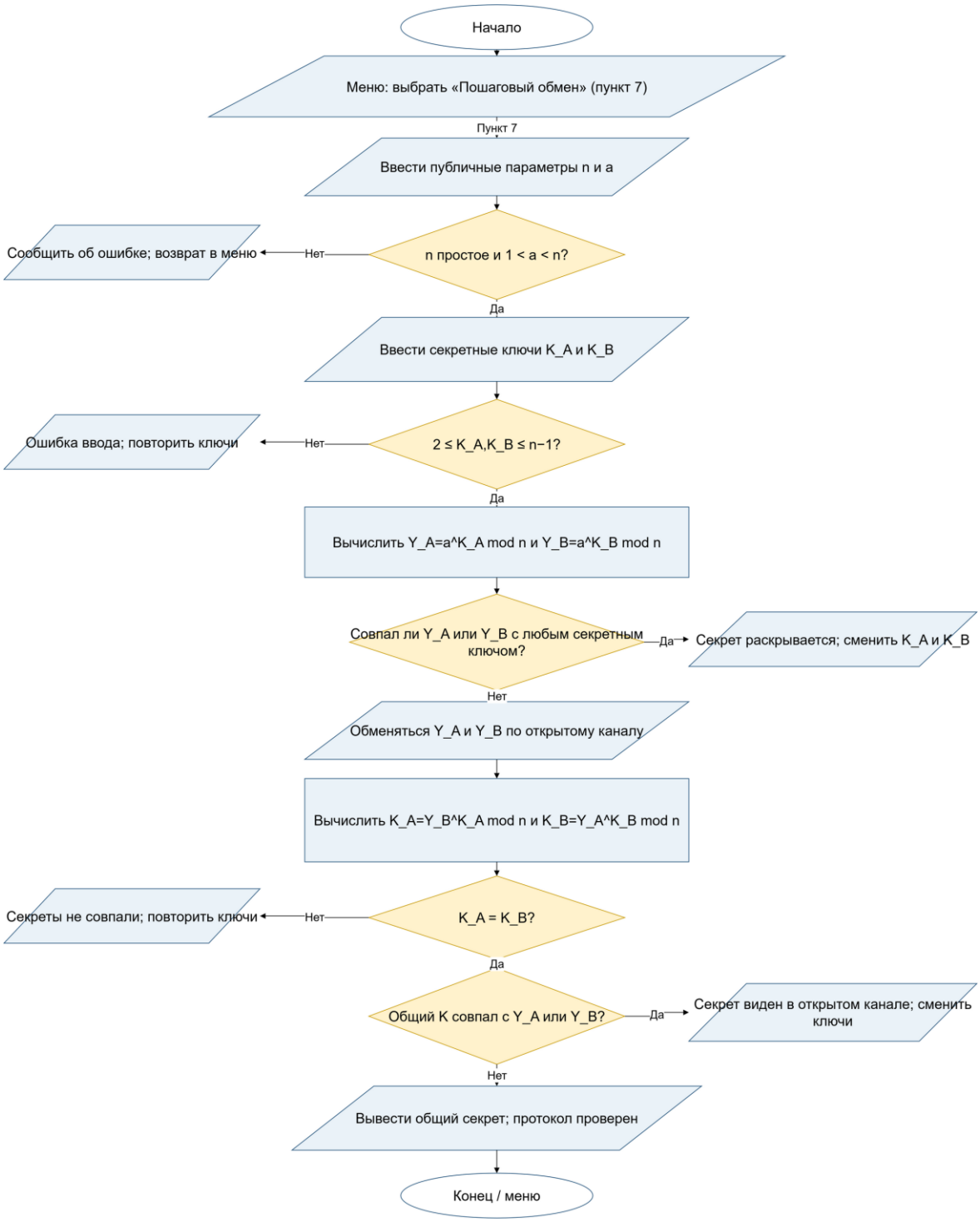


Рисунок 45. Блок-схема Диффи–Хеллмана с проверкой раскрытия ключей.

Код программы:

```
# ============================================================
# БЛОК K: ОБМЕН КЛЮЧАМИ
# Лаб 2_2 (задание 28) — Обмен ключами по Диффи-Хеллману
# Источник: ЛАБКРИПТ_2022, стр. 40
# ============================================================
#
# АЛГОРИТМ (по методичке):
#   Публичные параметры: n — большое простое, a (1 < a < n)
#   Секретные ключи:     K_A, K_B — случайные числа из [2, n-1]
#   Открытые ключи:      Y_A = a^K_A mod n,  Y_B = a^K_B mod n
#   Обмен:               Y_A и Y_B передаются по открытому каналу
#   Общий секрет:        K_A: K = Y_B^K_A mod n
#                        K_B: K = Y_A^K_B mod n
#   Проверка:            K_A == K_B (оба должны совпадать)
#
```

```
# ДОКАЗАТЕЛЬСТВО:
#   A: Y_B^K_A mod n = (a^K_B)^K_A mod n = a^(K_B*K_A) mod n = K
#   B: Y_A^K_B mod n = (a^K_A)^K_B mod n = a^(K_A*K_B) mod n = K
# ============================================================

import random
import sys


# ──────────────────────────────────────────────
#  Математические вспомогательные функции
# ──────────────────────────────────────────────

def mod_pow(base, exp, mod):
    """Быстрое возведение в степень по модулю (алгоритм «двоичного возведения»)."""
    result = 1
    base %= mod
    while exp > 0:
        if exp & 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp >>= 1
    return result


def is_prime(n):
    """Детерминированная проверка простоты методом пробных делений."""
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    i = 3
    while i * i <= n:
        if n % i == 0:
            return False
        i += 2
    return True


def miller_rabin(n, k=10):
    """Тест Миллера–Рабина для больших чисел (вероятностный, k раундов)."""
    if n < 2:
        return False
    if n == 2 or n == 3:
        return True
    if n % 2 == 0:
        return False
    # Представляем n-1 = 2^r * d
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = mod_pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = (x * x) % n
            if x == n - 1:
                break
        else:
            return False
    return True


def generate_prime(bits=512):
    """Генерация случайного простого числа заданной битности (тест Миллера–Рабина)."""
    while True:
        candidate = random.getrandbits(bits) | (1 << (bits - 1)) | 1  # нечётное, старший бит = 1
        if miller_rabin(candidate):
            return candidate


def find_primitive_root(p):
    """
    Находит первообразный корень (примитивный корень) по модулю простого p.
    Для простого p φ(p) = p-1. Перебираем кандидатов от 2.
    """
    phi = p - 1
    # Факторизация phi
    factors = set()
    temp = phi
    d = 2
    while d * d <= temp:
```

```
        if temp % d == 0:
            factors.add(d)
            while temp % d == 0:
                temp //= d
        d += 1
    if temp > 1:
        factors.add(temp)
    # Проверяем кандидатов
    for g in range(2, p):
        ok = True
        for factor in factors:
            if mod_pow(g, phi // factor, p) == 1:
                ok = False
                break
        if ok:
            return g
    return -1  # не найден (не должно случиться для простого p)


# ──────────────────────────────────────────────
#  Класс протокола Диффи-Хеллмана
# ──────────────────────────────────────────────

class DiffieHellman:
    """
    Протокол обмена ключами Диффи–Хеллмана.

    Публичные параметры:  n (простое), a (1 < a < n)
    Секретные ключи:      K_A, K_B ∈ [2, n-1]
    Открытые ключи:       Y_A = a^K_A mod n,  Y_B = a^K_B mod n
    Общий секрет:         K = Y_B^K_A mod n  =  Y_A^K_B mod n
    """

    def __init__(self):
        self.n = None      # публичный модуль (большое простое)
        self.a = None      # публичное основание a
        self.K_A = None    # секретный ключ пользователя A
        self.K_B = None    # секретный ключ виртуального пользователя B
        self.Y_A = None    # открытый ключ A
        self.Y_B = None    # открытый ключ B
        self.shared_K_A = None   # общий секрет, вычисленный стороной A
        self.shared_K_B = None   # общий секрет, вычисленный стороной B

    # ── Установка публичных параметров ──────────────────

    def set_params(self, n, a):
        """Устанавливает публичные параметры n и a с проверками."""
        # Не используем пробное деление для больших n: проверяем простоту
        # тестом Миллера–Рабина с достаточным числом раундов.
        if not miller_rabin(n, k=32):
            print("  Ошибка: n должно быть простым числом!")
            return False
        if not (1 < a < n):
            print(f"  Ошибка: a должно быть в диапазоне (1 < a < n={n})!")
            return False
        self.n = n
        self.a = a
        # Сброс предыдущих ключей
        self.K_A = self.K_B = self.Y_A = self.Y_B = None
        self.shared_K_A = self.shared_K_B = None
        print(f"\n  Публичные параметры установлены:")
        print(f"    n = {n}")
        print(f"    a = {a}  (основание степени)")
        return True

    def auto_generate_params(self, bits=24):
        """Генерирует учебные параметры; подбор корня ограничен 32 битами."""
        if not 4 <= bits <= 32:
            print("  Ошибка: учебная автогенерация поддерживает битность от 4 до 32.")
            return False
        print(f"\n  Генерация простого числа ({bits} бит)...")
        n = generate_prime(bits)
        print(f"  Найдено простое n = {n}")
        print(f"  Поиск примитивного корня по модулю n...")
        a = find_primitive_root(n)
        if a == -1:
            print("  Ошибка: не удалось найти примитивный корень.")
            return False
        print(f"  Найден примитивный корень a = {a}")
        self.n = n
        self.a = a
        self.K_A = self.K_B = self.Y_A = self.Y_B = None
        self.shared_K_A = self.shared_K_B = None
        return True

    # ── Шаг 1: Выбор секретных ключей ──────────────────
```

```
    def set_secret_key_A(self, K_A):
        """Задаёт секретный ключ пользователя A."""
        if self.n is None:
            print("  Ошибка: сначала установите публичные параметры!")
            return False
        if not (2 <= K_A <= self.n - 1):
            print(f"  Ошибка: K_A должно быть в диапазоне [2, n-1 = {self.n - 1}]!")
            return False
        self.K_A = K_A
        self.Y_A = self.Y_B = None
        self.shared_K_A = self.shared_K_B = None
        print(f"  Секретный ключ A: K_A = {K_A}")
        return True

    def auto_secret_key_B(self):
        """Виртуальный пользователь B сам выбирает случайный секретный ключ."""
        if self.n is None:
            print("  Ошибка: сначала установите публичные параметры!")
            return False
        self.K_B = random.randint(2, self.n - 1)
        self.Y_A = self.Y_B = None
        self.shared_K_A = self.shared_K_B = None
        print(f"  Виртуальный пользователь B выбрал: K_B = {self.K_B}  (СЕКРЕТ — известен только B)")
        return True

    def set_secret_key_B(self, K_B):
        """Задаёт секретный ключ пользователя B, введённый вручную."""
        if self.n is None:
            print("  Ошибка: сначала установите публичные параметры!")
            return False
        if not (2 <= K_B <= self.n - 1):
            print(f"  Ошибка: K_B должно быть в диапазоне [2, n-1 = {self.n - 1}]!")
            return False
        self.K_B = K_B
        self.Y_A = self.Y_B = None
        self.shared_K_A = self.shared_K_B = None
        print(f"  Секретный ключ B: K_B = {K_B}")
        return True

    # ── Проверка: открытый ключ не должен совпадать с секретным ──

    def check_public_keys_do_not_reveal_secrets(self):
        """
        Проверяет, что открытые ключи не совпали с секретными.
        Если Y_A == K_A, Y_B == K_B или открытый ключ одной стороны
        равен секретному ключу другой — секрет раскрыт, ключи надо сменить.
        """
        if self.Y_A is None or self.Y_B is None:
            return True

        checks = (
            ("Y_A", self.Y_A, "K_A", self.K_A),
            ("Y_B", self.Y_B, "K_B", self.K_B),
            ("Y_A", self.Y_A, "K_B", self.K_B),
            ("Y_B", self.Y_B, "K_A", self.K_A),
        )

        for pub_name, pub_val, sec_name, sec_val in checks:
            if sec_val is not None and pub_val == sec_val:
                print(f"\n  ✗ ОШИБКА: {pub_name} == {sec_name} = {pub_val}.")
                print("    Открытый ключ совпал с секретным — секрет раскрыт.")
                print("    Выберите другие секретные ключи A и B.")
                return False

        return True

    # ── Шаг 2: Вычисление открытых ключей ────────────────────

    def compute_public_keys(self):
        """
        Шаг 2 алгоритма:
          Y_A = a^K_A mod n
          Y_B = a^K_B mod n
        После вычисления проверяет, что открытые ключи не совпали с секретными.
        """
        if None in (self.n, self.a, self.K_A, self.K_B):
            print("  Ошибка: не все параметры установлены!")
            return False
        self.Y_A = mod_pow(self.a, self.K_A, self.n)
        self.Y_B = mod_pow(self.a, self.K_B, self.n)
        print(f"\n  Открытые ключи (передаются по открытому каналу):")
        print(f"    Y_A = a^K_A mod n = {self.a}^{self.K_A} mod {self.n} = {self.Y_A}")
        print(f"    Y_B = a^K_B mod n = {self.a}^{self.K_B} mod {self.n} = {self.Y_B}")

        if not self.check_public_keys_do_not_reveal_secrets():
            self.Y_A = self.Y_B = None
            self.shared_K_A = self.shared_K_B = None
```

```
            return False

        return True

    # ── Шаг 3: Вычисление общего секрета ──────────────────

    def compute_shared_secret(self):
        """
        Шаг 3 алгоритма:
          K_A: K = Y_B^K_A mod n
          K_B: K = Y_A^K_B mod n
        Проверки:
          - K у A и B совпадают;
          - K не равен ни одному открытому ключу Y_A, Y_B
            (иначе секрет виден из открытого канала).
        """
        if None in (self.Y_A, self.Y_B, self.K_A, self.K_B):
            print("  Ошибка: сначала вычислите открытые ключи!")
            return False

        self.shared_K_A = mod_pow(self.Y_B, self.K_A, self.n)
        self.shared_K_B = mod_pow(self.Y_A, self.K_B, self.n)

        print(f"\n  Вычисление общего секрета:")
        print(f"    A: K = Y_B^K_A mod n = {self.Y_B}^{self.K_A} mod {self.n} = {self.shared_K_A}")
        print(f"    B: K = Y_A^K_B mod n = {self.Y_A}^{self.K_B} mod {self.n} = {self.shared_K_B}")

        if self.shared_K_A != self.shared_K_B:
            print(f"\n  ✗ ОШИБКА: K у A ≠ K у B  ({self.shared_K_A} ≠ {self.shared_K_B})")
            return False

        # Главная проверка: общий секрет не должен совпасть с открытым ключом
        if self.shared_K_A == self.Y_A or self.shared_K_A == self.Y_B:
            which = "Y_A" if self.shared_K_A == self.Y_A else "Y_B"
            print(f"\n  ✗ ОШИБКА: общий секрет K = {self.shared_K_A} совпал с открытым ключом {which}.")
            print("    Такой секрет виден из открытого канала.")
            print("    Выберите другие секретные ключи A и B.")
            self.shared_K_A = self.shared_K_B = None
            return False

        print(f"\n  ✓ ПРОВЕРКА ПРОЙДЕНА: K у A = K у B = {self.shared_K_A}")
        print(f"    Общий секретный ключ успешно выработан!")
        return True

    # ── Полная демонстрация протокола ─────────────────────

    def full_demo(self):
        """Полная пошаговая демонстрация протокола Диффи-Хеллмана."""
        sep = "─" * 60
        print(f"\n{sep}")
        print("  ПРОТОКОЛ ОБМЕНА КЛЮЧАМИ ДИФФИ-ХЕЛЛМАНА")
        print(f"  Источник: ЛАБКРИПТ_2022, Блок K, Задание 28")
        print(sep)

        print("\n  Шаг 1. Публичные параметры (известны всем):")
        print(f"    n = {self.n}  (простое число)")
        print(f"    a = {self.a}  (1 < a < n)")

        print("\n  Шаг 2. Выбор секретных ключей:")
        print(f"    Пользователь A выбирает K_A = {self.K_A}  (из интервала [2, n-1])")
        print(f"    Пользователь B выбирает K_B = {self.K_B}  (из интервала [2, n-1])")

        print("\n  Шаг 3. Вычисление открытых ключей:")
        if not self.compute_public_keys():
            print(f"\n{sep}")
            print("  ИТОГ: Ошибка в протоколе!")
            print(sep)
            return False

        print("\n  Шаг 4. Обмен открытыми ключами по открытому каналу:")
        print(f"    A → B: Y_A = {self.Y_A}")
        print(f"    B → A: Y_B = {self.Y_B}")

        print("\n  Шаг 5. Независимое вычисление общего секрета:")
        ok = self.compute_shared_secret()

        print(f"\n{sep}")
        if ok:
            print(f"  ИТОГ: Общий секретный ключ K = {self.shared_K_A}")
        else:
            print("  ИТОГ: Ошибка в протоколе!")
        print(sep)
        return ok


# ──────────────────────────────────────────────────────
#  Меню
```

```
# ──────────────────────────────────────────

def print_menu(dh):
    """Выводит текущее меню с состоянием."""
    print("\n" + "=" * 60)
    print("  БЛОК K: ОБМЕН КЛЮЧАМИ (Задание 28 — Диффи-Хеллман)")
    print("=" * 60)

    # Текущее состояние параметров
    if dh.n:
        print(f"  n = {dh.n}  |  a = {dh.a}")
    else:
        print("  Параметры не установлены")
    if dh.K_A:
        print(f"  K_A = {dh.K_A}  |  K_B = {'установлен' if dh.K_B else 'не установлен'}")
    if dh.shared_K_A:
        print(f"  Общий ключ K = {dh.shared_K_A}")

    print()
    print("  1 — Ввести публичные параметры (n, a) вручную")
    print("  2 — Авто-генерация параметров (простое n и генератор a)")
    print("  3 — Ввести секретный ключ A (K_A) вручную")
    print("  4 — Ввести секретный ключ B (K_B) вручную")
    print("  5 — Вычислить открытые ключи Y_A и Y_B")
    print("  6 — Вычислить общий секрет и проверить протокол")
    print("  7 — Пошаговый обмен (параметры, ключи, проверка)")
    print("  8 — Пример из методички (малые числа)")
    print("  0 — Выход")
    print()


def demo_textbook(dh):
    """
    Демонстрация с малыми числами по образцу методички.
    Используем простое n=23, генератор a=5 — классический учебный пример.
    """
    print("\n" + "—" * 60)
    print("  ПРИМЕР ПО МЕТОДИЧКЕ (малые числа)")
    print("—" * 60)
    print("  Публичные параметры: n = 23 (простое), a = 5")
    print("  Пользователь A выбирает K_A = 6")
    print("  Пользователь B выбирает K_B = 15")

    dh.n = 23
    dh.a = 5
    dh.K_A = 6
    dh.K_B = 15
    dh.Y_A = None
    dh.Y_B = None
    dh.shared_K_A = None
    dh.shared_K_B = None

    print("\n  Открытые ключи:")
    if not dh.compute_public_keys():
        print("  Демонстрация прервана: открытый ключ раскрывает секрет.")
        return
    print("\n  Общий секрет:")
    dh.compute_shared_secret()


def manual_protocol(dh):
    """Последовательно запрашивает параметры и ключи, затем проверяет обмен."""
    print("\n" + "—" * 60)
    print("  ПОШАГОВЫЙ ОБМЕН ДИФФИ-ХЕЛЛМАНА")
    print("—" * 60)

    print("\n1. Введите публичные параметры")
    try:
        n = int(input("  Простое число n: "))
        a = int(input("  Основание a (1 < a < n): "))
    except ValueError:
        print("  Ошибка: введите целые числа.")
        return False
    if not dh.set_params(n, a):
        return False

    while True:
        print("\n2. Введите секретные ключи A и B")
        try:
            K_A = int(input(f"  Секретный ключ A (2..{n - 1}): "))
            K_B = int(input(f"  Секретный ключ B (2..{n - 1}): "))
        except ValueError:
            print("  Ошибка: введите целые числа; повторите ввод ключей.")
            continue

        if not dh.set_secret_key_A(K_A) or not dh.set_secret_key_B(K_B):
            print("  Повторите ввод секретных ключей.")
```

```
            continue

        print("\n3. Вычислите открытые ключи")
        if not dh.compute_public_keys():
            print("  Повторите ввод секретных ключей A и B.")
            continue

        print("\n4. Вычислите общий секрет и проверьте протокол")
        if dh.compute_shared_secret():
            print("\n  Протокол успешно завершён.")
            return True
        print("\n  Обмен не принят. Введите другие секретные ключи A и B.")


def main():
    dh = DiffieHellman()

    while True:
        print_menu(dh)
        choice = input("  Ваш выбор: ").strip()

        if choice == "1":
            try:
                n = int(input("  Введите простое число n: "))
                a = int(input("  Введите основание a (1 < a < n): "))
                dh.set_params(n, a)
            except ValueError:
                print("  Ошибка: введите целые числа.")

        elif choice == "2":
            try:
                bits = int(input("  Битность n для учебной автогенерации (4..32) [24]: ").strip() or "24")
                dh.auto_generate_params(bits)
            except ValueError:
                print("  Ошибка: введите целое число.")

        elif choice == "3":
            if dh.n is None:
                print("  Ошибка: сначала установите параметры (п.1 или п.2).")
            else:
                try:
                    K_A = int(input(f"  Введите секретный ключ K_A (из [2, {dh.n - 1}]): "))
                    dh.set_secret_key_A(K_A)
                except ValueError:
                    print("  Ошибка: введите целое число.")

        elif choice == "4":
            if dh.n is None:
                print("  Ошибка: сначала установите параметры (п.1 или п.2).")
            else:
                try:
                    K_B = int(input(f"  Введите секретный ключ K_B (из [2, {dh.n - 1}]): "))
                    dh.set_secret_key_B(K_B)
                except ValueError:
                    print("  Ошибка: введите целое число.")

        elif choice == "5":
            dh.compute_public_keys()

        elif choice == "6":
            dh.compute_shared_secret()

        elif choice == "7":
            manual_protocol(dh)

        elif choice == "8":
            demo_textbook(dh)

        elif choice == "0":
            print("  Выход.")
            break

        else:
            print("  Неверный выбор.")

        input("\n  Нажмите Enter для продолжения...")


if __name__ == "__main__":
    main()
```

Перед вычислением общего секрета программа проверяет, не совпал ли любой открытый ключ с секретным ключом. Затем сверяет секреты A и B и отклоняет общий секрет, совпавший с Y_A или Y_B.

Результат работы программы:

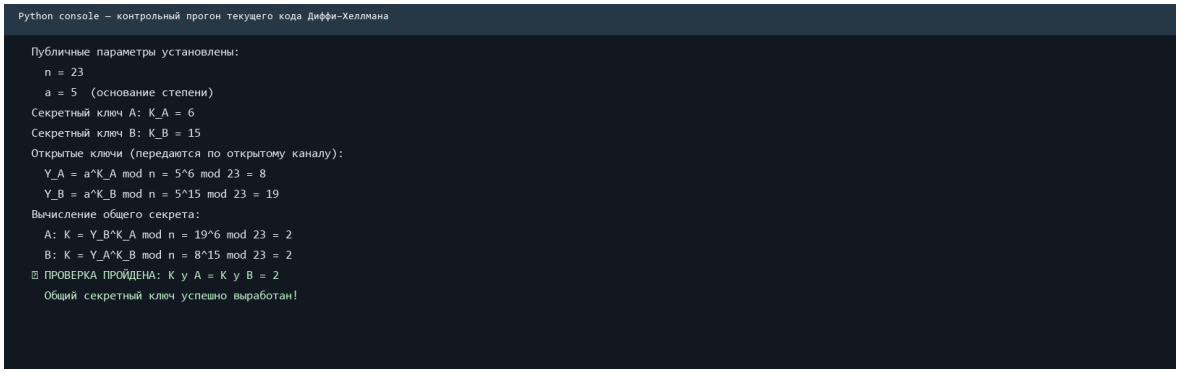


Рисунок 46. Контрольный прогон Диффи–Хеллмана по актуальному коду.

Работа в тетради:

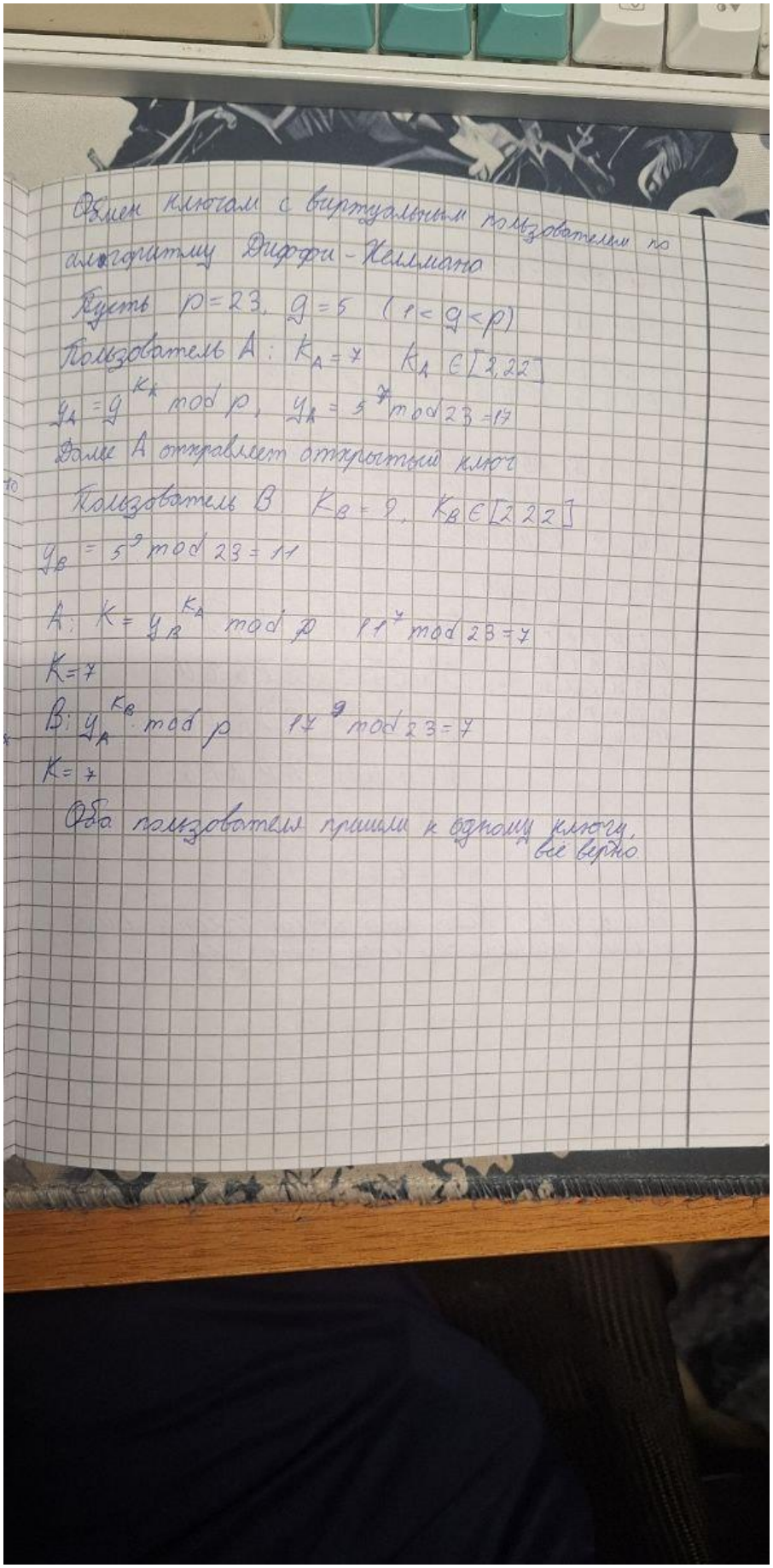


Рисунок 47. Обмен ключами в тетради.

Заключение

В работе рассмотрены криптографические алгоритмы из блоков А–К: от классических шифров до цифровых подписей и обмена ключами. Общий интерфейс позволяет запускать реализации из одного меню, а приведённые схемы, исходный код и результаты выполнения помогают проследить основные этапы каждого алгоритма.

Проверка обратных преобразований, контрольных примеров и результатов протоколов позволяет оценить корректность работы реализованных программ на приведённых входных данных. Актуальные версии алгоритмов МАГМА, Кузнечик и ГОСТ Р 34.10-2012 вынесены в приложение А.

Приложение А. Дополнительные реализации алгоритмов

В этом приложении приведены дополнительные версии программ МАГМА и Кузнечик. Основные программы, включённые в общий интерфейс, размещены в соответствующих разделах отчёта.

17. Шифр МАГМА

Файл программы: 17.MAGMA.py

```
# -*- coding: utf-8 -*-
"""
17. Шифр МАГМА (ГОСТ Р 34.12-2015)
Реализация блочного шифра «Магма» с режимом ECB.
Блок: 64 бита, ключ: 256 бит, 32 раунда (сеть Фейстеля).
Тестирование по ГОСТ Р 34.12-2018 (Приложение А).
Источник: ЛАБКРИПТ_2022, Блок G, задание 17.
ВВОД (как в методичке):
  - Открытый текст: произвольная строка (кодируется в UTF-8)
  - Ключ: 64 шестнадцатеричных символа (256 бит)
АЛГОРИТМ (по ГОСТ Р 34.12-2018 раздел 5):
  Блок a = a1|a0 (a1 — старшие 32 бит, a0 — младшие 32 бит).
  Раунд: G[k](a1,a0) = (a0, g[k](a0) XOR a1)
  Последний раунд: G*[k](a1,a0) = (g[k](a0) XOR a1, a0)
  Ключ K делится на K1..K8 (big-endian, 32 бит каждый).
  Расписание: K1..K8 × 3, затем K8..K1.
"""
# ──────────────────────────────────────────────
#  S-блоки ГОСТ Р 34.12-2015 (Приложение А)
# ──────────────────────────────────────────────
S_BOX = (
    (12, 4, 6, 2, 10, 5, 11, 9, 14, 8, 13, 7, 0, 3, 15, 1),
    (6, 8, 2, 3, 9, 10, 5, 12, 1, 14, 4, 7, 11, 13, 0, 15),
    (11, 3, 5, 8, 2, 15, 10, 13, 14, 1, 7, 4, 12, 9, 6, 0),
    (12, 8, 2, 1, 13, 4, 15, 6, 7, 0, 10, 5, 3, 14, 9, 11),
    (7, 15, 5, 10, 8, 1, 6, 13, 0, 9, 3, 14, 11, 4, 2, 12),
    (5, 13, 15, 6, 9, 2, 12, 10, 11, 7, 8, 1, 4, 3, 14, 0),
    (8, 14, 2, 5, 6, 9, 1, 12, 15, 4, 11, 0, 13, 10, 3, 7),
    (1, 7, 14, 13, 0, 5, 8, 3, 4, 15, 10, 6, 9, 12, 11, 2),
)
# ──────────────────────────────────────────────
#  Нелинейное преобразование T (8 × S-блоков)
# ──────────────────────────────────────────────
def t_direct(a: int) -> int:
    a &= 0xFFFFFFFF
    out = 0
    for i in range(8):
        nibble = (a >> (4 * i)) & 0xF
        out |= S_BOX[i][nibble] << (4 * i)
    return out & 0xFFFFFFFF
# ──────────────────────────────────────────────
#  Циклический сдвиг влево на 11 бит
# ──────────────────────────────────────────────
def rotl32(x: int, n: int) -> int:
    x &= 0xFFFFFFFF
    return ((x << n) | (x >> (32 - n))) & 0xFFFFFFFF
# ──────────────────────────────────────────────
#  Раундовая функция g[k](a) = rotl11(T(a + k mod 2^32))
# ──────────────────────────────────────────────
def g_round(k: int, a: int) -> int:
    x = (a + k) & 0xFFFFFFFF
    x = t_direct(x)
    return rotl32(x, 11)
# ──────────────────────────────────────────────
#  Расписание ключей
#  K = K1|K2|...|K8 (big-endian 32-бит слова)
#  Раунды 1-24: K1,K2,...,K8 повторить 3 раза
#  Раунды 25-32: K8,K7,...,K1
# ──────────────────────────────────────────────
def round_keys_from_key(key32: bytes) -> list:
    if len(key32) != 32:
        raise ValueError("Ключ должен быть 32 байта (256 бит)")
    K = [int.from_bytes(key32[4 * i: 4 * (i + 1)], "big") for i in range(8)]
    rk = []
    for i in range(32):
        if i < 24:
            rk.append(K[i % 8])
        else:
            rk.append(K[7 - (i - 24)])
    return rk
# ──────────────────────────────────────────────
#  Шифрование одного 64-битного блока (big-endian)
#  a = a1|a0  (a1 — старшие 32 бит, a0 — младшие 32 бит)
#  G[k](a1,a0)  = (a0,  g[k](a0) XOR a1)   — обычный раунд
#  G*[k](a1,a0) = (g[k](a0) XOR a1, a0)    — последний раунд
# ──────────────────────────────────────────────
```

```
def encrypt_block_magma(block: bytes, rk: list) -> bytes:
    a1 = int.from_bytes(block[0:4], "big")
    a0 = int.from_bytes(block[4:8], "big")
    for i in range(31):
        # G[rk[i]](a1, a0)
        a1, a0 = a0, (g_round(rk[i], a0) ^ a1) & 0xFFFFFFFF
    # G*[rk[31]](a1, a0) — без перестановки полублоков
    b1 = (g_round(rk[31], a0) ^ a1) & 0xFFFFFFFF
    b0 = a0
    return b1.to_bytes(4, "big") + b0.to_bytes(4, "big")

def decrypt_block_magma(block: bytes, rk: list) -> bytes:
    b1 = int.from_bytes(block[0:4], "big")
    b0 = int.from_bytes(block[4:8], "big")
    # Обратное G*[rk[31]]: a1 = g[k](b0) XOR b1, a0 = b0
    a1 = (g_round(rk[31], b0) ^ b1) & 0xFFFFFFFF
    a0 = b0
    # Обратные раунды 30..0
    for i in range(30, -1, -1):
        # Обратное G[rk[i]](a1, a0) = (old_a0, ...) = (a0 прежде свопа)
        # G дало: new_a1=a0_old, new_a0 = g[k](a0_old) XOR a1_old
        # Значит: a0_old = a1, a1_old = g[k](a1) XOR a0
        a1, a0 = (g_round(rk[i], a1) ^ a0) & 0xFFFFFFFF, a1
    return a1.to_bytes(4, "big") + a0.to_bytes(4, "big")

# ──────────────────────────────────────────────
#  ECB-режим с PKCS#7-дополнением (блок 8 байт)
# ──────────────────────────────────────────────
def pad_pkcs7(data: bytes, block_size: int) -> bytes:
    pad_len = block_size - (len(data) % block_size)
    return data + bytes([pad_len] * pad_len)

def unpad_pkcs7(data: bytes) -> bytes:
    if not data:
        raise ValueError("Нельзя снять дополнение с пустых данных.")
    pad_len = data[-1]
    if pad_len < 1 or pad_len > 8 or data[-pad_len:] != bytes([pad_len]) * pad_len:
        raise ValueError("Неверный ключ или повреждённый шифртекст: дополнение некорректно.")
    return data[:-pad_len]

def magma_ecb_encrypt(plaintext: bytes, key: bytes) -> bytes:
    rk = round_keys_from_key(key)
    padded = pad_pkcs7(plaintext, 8)
    ciphertext = b''
    for i in range(0, len(padded), 8):
        ciphertext += encrypt_block_magma(padded[i:i + 8], rk)
    return ciphertext

def magma_ecb_decrypt(ciphertext: bytes, key: bytes) -> bytes:
    rk = round_keys_from_key(key)
    plaintext = b''
    for i in range(0, len(ciphertext), 8):
        plaintext += decrypt_block_magma(ciphertext[i:i + 8], rk)
    return unpad_pkcs7(plaintext)

# ──────────────────────────────────────────────
#  Тест по ГОСТ Р 34.12-2018 (Приложение А)
# ──────────────────────────────────────────────
def test_gost_vectors():
    print("=" * 70)
    print("ТЕСТ МАГМА по ГОСТ Р 34.12-2018 (Приложение А)")
    print("=" * 70)
    key = bytes.fromhex(
        "ffeeddccbbaa99887766554433221100"
        "f0f1f2f3f4f5f6f7f8f9fafbfcfdfeff"
    )
    plaintext          = bytes.fromhex("fedcba9876543210")
    expected_ciphertext= bytes.fromhex("4ee901e5c2d8ca3d")
    rk         = round_keys_from_key(key)
    ciphertext = encrypt_block_magma(plaintext, rk)
    decrypted  = decrypt_block_magma(ciphertext, rk)
    print(f"Ключ (hex)        : {key.hex()}")
    print(f"Открытый текст    : {plaintext.hex()}")
    print(f"Ожидаемый шифр    : {expected_ciphertext.hex()}")
    print(f"Полученный шифр   : {ciphertext.hex()}")
    print(f"Тест шифрования   : {'ДА ✓' if ciphertext == expected_ciphertext else 'НЕТ ✗'}")
    print(f"Расшифрование     : {decrypted.hex()}")
    print(f"Тест расшифрования: {'ДА ✓' if decrypted == plaintext else 'НЕТ ✗'}")
    print("=" * 70)
    return ciphertext == expected_ciphertext and decrypted == plaintext

# ──────────────────────────────────────────────
#  Ввод ключа от пользователя
# ──────────────────────────────────────────────
def read_key_from_user() -> bytes:
    """Запрашивает 256-битный ключ в виде 64 hex-символов."""
    print("Ключ вводится как 64 шестнадцатеричных символа (256 бит).")
    print("Пример: ffeeddccbbaa99887766554433221100f0f1f2f3f4f5f6f7f8f9fafbfcfdfeff")
    key_hex = input("Введите ключ: ").strip().replace(" ", "")
```

```
    if len(key_hex) != 64:
        raise ValueError("Ключ должен содержать ровно 64 hex-символа (32 байта).")
    try:
        return bytes.fromhex(key_hex)
    except ValueError as exc:
        raise ValueError("Ключ должен состоять только из символов 0-9 и A-F.") from exc

# ──────────────────────────────────────────────
#  Интерактивные функции
# ──────────────────────────────────────────────
def interactive_encrypt_hex():
    """
    Зашифровать hex-блок (как в контрольном примере ГОСТ Р 34.12-2018).
    Ввод : hex-строка, кратная 16 символам (= 8 байт на блок).
    Пример: PT = fedcba9876543210  →  CT = 4ee901e5c2d8ca3d
    """
    pt_hex = input("Введите открытый текст в hex (кратно 16 символам): ").strip().replace(" ", "")
    if not pt_hex:
        raise ValueError("Блок не должен быть пустым.")
    if len(pt_hex) % 16 != 0:
        raise ValueError("Длина hex должна быть кратна 16 (8 байт на блок).")
    try:
        pt_bytes = bytes.fromhex(pt_hex)
    except ValueError as exc:
        raise ValueError("Допустимы только символы 0-9 и A-F.") from exc
    key = read_key_from_user()
    rk = round_keys_from_key(key)
    ciphertext = b''
    for i in range(0, len(pt_bytes), 8):
        ciphertext += encrypt_block_magma(pt_bytes[i:i + 8], rk)
    print(f"Шифртекст (hex): {ciphertext.hex()}")


def interactive_decrypt():
    """Расшифровать hex-шифртекст."""
    ciphertext_hex = input("Введите шифртекст в hex: ").strip().replace(" ", "")
    if not ciphertext_hex:
        raise ValueError("Шифртекст не должен быть пустым.")
    if len(ciphertext_hex) % 16 != 0:
        raise ValueError("Длина hex-шифртекста должна быть кратна 16 символам (8 байт на блок).")
    try:
        ciphertext = bytes.fromhex(ciphertext_hex)
    except ValueError as exc:
        raise ValueError("Шифртекст должен состоять только из символов 0-9 и A-F.") from exc
    key = read_key_from_user()
    plaintext = magma_ecb_decrypt(ciphertext, key)
    try:
        print(f"Открытый текст: {plaintext.decode('utf-8')}")
    except UnicodeDecodeError:
        print(f"Результат (hex): {plaintext.hex()}")
# ──────────────────────────────────────────────
#  Главное меню
# ──────────────────────────────────────────────
def interactive_menu():
    print("\n" + "=" * 70)
    print("ШИФР МАГМА (ГОСТ Р 34.12-2015)  —  Задание 17")
    print("Сеть Фейстеля, 32 раунда, блок 64 бит, ключ 256 бит")
    print("Источник: ЛАБКРИПТ_2022, Блок G, стр. 29")
    print("=" * 70)
    print()
    print("ВВОД:")
    print("  Открытый текст : hex-строка, кратная 16 символам (8 байт на блок)")
    print("  Ключ           : 64 шестнадцатеричных символа (256 бит)")
    print("  Пример ГОСТ    : PT=fedcba9876543210 → CT=4ee901e5c2d8ca3d")
    print()
    while True:
        print("1 — Зашифровать hex-блок")
        print("2 — Расшифровать hex-шифртекст")
        print("0 — Выход")
        choice = input("Выберите действие: ").strip()
        if choice == "0":
            break
        elif choice == "1":
            try:
                interactive_encrypt_hex()
            except (ValueError, UnicodeError) as exc:
                print(f"Ошибка ввода: {exc}")
        elif choice == "2":
            try:
                interactive_decrypt()
            except (ValueError, UnicodeError) as exc:
                print(f"Ошибка ввода: {exc}")
        else:
            print("Выберите 0, 1 или 2.")
        print()

if __name__ == "__main__":
    interactive_menu()
```

20. Шифр Кузнечик

Файл программы: 20.KUZNECHIK.py

```
# -*- coding: utf-8 -*-
"""
20. Шифр КУЗНЕЧИК (ГОСТ Р 34.12-2015)
Реализация блочного шифра «Кузнечик».
Блок: 128 бит, ключ: 256 бит, 10 раундов.
Тестирование по ГОСТ Р 34.12-2015 (Приложение А).
"""

PI = [
    252, 238, 221, 17, 207, 110, 49, 22, 251, 196, 250, 218, 35, 197, 4, 77,
    233, 119, 240, 219, 147, 46, 153, 186, 23, 54, 241, 187, 20, 205, 95, 193,
    249, 24, 101, 90, 226, 92, 239, 33, 129, 28, 60, 66, 139, 1, 142, 79,
    5, 132, 2, 174, 227, 106, 143, 160, 6, 11, 237, 152, 127, 212, 211, 31,
    235, 52, 44, 81, 234, 200, 72, 171, 242, 42, 104, 162, 253, 58, 206, 204,
    181, 112, 14, 86, 8, 12, 118, 18, 191, 114, 19, 71, 156, 183, 93, 135,
    21, 161, 150, 41, 16, 123, 154, 199, 243, 145, 120, 111, 157, 158, 178, 177,
    50, 117, 25, 61, 255, 53, 138, 126, 109, 84, 198, 128, 195, 189, 13, 87,
    223, 245, 36, 169, 62, 168, 67, 201, 215, 121, 214, 246, 124, 34, 185, 3,
    224, 15, 236, 222, 122, 148, 176, 188, 220, 232, 40, 80, 78, 51, 10, 74,
    167, 151, 96, 115, 30, 0, 98, 68, 26, 184, 56, 130, 100, 159, 38, 65,
    173, 69, 70, 146, 39, 94, 85, 47, 140, 163, 165, 125, 105, 213, 149, 59,
    7, 88, 179, 64, 134, 172, 29, 247, 48, 55, 107, 228, 136, 217, 231, 137,
    225, 27, 131, 73, 76, 63, 248, 254, 141, 83, 170, 144, 202, 216, 133, 97,
    32, 113, 103, 164, 45, 43, 9, 91, 203, 155, 37, 208, 190, 229, 108, 82,
    89, 166, 116, 210, 230, 244, 180, 192, 209, 102, 175, 194, 57, 75, 99, 182,
]
PI_INV = [0] * 256
for i in range(256):
    PI_INV[PI[i]] = i

L_VEC = [148, 32, 133, 16, 194, 192, 1, 251, 1, 192, 194, 16, 133, 32, 148, 1]

def gf_mul(a, b):
    p = 0
    hi_bit_set = 0
    for _ in range(8):
        if b & 1:
            p ^= a
        hi_bit_set = a & 0x80
        a <<= 1
        if hi_bit_set:
            a ^= 0x1c3
        a &= 0xff
        b >>= 1
    return p

def S(block):
    return bytes(PI[b] for b in block)

def S_inv(block):
    return bytes(PI_INV[b] for b in block)

def R(block):
    a = 0
    for i in range(16):
        a ^= gf_mul(block[i], L_VEC[i])
    return bytes([a]) + block[:15]

def R_inv(block):
    a = block[0]
    block = block[1:] + bytes([0])
    result = block[:-1] + bytes([0])
    new_last = 0
    for i in range(15):
        new_last ^= gf_mul(result[i], L_VEC[i])
    new_last ^= gf_mul(a, L_VEC[15])
    return result[:15] + bytes([new_last])

def L(block):
    for _ in range(16):
        block = R(block)
    return block

def L_inv(block):
    for _ in range(16):
        block = R_inv(block)
    return block

def F(k, a1, a0):
    tmp = bytes(a ^ b for a, b in zip(k, a1))
    tmp = L(S(tmp))
    return tmp, bytes(a ^ b for a, b in zip(tmp, a0))

def C(i):
    c = bytes([0] * 15 + [i])
    return L(c)
```

```
def key_expansion(key):
    k1 = key[:16]
    k2 = key[16:]
    round_keys = [k1, k2]
    a1, a0 = k1, k2
    for i in range(1, 5):
        for j in range(8):
            c = C(8 * (i - 1) + j + 1)
            a1, a0 = F(c, a1, a0)
        round_keys.extend([a1, a0])
    return round_keys

def encrypt_block(block, round_keys):
    state = block
    for i in range(9):
        state = bytes(a ^ b for a, b in zip(state, round_keys[i]))
        state = S(state)
        state = L(state)
    state = bytes(a ^ b for a, b in zip(state, round_keys[9]))
    return state

def decrypt_block(block, round_keys):
    state = block
    state = bytes(a ^ b for a, b in zip(state, round_keys[9]))
    for i in range(8, -1, -1):
        state = L_inv(state)
        state = S_inv(state)
        state = bytes(a ^ b for a, b in zip(state, round_keys[i]))
    return state

def pad_pkcs7(data: bytes, block_size: int) -> bytes:
    pad_len = block_size - (len(data) % block_size)
    return data + bytes([pad_len] * pad_len)

def unpad_pkcs7(data: bytes) -> bytes:
    if not data:
        raise ValueError("Нельзя снять дополнение с пустых данных.")
    pad_len = data[-1]
    if pad_len < 1 or pad_len > 16 or data[-pad_len:] != bytes([pad_len]) * pad_len:
        raise ValueError("Неверный ключ или повреждённый шифртекст: дополнение некорректно.")
    return data[:-pad_len]

def kuznechik_ecb_encrypt(plaintext: bytes, key: bytes) -> bytes:
    round_keys = key_expansion(key)
    padded = pad_pkcs7(plaintext, 16)
    ciphertext = b''
    for i in range(0, len(padded), 16):
        block = padded[i:i+16]
        ciphertext += encrypt_block(block, round_keys)
    return ciphertext

def kuznechik_ecb_decrypt(ciphertext: bytes, key: bytes) -> bytes:
    round_keys = key_expansion(key)
    plaintext = b''
    for i in range(0, len(ciphertext), 16):
        block = ciphertext[i:i+16]
        plaintext += decrypt_block(block, round_keys)
    return unpad_pkcs7(plaintext)

def test_gost_vectors():
    print("=" * 70)
    print("ТЕСТ КУЗНЕЧИК по ГОСТ Р 34.12-2015 (Приложение А)")
    print("=" * 70)
    key = bytes.fromhex(
        "8899aabbccddeeff0011223344556677"
        "fedcba98765432100123456789abcdef"
    )
    plaintext = bytes.fromhex("1122334455667700ffeeddccbbaa9988")
    expected_ciphertext = bytes.fromhex("7f679d90bebc24305a468d42b9d4edcd")
    round_keys = key_expansion(key)
    ciphertext = encrypt_block(plaintext, round_keys)
    print(f"Ключ (hex): {key.hex()}")
    print(f"Открытый текст: {plaintext.hex()}")
    print(f"Ожидаемый шифртекст: {expected_ciphertext.hex()}")
    print(f"Полученный шифртекст: {ciphertext.hex()}")
    enc_ok = ciphertext == expected_ciphertext
    print(f"Тест шифрования пройден: {'ДА' if enc_ok else 'НЕТ'}")
    decrypted = decrypt_block(ciphertext, round_keys)
    print(f"Расшифрованный текст: {decrypted.hex()}")
    dec_ok = decrypted == plaintext
    print(f"Расшифрование верно: {'ДА' if dec_ok else 'НЕТ'}")
    print("\nПроверка S-преобразования (A.1.1):")
    s_input = bytes.fromhex("ffeeddccbbaa99881122334455667700")
    s_expected = bytes.fromhex("b66cd8887d38e8d77765aeea0c9a7efc")
    s_result = S(s_input)
    print(f"S({s_input.hex()}) = {s_result.hex()}")
    print(f"Ожидается: {s_expected.hex()}")
    s_ok = s_result == s_expected
```

```
    print(f"Тест S пройден: {'ДА' if s_ok else 'НЕТ'}")
    print("\nПроверка R-преобразования (A.1.2):")
    r_input = bytes.fromhex("00000000000000000000000000000100")
    r_expected = bytes.fromhex("94000000000000000000000000000001")
    r_result = R(r_input)
    print(f"R({r_input.hex()}) = {r_result.hex()}")
    print(f"Ожидается: {r_expected.hex()}")
    r_ok = r_result == r_expected
    print(f"Тест R пройден: {'ДА' if r_ok else 'НЕТ'}")
    print("\nПроверка L-преобразования (A.1.3):")
    l_input = bytes.fromhex("64a59400000000000000000000000000")
    l_expected = bytes.fromhex("d456584dd0e3e84cc3166e4b7fa2890d")
    l_result = L(l_input)
    print(f"L({l_input.hex()}) = {l_result.hex()}")
    print(f"Ожидается: {l_expected.hex()}")
    l_ok = l_result == l_expected
    print(f"Тест L пройден: {'ДА' if l_ok else 'НЕТ'}")
    print("=" * 70)
    return enc_ok and dec_ok

def test_long_text():
    print("\n" + "=" * 70)
    print("ТЕСТ НА ТЕКСТЕ 1000+ СИМВОЛОВ")
    print("=" * 70)

    long_text = """Криптография — наука о методах обеспечения конфиденциальности,
целостности данных, аутентификации и невозможности отказа от авторства.
Шифр «Кузнечик» (ГОСТ Р 34.12-2015) является одним из стандартных
блочных шифров Российской Федерации. Он использует SP-сеть с 10 раундами,
размером блока 128 бит и ключом 256 бит.

Шифр «Кузнечик» был разработан для замены устаревшего шифра «Магма»
(ГОСТ 28147-89) и обеспечивает более высокий уровень безопасности
благодаря увеличенному размеру блока.

Основные преобразования шифра:
1. S-преобразование (нелинейная подстановка)
2. R-преобразование (линейный сдвиговый регистр)
3. L-преобразование (16-кратное применение R)
4. Сложение с раундовым ключом по модулю 2 (XOR)

Ключевое расписание шифра «Кузнечик» основано на сети Фейстеля
и использует константы C_i, вычисляемые как L([i]).

Данный алгоритм широко применяется для защиты конфиденциальной информации
в государственных и коммерческих системах Российской Федерации.

The Kuznyechik cipher is a symmetric block cipher standardized in
GOST R 34.12-2015. It operates on 128-bit blocks with a 256-bit key
and uses a substitution-permutation network structure with 10 rounds.
""" * 2

    print(f"Длина текста: {len(long_text)} символов")
    key = bytes.fromhex(
        "8899aabbccddeeff0011223344556677"
        "fedcba98765432100123456789abcdef"
    )
    plaintext_bytes = long_text.encode('utf-8')
    print(f"Длина в байтах: {len(plaintext_bytes)}")
    ciphertext = kuznechik_ecb_encrypt(plaintext_bytes, key)
    print(f"Длина шифртекста: {len(ciphertext)} байт")
    print(f"Первые 64 байта шифртекста (hex): {ciphertext[:64].hex()}")
    decrypted = kuznechik_ecb_decrypt(ciphertext, key)
    decrypted_text = decrypted.decode('utf-8')
    print(f"Расшифрование успешно: {'ДА' if decrypted_text == long_text else 'НЕТ'}")
    print("=" * 70)
    return decrypted_text == long_text

def demo_phrase():
    print("\n" + "=" * 70)
    print("ДЕМОНСТРАЦИЯ НА ФРАЗЕ ВАРИАНТА")
    print("=" * 70)
    phrase = "Леопард не может изменить своих пятен"
    print(f"Фраза: {phrase}")
    key = bytes.fromhex(
        "8899aabbccddeeff0011223344556677"
        "fedcba98765432100123456789abcdef"
    )
    plaintext_bytes = phrase.encode('utf-8')
    ciphertext = kuznechik_ecb_encrypt(plaintext_bytes, key)
    print(f"Шифртекст (hex): {ciphertext.hex()}")
    decrypted = kuznechik_ecb_decrypt(ciphertext, key)
    decrypted_text = decrypted.decode('utf-8')
    print(f"Расшифрованный текст: {decrypted_text}")
    print(f"Совпадение: {'ДА' if decrypted_text == phrase else 'НЕТ'}")
    print("=" * 70)

def read_key_from_user() -> bytes:
    """Запрашивает ключ Кузнечика длиной 256 бит в hex-формате."""
    key_hex = input(
        "Введите ключ (64 hex-символа, пример "
```

```
        "8899aabbccddeeff0011223344556677fedcba98765432100123456789abcdef): "
    ).strip().replace(" ", "")
    if len(key_hex) != 64:
        raise ValueError("Ключ должен содержать ровно 64 hex-символа (32 байта).")
    try:
        return bytes.fromhex(key_hex)
    except ValueError as exc:
        raise ValueError("Ключ должен состоять только из символов 0-9 и A-F.") from exc

def interactive_menu():
    print("Кузнечик использует ключ длиной 256 бит (32 байта).")
    print("Ключ вводится вручную как 64 шестнадцатеричных символа.")
    while True:
        print("\n1 - Зашифровать текст")
        print("2 - Расшифровать hex-шифртекст")
        print("3 - Показать тестовый пример ГОСТ")
        print("0 - Выход")
        choice = input("Выберите действие: ").strip()
        if choice == "0":
            break
        if choice == "3":
            test_gost_vectors()
            continue
        if choice not in ("1", "2"):
            print("Выберите 0, 1, 2 или 3.")
            continue
        try:
            if choice == "1":
                plaintext = input("Введите открытый текст: ")
                key = read_key_from_user()
                ciphertext = kuznechik_ecb_encrypt(plaintext.encode("utf-8"), key)
                print(f"Шифртекст (hex): {ciphertext.hex()}")
            else:
                ciphertext_hex = input("Введите шифртекст в hex: ").strip().replace(" ", "")
                if not ciphertext_hex:
                    raise ValueError("Шифртекст не должен быть пустым.")
                if len(ciphertext_hex) % 32 != 0:
                    raise ValueError("Длина hex-шифртекста должна быть кратна 32 символам (16 байт).")
                try:
                    ciphertext = bytes.fromhex(ciphertext_hex)
                except ValueError as exc:
                    raise ValueError("Шифртекст должен состоять только из символов 0-9 и A-F.") from exc
                key = read_key_from_user()
                plaintext = kuznechik_ecb_decrypt(ciphertext, key)
                try:
                    print(f"Открытый текст: {plaintext.decode('utf-8')}")
                except UnicodeDecodeError:
                    print(f"Результат (hex): {plaintext.hex()}")
        except ValueError as exc:
            print(f"Ошибка ввода: {exc}")

if __name__ == "__main__":
    print("\n" + "=" * 70)
    print("ШИФР КУЗНЕЧИК (ГОСТ Р 34.12-2015)")
    print("=" * 70)
    interactive_menu()
```

Приложение Б. Полная таблица расчёта A5/1 для слова ПЕРЕСДАЧА

Ниже приведена полная трассировка учебной реализации A5/1. Для каждого такта показаны состояния трёх регистров после шага, управляющие биты, большинство, обратная связь и решение о сдвиге. На этапах генерации также приведены бит гаммы, бит открытого текста и результат их XOR.

Параметр	Значение
Ключ в программе	АВГУСТ
Ключ UTF-8, первые 8 байт (hex)	D090D092D093D0A3
Номер кадра	155 (22 бита: 0000000000000010011011)
Открытый текст	ПЕРЕСДАЧА (UTF-8: 18 байт, 144 бит)
Гамма (hex)	D1842338C5696D732372388B7D231FE74044
Шифртекст (hex)	011BF3AD15C9BDE6F3D3E81FADB3CF4090D4

Этапы: 64 такта загрузки ключа, 22 такта загрузки кадра, 100 тактов холостого хода и 144 такта генерации. R1 содержит 19 бит, R2 — 22 бита, R3 — 23 бита. Индексы управляющих и обратной связи в таблице совпадают с индексами Python и начинаются с нуля.

A5/1 — таблица расчёта для слова «ПЕРЕСДАЧА»

Состояния R1–R3 указаны после такта; значения — двоичные строки. Индексы S и обратной связи в книге начинаются с 0.

Такт	Этап	Такты этапа	Бит входа	R1 после такта (19 бит)	R2 после такта (22 бита)	R3 после такта (23 бита)	S1 = R1[8]	S2 = R2[10]	S3 = R3[10]	F — большинство	Обратная связь R1	Обратная связь R2	Обратная связь R3	Сдвиг R1 (1/0)	Сдвиг R2 (1/0)	Сдвиг R3 (1/0)
0	Начальное состояние	0		0000000000000000000₂	0000000000000000000000₂	00000000000000000000000₂								0	0	0
1	Загрузка ключа	1	1	0100000000000000000₂	0100000000000000000000₂	01000000000000000000000₂	0	0	0		0	0	0	1	1	1
2	Загрузка ключа	2	1	0110000000000000000₂	0110000000000000000000₂	01100000000000000000000₂	0	0	0		0	0	0	1	1	1
3	Загрузка ключа	3	0	0011000000000000000₂	0011000000000000000000₂	00110000000000000000000₂	0	0	0		0	0	0	1	1	1
4	Загрузка ключа	4	1	0101100000000000000₂	0101100000000000000000₂	01011000000000000000000₂	0	0	0		0	0	0	1	1	1
5	Загрузка ключа	5	0	0010110000000000000₂	0010110000000000000000₂	00101100000000000000000₂	0	0	0		0	0	0	1	1	1
6	Загрузка ключа	6	0	0001011000000000000₂	0001011000000000000000₂	00010110000000000000000₂	0	0	0		0	0	0	1	1	1
7	Загрузка ключа	7	0	0000101100000000000₂	0000101100000000000000₂	00001011000000000000000₂	0	0	0		0	0	0	1	1	1
8	Загрузка ключа	8	0	0000010110000000000₂	0000010110000000000000₂	10000101100000000000000₂	0	0	0		0	0	1	1	1	1
9	Загрузка ключа	9	1	0100001011000000000₂	0100001011000000000000₂	10000010110000000000000₂	1	0	0		0	0	1	1	1	1
10	Загрузка ключа	10	0	0010000101100000000₂	0010000101100000000000₂	01000001011000000000000₂	1	0	0		0	0	0	1	1	1
11	Загрузка ключа	11	0	0001000010110000000₂	0001000010110000000000₂	10100000101100000000000₂	0	1	1		0	0	1	1	1	1
12	Загрузка ключа	12	1	0100100001011000000₂	0100100001011000000000₂	00010000010110000000000₂	1	1	1		0	0	0	1	1	1
13	Загрузка ключа	13	0	0010010000101100000₂	0010010000101100000000₂	00001000001011000000000₂	0	0	0		0	0	0	1	1	1
14	Загрузка ключа	14	0	1001001000010110000₂	0001001000010110000000₂	00000100000101100000000₂	0	1	1		1	0	0	1	1	1
15	Загрузка ключа	15	0	1100100100001011000₂	0000100100001011000000₂	00000010000010110000000₂	0	0	0		1	0	0	1	1	1
16	Загрузка ключа	16	0	0110010010000101100₂	0000010010000101100000₂	00000001000001011000000₂	0	0	0		0	0	0	1	1	1
17	Загрузка ключа	17	1	0111001001000010110₂	0100001001000010110000₂	11000000100000101100000₂	1	0	0		0	0	1	1	1	1
18	Загрузка ключа	18	1	0111110010000010111₂	0110000100100001011000₂	00100000010000010110000₂	0	0	0		0	0	0	1	1	1
19	Загрузка ключа	19	0	0011110010010000101₂	0011000010010000101100₂	00010000001000001011000₂	0	1	0		0	0	0	1	1	1
20	Загрузка ключа	20	1	0101111001001000010₂	0101100001001000010110₂	01001000000100000101100₂	1	0	1		0	0	0	1	1	1
21	Загрузка ключа	21	0	1010111100100100001₂	1010110000100100001011₂	10100100000010000010110₂	0	0	0		1	1	1	1	1	1
22	Загрузка ключа	22	0	0101011110010010000₂	0101011000010010000101₂	01010010000001000001011₂	0	1	0		0	0	0	1	1	1
23	Загрузка ключа	23	0	0010101111001001000₂	1010101100001001000010₂	00101001000000100000101₂	1	0	0		0	1	0	1	1	1
24	Загрузка ключа	24	0	0001010111100100100₂	1101010110000100100001₂	10010100100000010000010₂	1	0	0		0	1	1	1	1	1
25	Загрузка ключа	25	1	0100101011110010010₂	1010101011000010010000₂	10001010010000001000001₂	1	0	0		0	1	1	1	1	1
26	Загрузка ключа	26	0	1010010101111001001₂	0101010111000001001000₂	11000101001000000100000₂	1	0	0		1	0	1	1	1	1
27	Загрузка ключа	27	0	1101001010111100100₂	0010101010110000100100₂	11100010100100000010000₂	0	1	1		1	0	1	1	1	1
28	Загрузка ключа	28	1	0010100101011110010₂	0101010101011000010010₂	00110001010010000001000₂	1	1	0		0	0	0	1	1	1
29	Загрузка ключа	29	0	0001010010101111001₂	1010101010101100001001₂	10011000101001000000100₂	0	0	0		0	1	1	1	1	1

Такт	Этап	Такты этапа	Бит входа	R1 после такта (19 бит)	R2 после такта (22 бита)	R3 после такта (23 бита)	S1 = R1[8]	S2 = R2[10]	S3 = R3[10]	F — большинство	Обратная связь R1	Обратная связь R2	Обратная связь R3	Сдвиг R1 (1/0)	Сдвиг R2 (1/0)	Сдвиг R3 (1/0)
24	Загрузка ключа	24	0	0001010111100100100₂	1101010110000100100001₂	10010100100000010000010₂	1	0	0		0	1	1	1	1	1
25	Загрузка ключа	25	1	0100101011110010010₂	1010101011000010010000₂	10001010010000001000001₂	1	0	0		0	1	1	1	1	1
26	Загрузка ключа	26	0	1010010101111001001₂	0101010101100001001000₂	11000101001000000100000₂	1	0	0		1	0	1	1	1	1
27	Загрузка ключа	27	0	1101001010111100100₂	0010101010110000100100₂	11100010100100000010000₂	0	1	1		1	0	1	1	1	1
28	Загрузка ключа	28	1	0010100101011110010₂	0101010101011000010010₂	00110001010010000001000₂	1	1	0		0	0	0	1	1	1
29	Загрузка ключа	29	0	0001010010101111001₂	1010101010101100001001₂	10011000101001000000100₂	0	0	0		0	1	1	1	1	1
30	Загрузка ключа	30	0	0000101001010111100₂	1101010101010110000100₂	11001100010100100000010₂	1	1	1		0	1	1	1	1	1
31	Загрузка ключа	31	1	0100010100101011110₂	0010101010101011000010₂	10100110001010010000001₂	0	0	0		0	0	1	1	1	1
32	Загрузка ключа	32	0	0010001010010101111₂	1001010101010101100001₂	11010011000101001000000₂	0	1	1		0	1	1	1	1	1
33	Загрузка ключа	33	1	0101000101001010111₂	1000101010101010110000₂	10101001100010100100000₂	1	0	0		0	1	1	1	1	1
34	Загрузка ключа	34	1	1110100010100101011₂	0000010101010101011000₂	10010100110001010010000₂	0	1	0		1	0	1	1	1	1
35	Загрузка ключа	35	0	1111010001010010101₂	0000001010101010101100₂	01001010011000101001000₂	1	0	0		1	0	0	1	1	1
36	Загрузка ключа	36	1	0011101000101001010₂	0100000101010101010110₂	01100101001100010100100₂	0	1	1		0	0	0	1	1	1
37	Загрузка ключа	37	0	1001110100010100101₂	1010000010101010101011₂	00110010100110001010010₂	0	0	1		1	1	0	1	1	1
38	Загрузка ключа	38	0	1100111010001010010₂	0101000001010101010101₂	10011001010011000101001₂	0	1	0		1	0	1	1	1	1
39	Загрузка ключа	39	0	1110011101000101001₂	1010100000101010101010₂	01001100101001100010100₂	1	0	0		1	1	0	1	1	1
40	Загрузка ключа	40	0	0111001110100010100₂	1101010000010101010101₂	10100110010100110001010₂	0	1	1		0	1	1	1	1	1
41	Загрузка ключа	41	1	1111100111010001010₂	1010101000001010101010₂	10010011001010011000101₂	1	0	0		1	1	1	1	1	1
42	Загрузка ключа	42	0	1111110011101000101₂	1101010100000101010101₂	11001001100101001100010₂	1	0	1		1	1	1	1	1	1
43	Загрузка ключа	43	0	0111111001110100010₂	1110101010100001010101₂	01100100110010100110001₂	1	0	0		0	1	0	1	1	1
44	Загрузка ключа	44	1	0111111100111010001₂	1011010101000001010101₂	11110010011001010011000₂	0	0	0		0	1	1	1	1	1
45	Загрузка ключа	45	0	1011111110011101000₂	1101010101000001010101₂	01111001001100101001100₂	0	0	1		1	1	0	1	1	1
46	Загрузка ключа	46	0	1101111111001110100₂	1110110101010000010101₂	00111100100110010100110₂	1	1	1		1	1	0	1	1	1
47	Загрузка ключа	47	1	0010111111100111010₂	1011011010101000001010₂	01011110010011001010011₂	1	0	0		0	1	0	1	1	1
48	Загрузка ключа	48	1	0101011111110011101₂	1001101101010100000101₂	01101111001001100101001₂	1	1	0		0	1	0	1	1	1
49	Загрузка ключа	49	1	0110101111111001110₂	1000110110101010000010₂	01110111100100110010100₂	1	0	1		0	1	0	1	1	1
50	Загрузка ключа	50	1	0111010111111100111₂	1000011011010101000001₂	01111011110010011001010₂	1	1	0		0	1	0	1	1	1
51	Загрузка ключа	51	0	0011101011111110011₂	1100001101101010100000₂	00111101111001001100101₂	1	0	0		0	1	0	1	1	1
52	Загрузка ключа	52	1	1101110101111111001₂	0010000110110101010000₂	11011110111100100110010₂	1	1	1		1	0	1	1	1	1
53	Загрузка ключа	53	0	0110111010111111100₂	0001000011011010101000₂	11101111011110010011001₂	0	1	1		0	0	1	1	1	1

Такт	Этап	Такты этапа	Бит входа	R1 после такта (19 бит)	R2 после такта (22 бита)	R3 после такта (23 бита)	S1 = R1[8]	S2 = R2[10]	S3 = R3[10]	F — большинство	Обратная связь R1	Обратная связь R2	Обратная связь R3	Сдвиг R1 (1/0)	Сдвиг R2 (1/0)	Сдвиг R3 (1/0)
54	Загрузка ключа	54	0	0011011101011111110_2	0000100001101101010100_2	$01110111101111001001100_2$	1	0	1		0	0	0	1	1	1
55	Загрузка ключа	55	0	1001101110101111111_2	0000010000110110101010_2	$00111011110111100100110_2$	0	1	1		1	0	0	1	1	1
56	Загрузка ключа	56	0	0100110111010111111_2	1000001000011011010101_2	$10011101111011110010011_2$	1	1	0		0	1	1	1	1	1
57	Загрузка ключа	57	1	0110011011101011111_2	1000000100001101101010_2	$10001110111101111001001_2$	1	0	1		0	1	1	1	1	1
58	Загрузка ключа	58	0	1011001101110101111_2	1100000010000110110101_2	$11000111011111011100100_2$	1	0	1		1	1	1	1	1	1
59	Загрузка ключа	59	1	0001100110111010111_2	1010000001000011011010_2	$00100011101111011110010_2$	0	0	1		0	1	0	1	1	1
60	Загрузка ключа	60	0	1000110011011101011_2	1101000000100001101101_2	$00010001111011101111001_2$	1	0	1		1	1	0	1	1	1
61	Загрузка ключа	61	0	1100011001101110101_2	1110100000010000110110_2	$00001000111011111011100_2$	1	1	0		1	1	0	1	1	1
62	Загрузка ключа	62	0	1110001100110111010_2	1111010000001000011011_2	$10000100011101111011110_2$	0	0	1		1	1	1	1	1	1
63	Загрузка ключа	63	1	0011000110011101101_2	0011101000000100001101_2	$00000010001110111101111_2$	0	0	1		0	0	0	1	1	1
64	Загрузка ключа	64	1	0101100011001101110_2	1101110100000010000110_2	$11000001000111011110111_2$	1	0	1		0	1	1	1	1	1
65	Загрузка кадра	1	0	1010110001100110111_2	1110111010000001000011_2	$01100000100011101111011_2$	1	0	0		1	1	0	1	1	1
66	Загрузка кадра	2	0	0101011000110011011_2	0111011101000000100001_2	$00110000010001110111101_2$	0	0	0		0	0	0	1	1	1
67	Загрузка кадра	3	0	0010101100011001101_2	1011101110100000010000_2	$00011000001000111011110_2$	0	0	0		0	1	0	1	1	1
68	Загрузка кадра	4	0	0001010110001100110_2	0101110111010000001000_2	$00001100000100011101111_2$	0	1	1		0	0	0	1	1	1
69	Загрузка кадра	5	0	1000101011000110011_2	0010111011101000000100_2	$10000110000010001110111_2$	1	0	0		1	0	1	1	1	1
70	Загрузка кадра	6	0	1100010101100011001_2	0001011101110100000010_2	$11000011000001000111011_2$	1	1	0		1	0	1	1	1	1
71	Загрузка кадра	7	0	1110001010110001100_2	1000101110111010000001_2	$11100001100000100011101_2$	0	1	0		1	1	1	1	1	1
72	Загрузка кадра	8	0	1111000101011000110_2	1100010111011101000000_2	$11110000110000010001110_2$	1	1	0		1	1	1	1	1	1
73	Загрузка кадра	9	0	0111100010101100011_2	0110001011101110100000_2	$01111000011000001000111_2$	0	0	0		0	0	0	1	1	1
74	Загрузка кадра	10	0	1011110001010110001_2	0011000101110111010000_2	$10111100001100000100011_2$	1	1	1		1	0	1	1	1	1
75	Загрузка кадра	11	0	0101111000101011000_2	0001100010111011101000_2	$01011110000110000010001_2$	0	1	1		0	0	0	1	1	1
76	Загрузка кадра	12	0	0010111100010101100_2	0000110001011101110100_2	$10101111000011000001000_2$	0	1	0		0	0	1	1	1	1
77	Загрузка кадра	13	0	0001011110001010110_2	0000011000101110111010_2	$11010111100001100000100_2$	0	0	0		0	0	1	1	1	1
78	Загрузка кадра	14	0	0000101111000101011_2	1000001100010111011101_2	$01101011110000110000010_2$	1	1	0		0	1	0	1	1	1
79	Загрузка кадра	15	1	1100010111100010101_2	1000000110001011101110_2	$01110101111000011000001_2$	1	0	0		1	1	0	1	1	1
80	Загрузка кадра	16	0	0110001011110001010_2	1100000011000101110111_2	$00111010111100001100000_2$	1	0	1		0	1	0	1	1	1
81	Загрузка кадра	17	0	1011000101111000101_2	0110000001100010111011_2	$00011101011110000110000_2$	1	0	1		1	0	0	1	1	1
82	Загрузка кадра	18	1	0001100010111100010_2	0111000001100010111101_2	$11001110101111000011000_2$	0	1	1		0	0	1	1	1	1
83	Загрузка кадра	19	1	0100110001011110001_2	1111100000011000101110_2	$00100111010111100001100_2$	1	1	1		0	1	0	1	1	1

Такт	Этап	Такты этапа	Бит входа	R1 после такта (19 бит)	R2 после такта (22 бита)	R3 после такта (23 бита)	S1 = R1[8]	S2 = R2[10]	S3 = R3[10]	F — большинство	Обратная связь R1	Обратная связь R2	Обратная связь R3	Сдвиг R1 (1/0)	Сдвиг R2 (1/0)	Сдвиг R3 (1/0)
84	Загрузка кадра	20	0	0010011000101111000_2	1111110000001100010111_2	$00010011101011110000110_2$	0	0	0		0	1	0	1	1	1
85	Загрузка кадра	21	1	1101001100010111100_2	0011111000000110001011_2	$11001001110101111000011_2$	0	0	1		1	0	1	1	1	1
86	Загрузка кадра	22	1	0010100110001011110_2	0101111100000011000101_2	$10100100111010111100001_2$	0	0	0		0	0	1	1	1	1
87	Холостой ход	1		0001010011000101111_2	0101111100000011000101_2	$11010010011101011110000_2$	1	0	1	1	0	1	1	1	0	1
88	Холостой ход	2		0000101001100010111_2	0101111100000011000101_2	$01101001001110101111000_2$	1	0	1	1	0	1	0	1	0	1
89	Холостой ход	3		1000010100110001011_2	1010111110000001100010_2	$01101001001110101111000_2$	0	0	1	0	1	1	1	1	1	0
90	Холостой ход	4		0100001010011000101_2	1101011111000000110001_2	$01101001001110101111000_2$	0	0	1	0	0	1	1	1	1	0
91	Холостой ход	5		0010000101001100010_2	1101011111000000110001_2	$10110100100111010111100_2$	1	0	1	1	0	1	1	1	0	1
92	Холостой ход	6		0001000010100110001_2	1110101111100000011000_2	$11011010010011101011110_2$	0	0	0	0	0	1	1	1	1	1
93	Холостой ход	7		0000100001010011000_2	0111010111110000001100_2	$11011010010011101011110_2$	1	1	0	1	0	0	0	1	1	0
94	Холостой ход	8		0000010000101001100_2	0111010111110000001100_2	$01101101001001110101111_2$	0	1	0	0	0	0	0	1	0	1
95	Холостой ход	9		0000010000101001100_2	0011101011111000000110_2	$00110110100100111010111_2$	0	1	1	1	1	0	0	0	1	1
96	Холостой ход	10		1000001000010100110_2	0011101011111000000110_2	$10011011010010011101011_2$	0	1	0	0	1	1	1	1	0	1
97	Холостой ход	11		1100000100001010011_2	0011101011111000000110_2	$11001101101001001110101_2$	0	1	0	0	1	1	1	1	0	1
98	Холостой ход	12		1100000100001010011_2	1001110101111100000011_2	$11100110110100100111010_2$	0	1	1	1	0	1	1	0	1	1
99	Холостой ход	13		0110000010000101001_2	1001110101111100000011_2	$11110011011010010011101_2$	0	1	0	0	0	0	1	1	0	1
100	Холостой ход	14		0011000001000010100_2	0100111010111110000001_2	$11111001101101001001110_2$	1	1	1	1	0	0	1	1	1	1
101	Холостой ход	15		0011000001000010100_2	1010011101011111000000_2	$11111100110110100100111_2$	0	1	1	1	1	1	1	0	1	1
102	Холостой ход	16		1001100000100001010_2	0101001110101111100000_2	$11111100110110101001011_2$	0	0	0	0	1	0	1	1	1	1
103	Холостой ход	17		1001100000100001010_2	0010100111010111110000_2	$01111111001101101001001_2$	0	1	1	1	1	0	0	0	1	1
104	Холостой ход	18		1100110000010000101_2	0001010011101011111000_2	$01111111001101101001001_2$	0	0	1	0	1	0	0	1	1	0
105	Холостой ход	19		1100110000010000101_2	0000101001110101111100_2	$00111111100110110100100_2$	0	1	1	1	0	0	0	0	1	1
106	Холостой ход	20		0110011000001000010_2	0000101001110101111100_2	$00011111110011011010010_2$	0	1	0	0	0	0	0	1	0	1
107	Холостой ход	21		1011001100000100001_2	0000101001110101111100_2	$00001111111001101101001_2$	0	1	0	0	1	0	0	1	0	1
108	Холостой ход	22		1011001100000100001_2	0000010100111010111110_2	$00000111111110011011010_2$	0	1	1	1	0	0	0	0	1	1
109	Холостой ход	23		1011001100000100001_2	1000001010011101011111_2	$00000011111110011011010_2$	0	1	1	1	0	1	0	0	1	1
110	Холостой ход	24		0101100110000010000_2	0100000101001110101111_2	$00000011111110011011010_2$	0	0	1	0	0	0	0	1	1	0
111	Холостой ход	25		0010110011000001000_2	0100000101001110101111_2	$00000001111111001101101_2$	1	0	1	1	0	0	0	1	0	1
112	Холостой ход	26		0001011001100000100_2	0100000101001110101111_2	$10000000111111100110110_2$	1	0	1	1	0	0	1	1	0	1
113	Холостой ход	27		1000101100110000010_2	0010000010100111010111_2	$10000000111111100110110_2$	0	0	1	0	1	0	0	1	1	0

Такт	Этап	Такты этапа	Бит входа	R1 после такта (19 бит)	R2 после такта (22 бита)	R3 после такта (23 бита)	S1 = R1[8]	S2 = R2[10]	S3 = R3[10]	F — большинство	Обратная связь R1	Обратная связь R2	Обратная связь R3	Сдвиг R1 (1/0)	Сдвиг R2 (1/0)	Сдвиг R3 (1/0)
114	Холостой ход	28		1000101100110000010_2	0001000001010011101011_2	$01000000011111110011011_2$	0	1	1	1	1	0	0	0	1	1
115	Холостой ход	29		1100010110011000001_2	0000100000101001110101_2	$01000000011111110011011_2$	0	0	1	0	1	0	0	1	1	0
116	Холостой ход	30		1110001011001100000_2	1000010000010100111010_2	$00100000001111111001101_2$	1	1	1	1	1	1	0	1	1	1
117	Холостой ход	31		1111000101100110000_2	1000010000010100111010_2	$00010000000111111100110_2$	1	0	1	1	1	1	0	1	0	1
118	Холостой ход	32		1111100010110011000_2	1100001000001010011101_2	$00001000000011111110011_2$	0	0	0	0	1	1	0	1	1	1
119	Холостой ход	33		1111100010110011000_2	1110000100000101001110_2	$00000100000001111111001_2$	1	0	0	0	0	1	0	0	1	1
120	Холостой ход	34		1111100010110011000_2	1111000010000010100111_2	$10000010000000111111100_2$	1	0	0	0	0	1	1	0	1	1
121	Холостой ход	35		1111100010110011000_2	0111100001000001010011_2	$11000001000000111111110_2$	1	0	0	0	0	0	1	0	1	1
122	Холостой ход	36		1111100010110011000_2	0011110000100000101001_2	$11100000100000011111111_2$	1	0	0	0	0	0	1	0	1	1
123	Холостой ход	37		0111110001011001100_2	1001111000010000010100_2	$11100000100000011111111_2$	1	1	0	1	0	1	1	1	1	0
124	Холостой ход	38		1011111000101100110_2	0100111100001000001010_2	$11110000010000000111111_2$	0	0	0	0	1	0	1	1	1	1
125	Холостой ход	39		1101111100010110011_2	1010011110000100000101_2	$11111000001000000011111_2$	0	0	0	0	1	1	1	1	1	1
126	Холостой ход	40		1110111110001011001_2	1101001111000010000010_2	$11111000001000000011111_2$	0	0	1	0	1	1	1	1	1	0
127	Холостой ход	41		1111011111000101100_2	1101001111000010000010_2	$11111100000100000001111_2$	1	0	1	1	1	1	1	1	0	1
128	Холостой ход	42		1111011111000101100_2	1110100111100001000001_2	$11111110000010000000111_2$	1	0	0	0	0	1	1	0	1	1
129	Холостой ход	43		0111101111100010110_2	1111010011110000100000_2	$11111110000010000000111_2$	1	1	0	1	0	1	1	1	1	0
130	Холостой ход	44		0011110111110001011_2	0111101001111000010000_2	$11111110000010000000111_2$	1	1	0	1	0	0	1	1	1	0
131	Холостой ход	45		0001111011111000101_2	0011110100111100001000_2	$11111110000010000000111_2$	1	1	0	1	0	0	1	1	1	0
132	Холостой ход	46		0000111101111100010_2	0001111010011110000100_2	$11111110000010000000111_2$	1	1	0	1	0	0	1	1	1	0
133	Холостой ход	47		0000011110111110001_2	0000111101001111000010_2	$11111111000001000000011_2$	0	0	0	0	0	0	1	1	1	1
134	Холостой ход	48		0000011110111110001_2	1000011110100111100001_2	$11111111100000100000001_2$	1	0	0	0	0	1	1	0	1	1
135	Холостой ход	49		0000001111011111000_2	1100001111010011110000_2	$11111111100000100000001_2$	1	1	0	1	0	1	0	1	1	0
136	Холостой ход	50		0000001111011111000_2	0110000111010011110000_2	$01111111100000100000000_2$	1	0	0	0	1	0	0	0	1	1
137	Холостой ход	51		1000000111101111000_2	0011000011110100111100_2	$01111111100000100000000_2$	1	1	0	1	1	0	1	1	1	0
138	Холостой ход	52		0100000011110111110_2	0001100001111010011110_2	$01111111100000100000000_2$	1	1	0	1	0	0	1	1	1	0
139	Холостой ход	53		1010000001110111111_2	1000110000111101001111_2	$01111111100000100000000_2$	1	1	0	1	1	1	1	1	1	0
140	Холостой ход	54		1101000000111101111_2	1000110000111101001111_2	$10111111111000001000000_2$	0	1	0	0	1	0	1	1	0	1
141	Холостой ход	55		1101000000111101111_2	0100011000011110100111_2	$11011111111100000100000_2$	0	1	1	1	0	0	1	0	1	1
142	Холостой ход	56		0110100000001111011_2	0001000110000111101011_2	$11011111111100000100000_2$	0	0	1	0	0	0	1	1	1	0
143	Холостой ход	57		0011010000001111011_2	0001000110000111101001_2	$11011111111100000100000_2$	0	0	1	0	0	0	1	1	1	0

Такт	Этап	Такты этапа	Бит входа	R1 после такта (19 бит)	R2 после такта (22 бита)	R3 после такта (23 бита)	S1 = R1[8]	S2 = R2[10]	S3 = R3[10]	F — большинство	Обратная связь R1	Обратная связь R2	Обратная связь R3	Сдвиг R1 (1/0)	Сдвиг R2 (1/0)	Сдвиг R3 (1/0)
144	Холостой ход	58		1001101000000111101_2	1000100011000011110100_2	$11011111111100000100000_2$	0	0	1	0	1	1	1	1	1	0
145	Холостой ход	59		1100110100000011110_2	0100010001100001111010_2	$11011111111100000100000_2$	0	0	1	0	1	0	1	1	1	0
146	Холостой ход	60		1100110100000011110_2	1010001000110000111101_2	$11101111111100000100000_2$	0	1	1	1	0	1	1	0	1	1
147	Холостой ход	61		1100110100000011110_2	1101000100011000011110_2	$11110111111111000001000_2$	0	1	1	1	0	1	1	0	1	1
148	Холостой ход	62		0110011010000001111_2	1110100010001100001111_2	$11110111111111000001000_2$	0	0	1	0	0	1	1	1	1	0
149	Холостой ход	63		1011001101000000111_2	1110100010001100001111_2	$11111011111111000000100_2$	1	0	1	1	1	0	1	1	0	1
150	Холостой ход	64		1101100110100000011_2	0111010001000110000111_2	$11111011111111000000100_2$	0	0	1	0	1	0	0	1	1	0
151	Холостой ход	65		0110110011010000001_2	0111010001000110000111_2	$01111011111111000000010_2$	1	0	1	1	0	0	0	1	0	1
152	Холостой ход	66		1011011001101000000_2	0111010001000110000111_2	$00111101111111111000001_2$	1	0	1	1	1	0	0	1	0	1
153	Холостой ход	67		0101101100110100000_2	0011101000100011000011_2	$00111101111111111000001_2$	0	0	1	0	0	0	1	1	1	0
154	Холостой ход	68		0101101100110100000_2	0001110100010001100001_2	$10011111011111111100000_2$	0	1	1	1	1	0	1	0	1	1
155	Холостой ход	69		1010110110011010000_2	1000111010001000110000_2	$10011111011111111100000_2$	0	0	1	0	1	1	1	1	1	0
156	Холостой ход	70		0101011011001101000_2	1000111010001000110000_2	$11001111101111111110000_2$	1	0	1	1	0	0	1	1	0	1
157	Холостой ход	71		1010101101100110100_2	1000111010001000110000_2	$11100111110111111111000_2$	1	0	1	1	1	0	1	1	0	1
158	Холостой ход	72		0101010110110011010_2	0100011101000100011000_2	$11110011111011111111100_2$	0	0	0	0	0	0	1	1	1	1
159	Холостой ход	73		1010101011011001101_2	0100011101000100011000_2	$01111001111101111111110_2$	1	0	1	1	1	0	0	1	0	1
160	Холостой ход	74		0101010101101100110_2	0100011101000100011000_2	$10111100111101111111111_2$	1	0	1	1	0	0	1	1	0	1
161	Холостой ход	75		1010101010110110011_2	0010001110100010001100_2	$10111100111101111111111_2$	0	0	1	0	1	0	1	1	1	0
162	Холостой ход	76		1101010101011011001_2	0001000111010001000110_2	$11011110011111011111111_2$	1	1	1	1	1	0	1	1	1	1
163	Холостой ход	77		1110101010101101100_2	1000100011101000100011_2	$11011110011111011111111_2$	0	0	1	0	1	1	1	1	1	0
164	Холостой ход	78		0111010101010110110_2	0100010001110100010001_2	$11101110011111011111111_2$	1	1	1	1	0	0	1	1	1	1
165	Холостой ход	79		0111010101010110110_2	1010001000111010001000_2	$01110111100111101111111_2$	0	1	1	1	1	1	0	0	1	1
166	Холостой ход	80		1011101010101011011_2	1010001000111010001000_2	$00111011110011111011111_2$	0	1	0	0	1	0	0	1	0	1
167	Холостой ход	81		0101110101010101101_2	0101000100011101000100_2	$00111011110011111011111_2$	1	1	0	1	0	0	0	1	1	0
168	Холостой ход	82		1010111010101010110_2	0010100010001110100010_2	$00011101111001111101111_2$	0	0	0	0	1	0	0	1	1	1
169	Холостой ход	83		0101011101010101011_2	0010100010001110100010_2	$00001110111100111110111_2$	1	0	1	1	0	1	0	1	0	1
170	Холостой ход	84		1010101110101010101_2	1001010001000111010001_2	$00001110111100111110111_2$	0	0	1	0	1	1	1	1	1	0
171	Холостой ход	85		0101010111010101010_2	1001010001000111010001_2	$10000111011110011111011_2$	1	0	1	1	0	1	1	1	0	1
172	Холостой ход	86		0010101011101010101_2	1001010001000111010001_2	$11000011101111001111101_2$	1	0	1	1	0	1	1	1	0	1

Такт	Этап	Такты этапа	Бит входа	R1 после такта (19 бит)	R2 после такта (22 бита)	R3 после такта (23 бита)	S1 = R1[8]	S2 = R2[10]	S3 = R3[10]	F — большинство	Обратная связь R1	Обратная связь R2	Обратная связь R3	Сдвиг R1 (1/0)	Сдвиг R2 (1/0)	Сдвиг R3 (1/0)	Выходной бит гаммы	Бит текста	Шифробит = гамма XOR
174	Холостой ход	88		0000101010111010101_2	1100101000100011101000_2	$11110000111011110011111_2$	0	0	0	0	0	1	1	1	1	1			
175	Холостой ход	89		0000010101011101010_2	0110010100010001110100_2	$11111000011101111001111_2$	1	1	1	1	0	0	1	1	1	1			
176	Холостой ход	90		0000001010101110101_2	0011001010001000111010_2	$11111000011101111001111_2$	0	0	1	0	0	0	1	1	1	0			
177	Холостой ход	91		1000000101010111010_2	0011001010001000111010_2	$11111100001110111100111_2$	1	0	1	1	1	1	1	1	0	1			
178	Холостой ход	92		0100000010101011101_2	1001100101000100011101_2	$11111100001110111100111_2$	0	0	1	0	0	1	1	1	1	0			
179	Холостой ход	93		0010000010101011110_2	1001100101000100011101_2	$11111110000111011110011_2$	1	0	1	1	0	1	1	1	0	1			
180	Холостой ход	94		1001000000101010111_2	1100110010100010001110_2	$01111111000011101111001_2$	0	0	0	0	1	1	0	1	1	1			
181	Холостой ход	95		1100100000010101011_2	1100110010100010001110_2	$00111111100001110111100_2$	0	1	0	0	1	1	0	1	0	1			
182	Холостой ход	96		1110010000001010101_2	1100110010100010001110_2	$00011111110000111011110_2$	0	1	0	0	1	1	0	1	0	1			
183	Холостой ход	97		0111001000000101010_2	1100110010100010001110_2	$10001111111000011101111_2$	0	1	0	0	0	1	1	1	0	1			
184	Холостой ход	98		0111001000000101010_2	1110011001010001000111_2	$01000111111100001110111_2$	0	1	1	1	0	1	0	0	1	1			
185	Холостой ход	99		0011100100000010101_2	0111001100101000100011_2	$01000111111100001110111_2$	0	0	1	0	0	0	0	1	1	0			
186	Холостой ход	100		0011100100000010101_2	0011100110010100010001_2	$00100011111100001110111_2$	0	1	1	1	0	0	0	0	1	1			